

Gestion centralisée de la sécurité avec KeyCloak

David THIBAU – 2026

david.thibau@gmail.com

Keycloak - Identity and Access Management for Modern Applications
Stian Thorgersen | Pedro Igor Silva

Agenda

J1 – Introduction

- Fonctionnalités, installation, UIs Admin et Utilisateur, 1ère application

J1 – Rappels sur les standards

- OAuth2, OpenID Connect, JWT, types de jetons

J1 – Keycloak et OpenID Connect

- Discovery, authentification, ID token, UserInfo, logout

J2 – Autorisation avec OAuth2

- Consentement, limitation des accès, personnalisation et validation du jeton

J2 – Types d'application

- Web, native ou mobile, services back-end

J2 – Intégration Keycloak

- Librairies, Spring Boot, Quarkus, Python, reverse proxy, débogage

J3 – Stratégies d'autorisation

- RBAC, GBAC, scopes OAuth2, Authorization Service

J3 – Exploitation courante

Utilisateurs, mots de passe et MFA, sessions et jetons, événements et audit

J3 – Adaptation et personnalisation

Organisation, fédération d'identités, flows, thèmes, SPI

J3 – Automatisation et CI/CD

Admin API, kcadm, scripts, configuration as code, GitOps

J4 – Architecture et haute disponibilité

Composants, cycle de vie, configuration, image optimisée, clustering

J5 – Durcissement et exploitation du cluster

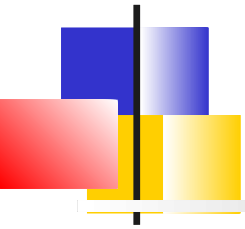
Performances, secrets, protections, nœuds, montées de version

J5 – Supervision et sauvegarde

Métriques, Prometheus et Grafana, logs, sauvegarde, audit

Annexes

FAPI, mTLS et CIBA, RFC 9700, DPoP, FIPS



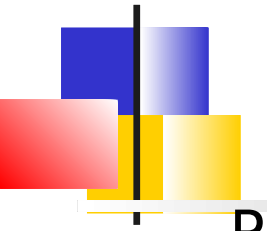
Introduction

Fonctionnalités et installation

Interfaces Admin et utilisateur

Sécuriser une 1ère application

Keycloak



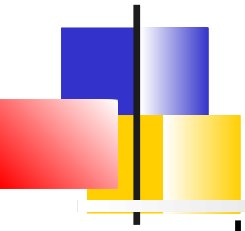
Produit OpenSource pour la gestion d'identité et des accès (IAM) dédié aux applications modernes (SPA, Mobile, API Web) :
=> Les applications n'ont plus besoin d'implémenter l'authentification ni le stockage de mot de passe
=> On obtient du SSO à l'intérieur de l'entreprise

Projet démarré en 2014, largement déployé en entreprise depuis

- Hautement personnalisable, Support pour l'authentification forte, application de stratégies pour les comptes utilisateur
- S'appuie sur les standards oAuth2.0, OpenIDConnect et SAML
- S'intègre avec des fournisseurs d'identité tierces : LDAP, OpenID Porvider, ...

Détails fonctionnalités

- ✔ **Authentification** : SSO et logout centralisé, MFA (OTP, WebAuthn / passkeys), flows configurables
- ✔ **Gestion des comptes** : inscription, mot de passe oublié, actions requises (CGU, vérification email, mise à jour du profil), console du compte
- ✔ **Fédération d'identités** : annuaires LDAP / AD, fournisseurs OIDC, SAML et sociaux
- ✔ **Autorisation** : rôles, groupes, scopes, Authorization Services
- ✔ **Administration** : console, API REST, CLI, événements utilisateur et admin
- ✔ **Personnalisation** : thèmes, internationalisation, extensions SPI
- ✔ **Exploitation** : clustering, métriques, déploiement conteneur et Kubernetes
- ✔ Les applications délèguent l'authentification à Keycloak et ne manipulent jamais les mots de passe : elles reçoivent des jetons standards



Protocoles supportés

Keycloak implémente les principaux standards d'identité :

- **OpenID Connect** : couche d'authentification au-dessus d'OAuth 2.0, jetons JWT
- **OAuth 2.0** : délégation d'autorisation, obtention de jetons d'accès
- **SAML 2.0** : assertions XML signées, historique des SSO d'entreprise

Chaque client est déclaré avec un protocole : openid-connect ou saml

Keycloak joue le rôle de fournisseur (OP OIDC ou IdP SAML) et peut aussi fédérer des fournisseurs externes des 2 types

SAML ou OpenID Connect ?

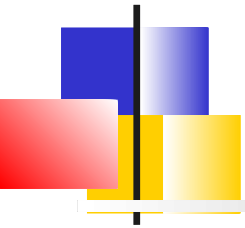
OpenID Connect

- Format JSON / REST, jetons JWT compacts
- Adapté aux SPA, applications mobiles, API et micro-services
- Choix recommandé pour les nouvelles applications

SAML 2.0

- Format XML, échanges via le navigateur (bindings Redirect et POST)
- Pas de notion de jeton d’accès pour appeler des API
- À conserver pour les applications existantes ou progiciels ne supportant que SAML

Stack technologique



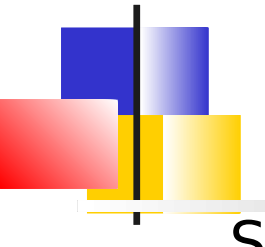
Avant Fin 2020 : Application web déployée sur le serveur Wildfly

Après Fin 2020 : Construit au-dessus du framework Quarkus[1] :

- Temps de démarrage réduit
- Faible empreinte mémoire
- Approche conteneur
- Meilleure expérience de développeur
- Meilleure convivialité

1. Une pile native Kubernetes et Cloud avec les meilleures bibliothèques et standards Java

Architecture et principes de fonctionnement



Serveur Java construit sur Quarkus, déployable en cluster

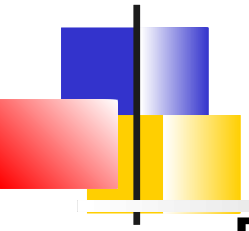
Persistance dans une base relationnelle : PostgreSQL, MySQL, MariaDB, Oracle, SQL Server

- Realms, clients, utilisateurs, rôles, configuration, sessions

Caches Infinispan : données de configuration et sessions, répliqués entre les nœuds

Principe : l'application délègue l'authentification à Keycloak

- L'utilisateur saisit ses identifiants uniquement sur Keycloak
- L'application reçoit des jetons signés par les clés du realm
- Les services vérifient localement la signature des jetons



Installation

- Différentes options pour l’installation :
  - Installation locale via une JVM
  - Démarrage de conteneur *Docker* ou *Podman*
  - Déploiement sur Kubernetes ou OpenShift
    avec en option l’utilisation de *Keycloak Kubernetes Operator* qui facilite l’installation, la configuration et l’exploitation



Répertoires de la distribution

bin/ : scripts *kc.sh / kc.bat* (start, start-dev, build, export, import, show-config), *kcadm.sh, kcreg.sh*

conf/ : *keycloak.conf*, configuration du cache (cache-ispn.xml)

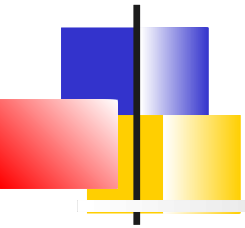
data/ : base H2 du mode dev, fichiers temporaires

- data/import : realms importés au démarrage avec --import-realm

lib/ : librairies du serveur, ne pas modifier

providers/ : extensions (SPI) sous forme de jar

themes/ : thèmes personnalisés



Introduction

Fonctionnalités et installation

Interfaces Admin et utilisateur

Sécuriser une 1ère application

Console d’administration

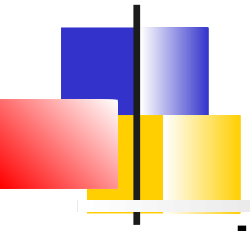
La console d’administration permet de gérer des **realms** :

- Ensemble d’utilisateurs
  - Rôles, groupes, sessions
- Ensemble d’applications clientes
  - Mécanismes d’obtention de jetons : Grant flows
  - Scopes : Accès aux données utilisateur

Un realm est complètement isolé des autres realms

- Par exemple, un realm pour les applications internes et les employés, et un autre pour les applications externes

Le realm *master* détermine la sécurité du serveur KeyCloak



Création de realm

- Le formulaire de création de realm ne demande qu'un nom
- Le nom est utilisé dans les URLs (Pas d'espace ni de caractères spéciaux)
  - Une nom d'affichage peut également être précisé
- Un *realm* peut s'exporter/s'importer via le format JSON

Création d'utilisateur

Après la création de *realm*, on crée des **utilisateurs** :

- *username* <=> login
- email, nom et prénom
- Actions requises au premier login : Vérification de son profil et de son email par exemple

On peut ensuite compléter les attributs keycloak par des attributs personnalisés

Avant que l'utilisateur puisse se logger, il faut définir un mot de passe (temporaire ou non)

Création de groupe

Un **groupe d'utilisateur** permet de mutualiser des attributs ou rôles aux utilisateurs appartenant au groupe.

Un groupe est donc :

- Un nom
- Une liste d'attributs
- Éventuellement une liste de rôles

Création de client

Un **client** représente une application intégrée à Keycloak.

Lors de la création d'un client, on indique :

- Son nom
- Si il doit être authentifié
- Le ou les flow(s) d'interaction avec Keycloak :
  Authorization Code, Client Credentials, Device Flow, CIBA
- Son usage ou pas du service d'autorisation de keycloak
- Les scopes possibles (accès aux données utilisateurs)
- Les rôles attribuables spécifiques de ce client

Création de rôle

Un **rôle** est généralement utilisé par les applications clientes pour déterminer les droits d'accès.

Au niveau de *KeyCloak*, un rôle est :

- Un nom
- Une description

Il existe un espace de noms global pour les rôles (**realm roles**) et un espace de nom dédié à un client.

Un rôle peut être associé à des utilisateurs ou des groupes

Un rôle peut être composite et donc contenir d'autres rôles.



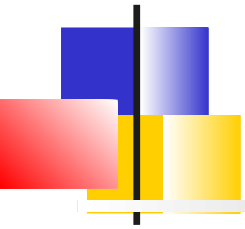
Interface utilisateur

Les utilisateurs accèdent à KeyCloak pour gérer leur propre compte.

L'URL d'accès est :
http://<serveur>/realms/<realm-name>/account/

Par exemple :

- Mettre à jour son profil d'utilisateur
- Mettre à jour son mot de passe
- Activer l'authentification à deux facteurs
- Lister ses applications, les applications auprès desquelles ils se sont authentifiés
- Lister les sessions ouvertes, y compris la déconnexion à distance d'autres sessions



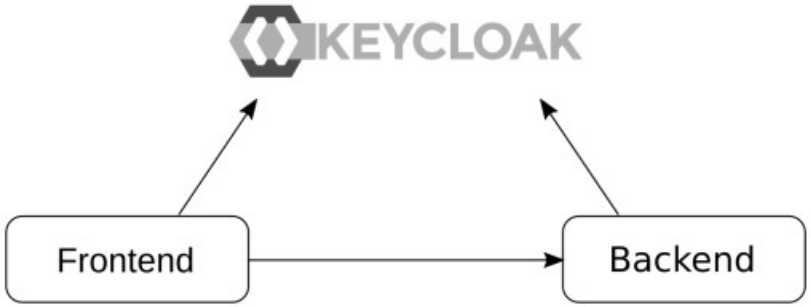
Introduction

Fonctionnalités et installation
Interfaces Admin et utilisateur
Sécuriser une 1ère application

Exemple : SPA + API Rest

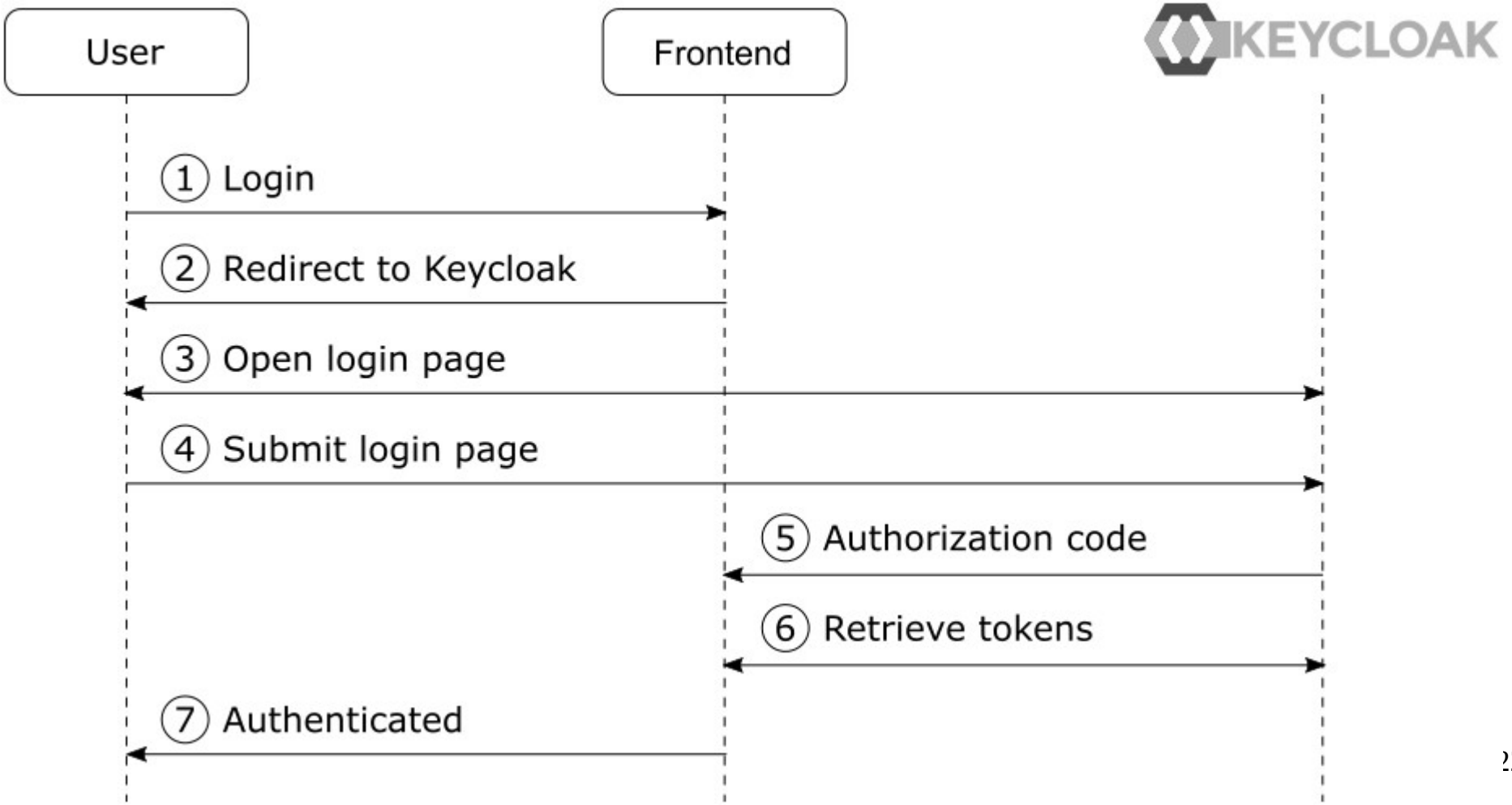
L'application exemple est composée de 2 parties :

- une **front-end** type SPA
  - Authentifiant l'utilisateur via Keycloak afin d'obtenir des jetons le représentant : ID, accès et rafraîchissement
  - Utilisant le jeton d'accès pour invoquer une API Rest
- un **backend** dont les ressources sont protégées par oAuth2



Authentification FRONT-END

OpenID / Authorization code flow



Claims Jetons

Les jetons obtenus contiennent des ***claims*** dépendant du ou des ***scope(s)*** demandés. Les jetons sont hautement personnalisables :

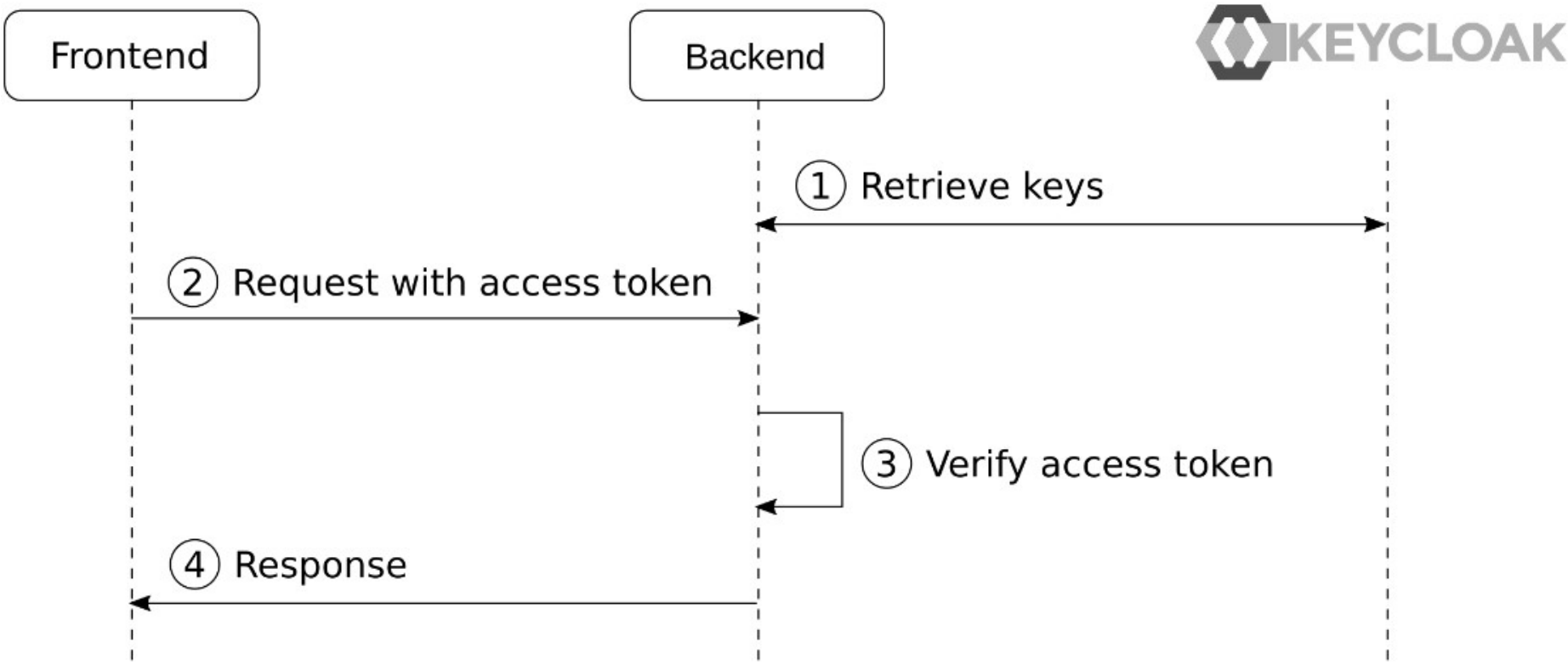
- ***ID_TOKEN*** : Établit l'identité de l'utilisateur
  - ***exp*** : Date d'expiration
  - ***iss*** : Issuer
  - ***sub*** : Identifiant unique de l'utilisateur
  - ***name, preferred_name, …***

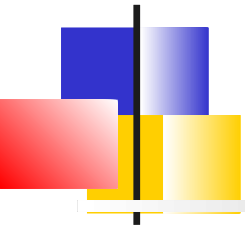
- ***ACCESS_TOKEN*** : Détermine les droits d'accès
  - ***allowed-origins*** : Pour le CORS
  - ***realm-access*** : Intersection entre les rôles de l'utilisateur et les rôles accessibles par le client
  - ***resource_access*** : Les rôles du client
  - ***scope*** : les scopes initiaux demandés

TP 1.3

Vérification des accès BACKEND

JSON Web Signature (JWS)





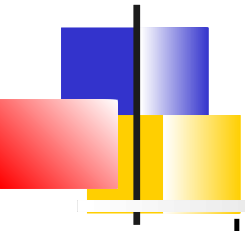
Rappels sur les standards

OAuth2

OpenID Connect

JWT

Transmission et protection des jetons



OAuth 2.0 : pourquoi ?

- Le problème d'origine : pour accéder à vos données chez un autre service, une application vous demandait… votre mot de passe
  - Accès total, impossible à limiter ou à révoquer sans changer le mot de passe, aucune traçabilité
- La réponse OAuth : un framework d'autorisation déléguée, normalisé par l'IETF
  - L'utilisateur accorde à une application un accès limité (scopes) et révocable
  - L'application reçoit un jeton d'accès, jamais le mot de passe
- Né chez Twitter et Google, devenu le standard de sécurisation des API et des applications d'entreprise
- OAuth n'authentifie pas l'utilisateur : c'est le rôle d'OpenID Connect, construit au-dessus

Historique et versions

2007 : OAuth 1.0 (Twitter, Ma.gnolia), RFC 5849 en 2010 : requêtes signées, complexe

Octobre 2012 : OAuth 2.0, RFC 6749 (framework) et RFC 6750 (bearer tokens)

- Plus simple, s'appuie sur TLS, incompatible avec la version 1.0

Février 2014 : OpenID Connect 1.0, couche d'identité au-dessus d'OAuth 2.0

Extensions : PKCE (2015), applications natives (2017), Device flow (2019), PAR (2021), DPoP (2023)

Janvier 2025 : RFC 9700, bonnes pratiques de sécurité

OAuth 2.1 : brouillon IETF (révision 15, mars 2026) qui consolide l'ensemble

- PKCE obligatoire, flows implicit et password supprimés, redirect URI exactes

Rôles définis dans *oAuth2*

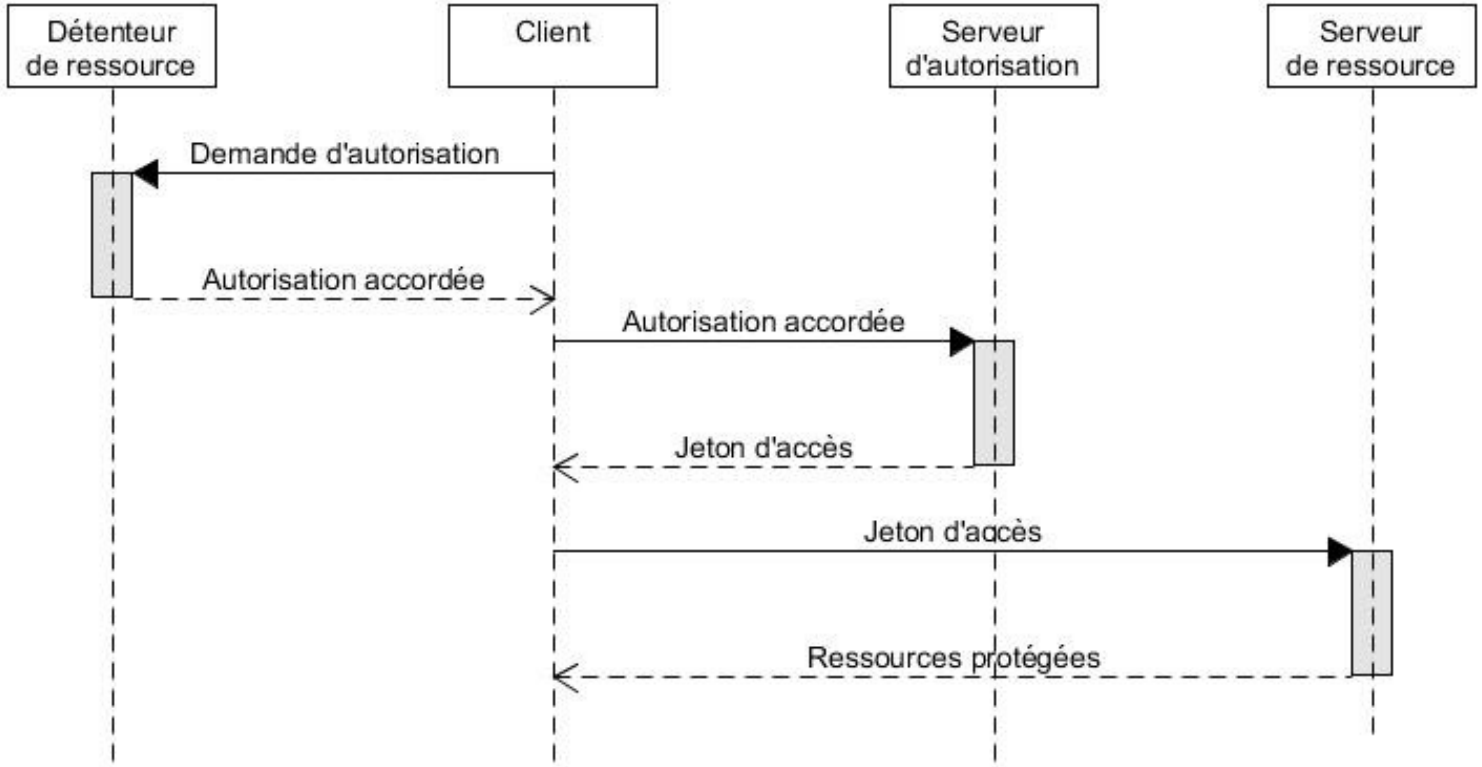
Propriétaire de la ressource : il s'agit généralement de l'utilisateur final qui possède les ressources auxquelles l'application veut accéder.

Serveur de ressources : C'est le service hébergeant les ressources protégées.

Client : Il s'agit de l'application qui souhaite accéder à la ressource

Serveur d'autorisation : C'est le serveur qui délivre l'accès au client. (Keycloak)

Étapes du protocole



Obtention de jetons
les différents flux ou Grant type

Selon le type d'application et le contexte d'usage, différents flows OAuth 2.0 permettent d'obtenir un jeton d'accès. Certains sont recommandés, d'autres à éviter :

- ***Authorization Code*** : Le plus utilisé, l'utilisateur consent et est authentifié chez le fournisseur de jeton. Un code est alors renvoyé à l'application client qui l'utilise pour obtenir le jeton. Adapté aux applications web, mobiles, SPA.
- ***Client Credentials :*** L'application cliente détient un secret qui lui suffit pour obtenir le jeton. L'application accède à la ressource en son propre nom (l'application est le détenteur de la ressource).
  Adapté aux backend consommant des APIs
- Le ***Password flow*** implique que l'application client obtienne les mots de passe de l'utilisateur
  Déprécié , à éviter.
- ***L'Implicit flow*** défini dans la spéc est à éviter car il expose le jeton. Identique à Authorization Code Flow mais sans code d'autorisation, retour direct du jeton.
  Déprécié, à éviter.
- ***Grants d'extension*** : Token Exchange (RFC 8693), JWT Authorization Grant (RFC 7523), Device flow, CIBA.

Types de clients

Les flux sont donc choisis en fonction du type de client.

- Les clients **confidentiels** sont des applications pouvant stocker des crédentiels en toute sécurité.
  EX : Application back-end

- Les clients **publics**, en revanche, sont des applications exécutées côté client qui ne sont pas en mesure de stocker des crédentiels.
  En général, ils s'exécutent dans le navigateur

Protections pour les clients publics

- Les clients publics s'exécutant dans un navigateur utilisent ***l'Authorization Code Flow***
- 2 mesures de protection supplémentaires sont en place :
  - ***Valid redirect URI*** : Le serveur d'autorisation n'enverra le code permettant de récupérer un jeton seulement sur une URI prédéfinie.
  - ***Proof Key for Code Exchange (PKCE, RFC 7636)*** : Extension d'OAuth 2.0 qui empêche quiconque ayant intercepté un code d'autorisation de l'échanger contre un jeton d'accès.

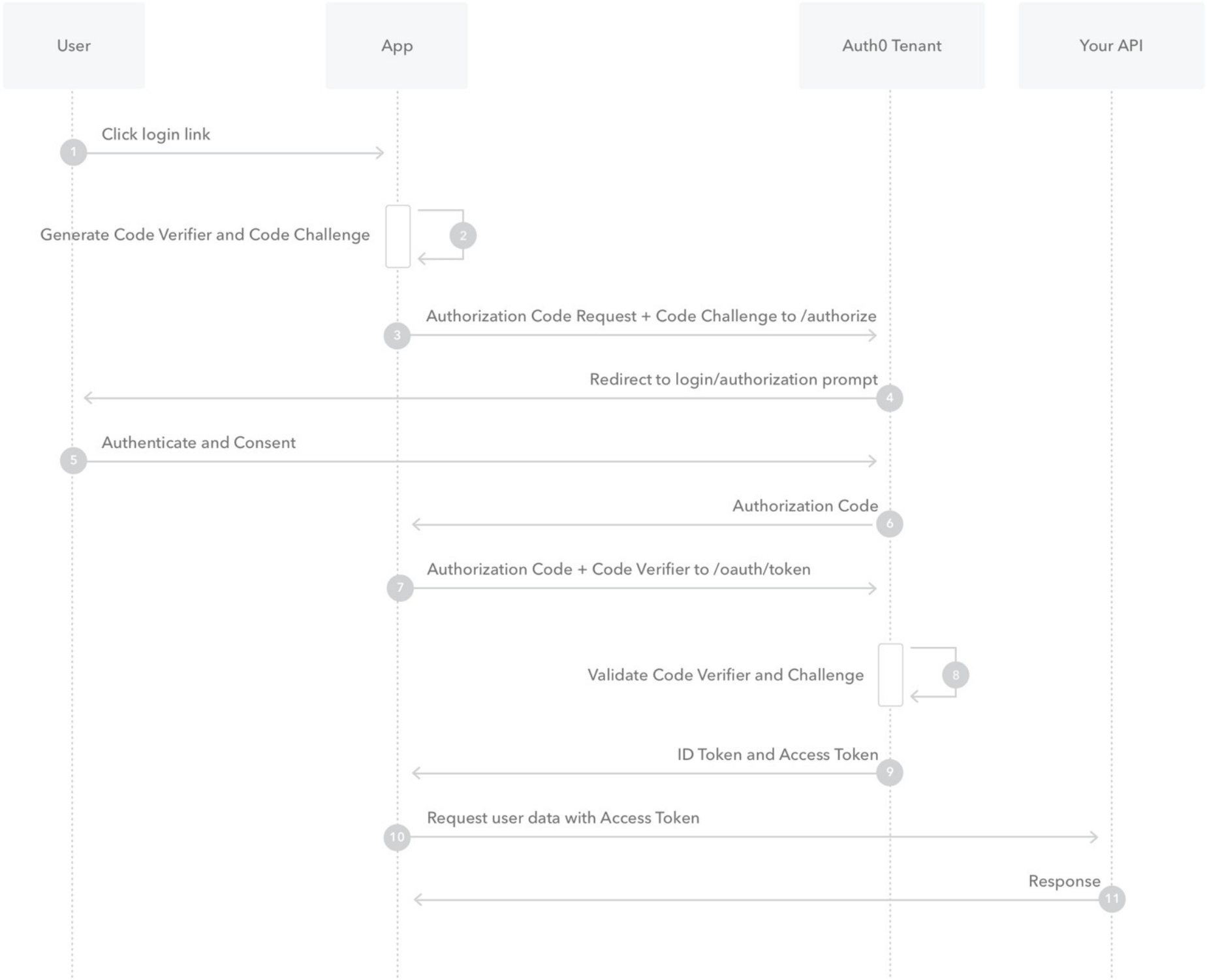
Flux avec PKCE

L’application cliente publique crée une chaîne aléatoire unique : le ***code_verifier***.

Le hash du *code_verifier* est ajouté à la requête de code d’autorisation sous le paramètre ***code_challenge***.

Une fois l’utilisateur authentifié et le code renvoyé, l’application demande les jetons avec le code d’autorisation et le *code_verifier*.

Si les codes correspondent, l’authentification est terminée et un *access_token* est renvoyé.


User
App
Auth0 Tenant
Your API
Click login link
1
Generate Code Verifier and Code Challenge
2
Authorization Code Request + Code Challenge to /authorize
3
Redirect to login/authorization prompt
4
Authenticate and Consent
5
Authorization Code
6
Authorization Code + Code Verifier to /oauth/token
7
Validate Code Verifier and Challenge
8
ID Token and Access Token
9
Request user data with Access Token
10
Response
11

Validation du jeton

Lors de la réception du jeton, le serveur de ressources doit valider l'authenticité du jeton et extraire ses informations.

Valider, c'est s'assurer que le jeton est **intègre**, émis par le bon serveur d'autorisation (***iss***), destiné à cette API (***aud***) et toujours valide (***exp***).

Différentes techniques sont possibles, selon le format du jeton :

- **Jetons opaques** et appel REST vers le serveur d'autorisation (introspection) : seul l'émetteur sait lire le jeton ; il renvoie un état à jour (révocation incluse), au prix d'un appel réseau par requête
- **Jetons JWT** et validation via paire de clés privée/publique : le serveur d'autorisation signe avec sa clé privée, l'API vérifie localement avec la clé publique publiée ; aucun appel, mais le jeton reste valide jusqu'à son expiration

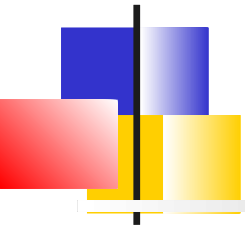
Arbitrage performance / fraîcheur : Keycloak émet des JWT, mais les deux techniques restent possibles (voir chapitre OAuth 2.0).

Spécifications additionnelles

Bearer tokens (RFC 6750) : Les jetons Bearer sont de loin le type de jetons d'accès le plus couramment utilisé, et ils sont généralement envoyés aux serveurs de ressources via l'en-tête *HTTP Authorization*.

Token Introspection (RFC 7662) : Dans OAuth 2.0, le contenu des jetons d'accès est potentiellement opaque pour les applications. Le endpoint d'introspection permet au client d'obtenir des informations sur le jeton d'accès sans comprendre son format.

Token Revocation (RFC 7009): OAuth 2.0 prend en compte la manière dont les jetons d'accès sont délivrés aux applications, mais pas la manière dont ils sont révoqués.
Ceci est couvert par le endpoint de révocation de jeton.



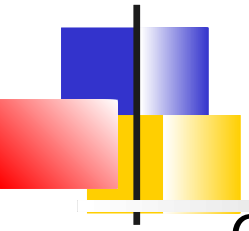
Rappels sur les standards

OAuth2

OpenID Connect

JWT

Transmission et protection des jetons



Introduction

OAuth 2.0 est un protocole d'autorisation : il ne dit pas qui est l'utilisateur

OpenID Connect ajoute une couche d'authentification au-dessus d'OAuth 2.0 :

- Le scope ***openid***, qui active le comportement OpenID
- L'**ID token** : preuve de l'authentification destinée à l'application
- Des **scopes et claims standards** décrivant l'utilisateur
- **L'endpoint UserInfo**

Cas d'usage : SSO d'entreprise, login social, fédération d'identités

L'application ne voit jamais le mot de passe ; la méthode d'authentification (MFA…) relève du fournisseur

Rôles de OpenID

Utilisateur final : équivalent du détenteur de ressource d'OAuth 2.0, l'utilisateur qui s'authentifie

Relying Party (RP) : l'application qui souhaite authentifier l'utilisateur, équivalent du client OAuth 2.0

Fournisseur OpenID (OP) : le fournisseur d'identité qui authentifie l'utilisateur (Keycloak)

L'application demande l'identité de l'utilisateur à l'OP

S'appuyant sur OAuth 2.0, elle peut obtenir en même temps un jeton d'accès

Flux d’OpenID

Code d’autorisation (response_type=code)

- Le code est échangé contre ID token, access token et refresh token
- Avec PKCE : flux recommandé pour tous les clients, y compris SPA et mobiles

Hybride (response_type=code id_token…)

- ID token renvoyé avec le code dès la réponse d’autorisation, usage rare hérité de FAPI 1.0, remplacé aujourd’hui par PKCE et PAR

Implicite (response_type=id_token token)

- Jetons dans l’URL, sans code : retiré d’OAuth 2.1, à proscrire (RFC 9700)

Ces flux nécessitent un navigateur et un utilisateur qui interagit

Flows sans navigateur

Grants OAuth 2.0 : avec le scope openid, un ID token s'ajoute dès qu'un utilisateur s'authentifie

Client credentials : pas d'utilisateur, le client agit pour son propre compte

- Pas d'ID token : OAuth 2.0 pur, machine à machine

Device Authorization Grant (RFC 8628) : TV, objets connectés, CLI

- L'utilisateur s'authentifie sur un autre appareil en saisissant un code

CIBA : authentification déclenchée par un backend (centre d'appels…), validée sur un autre canal

- Spécification OpenID Foundation : prouver l'identité est le but. Détail en annexe

Refresh token : renouvelle les jetons sans interaction, ID token compris (OIDC Core §12)

Password grant (ROPC) : l'application collecte le mot de passe, à proscrire

- **Pourquoi ?** Contourne MFA, fédération et SSO. Toléré en migration ou en test (« Direct access grants »)

Scopes et claims standards

- Le scope ***openid*** est obligatoire ; les autres scopes demandent des claims :
  - ***profile*** : name, given_name, family_name, preferred_username, locale…
  - ***email*** : email, email_verified
  - ***address*** : address
  - ***phone*** : phone_number, phone_number_verified
- ***offline_access*** : demande d'un refresh token utilisable hors session
- Les claims sont livrés dans l'ID token et/ou par UserInfo
- Dans Keycloak : client scopes du même nom, avec leurs mappers

Spécifications additionnelles OpenID

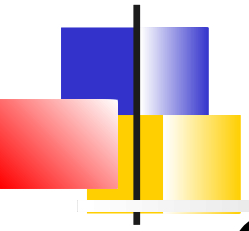
Discovery : découverte dynamique de la configuration du fournisseur

Dynamic Client Registration : enregistrement d'un client par API (Keycloak : client registration, kcreg)

Logout :

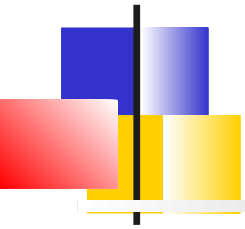
- **RP-Initiated Logout** : déconnexion initiée par l'application auprès du fournisseur
- **Back-Channel Logout** : notification de déconnexion de serveur à serveur
- **Front-Channel Logout** : notification par iframes dans le navigateur

Session Management : surveillance de la session par iframe pollant keycloak, compromise par le blocage des cookies tiers



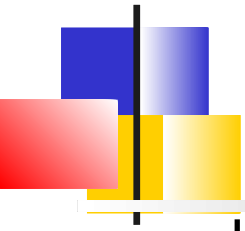
ID token et UserInfo

- *OpenID Connect* impose le format JWT pour l'ID token
  - Le client lit les claims après validation de la signature, de iss, aud, exp et nonce
- L'endpoint *UserInfo*, appelé avec l'access token, retourne des claims sur l'utilisateur
  - Configurable : pas nécessairement les mêmes informations que l'ID token
  - Pas de claims techniques (iss, aud, nonce…)



Rappels sur les standards

OAuth2
OpenID Connect
JWT
Transmission et protection des jetons



Introduction

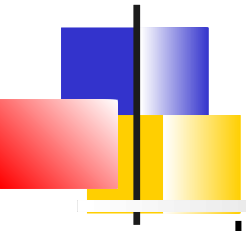
Keycloak utilise **JWT** comme format par défaut, y compris pour les jetons d'accès

Avantages :

- Format JSON facilement exploitable par les librairies
- Les serveurs de ressources valident le jeton localement, sans appel au serveur d'origine

Contreparties :

- Signé n'est pas chiffré : contenu lisible par tous, pas de données sensibles
- Non révocable avant expiration s'il est validé localement : durées de vie courtes
- Taille croissante avec les rôles et claims (en-têtes HTTP, cookies)



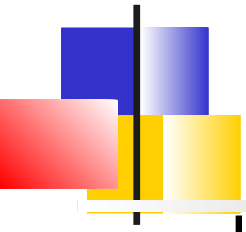
JOSE

JWT est issu de la famille de spécifications JOSE :

- **JWT** (RFC 7519) : format des revendications (claims)
- **JWS** (RFC 7515) : jeton signé, 3 parties en Base64URL : en-tête.charge utile.signature
- **JWE** (RFC 7516) : jeton chiffré, 5 parties
- **JWA** (RFC 7518) : algorithmes, asymétriques (RS256, ES256, PS256) ou à secret partagé (HS256)
- **JWK** (RFC 7517) : représentation JSON d'une clé ; JWKS : ensemble de clés

En-tête : alg (algorithme), kid (identifiant de la clé), typ (type de jeton)

Utilisation de JOSE dans OpenID Connect



- JWT + JWS : format de l'ID token, obligatoirement signé
- JWA : identifiants des algorithmes (RS256, ES256, PS256…)
- JWK / JWKS : publication des clés publiques sur jwks_uri
- JWE : chiffrement optionnel de l'ID token avec la clé publique du client. Réservé aux contextes réglementés (FAPI)
- Autres JWT signés : réponse UserInfo signée, logout token, private_key_jwt, preuve DPoP

Validation d’un JWT

Le discovery fournit l’URL ***jwks_uri*** : clés publiques du realm, mises en cache par le serveur de ressources

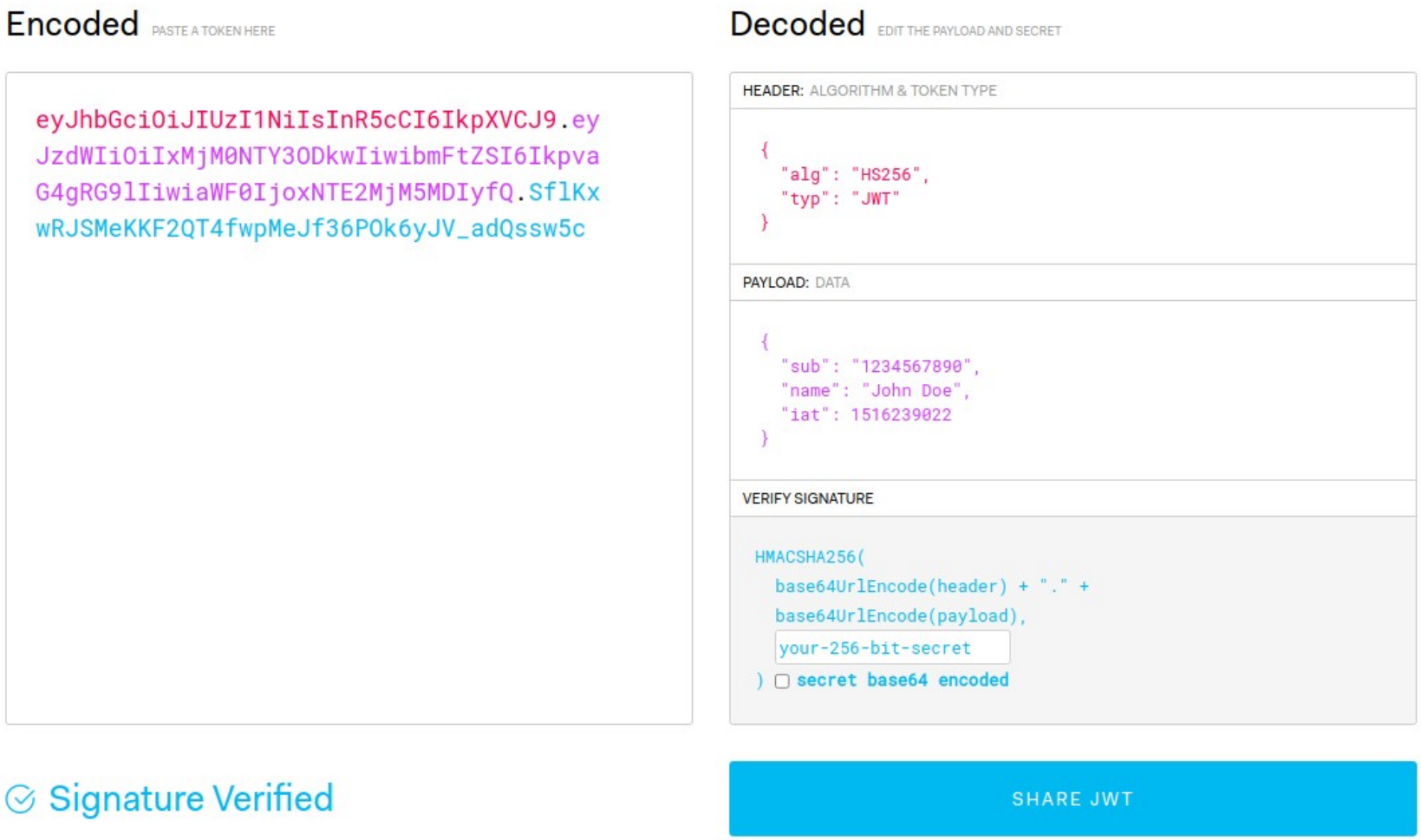
Étapes de validation :

- Algorithme dans une liste autorisée : refuser none, éviter la confusion RS256 / HS256
- Sélection de la clé par le *kid*, vérification de la signature
- Contrôle des claims : iss, aud, exp, nbf

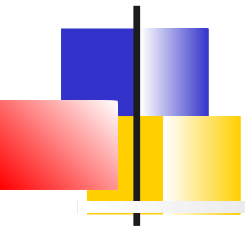
Rotation des clés : un *kid* inconnu doit déclencher le rafraîchissement du cache JWKS

En pratique : utiliser une librairie éprouvée plutôt que du code maison

jwt.io

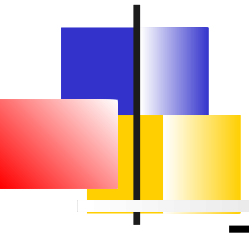


Attention : ne pas coller de jetons de production dans un site en ligne.



Rappels sur les standards

OAuth2
OpenID Connect
JWT
Transmission et protection des jetons



Introduction

Tous les jetons ne se valent pas en termes de sécurité.

Certains sont "présentables" (Bearer), d’autres nécessitent une preuve de possession (DPoP, mTLS), d’autres protège leur contenu (Phantom, Split).

Keycloak prend en charge plusieurs de ces approches selon les cas d’usage.

Bearer token

Jeton auto-suffisant, envoyé dans l'en-tête HTTP :

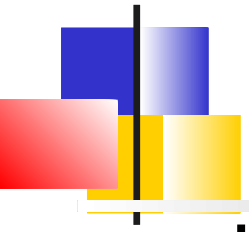
Authorization: Bearer <token>

Le serveur de ressource accepte le jeton sans preuve supplémentaire.

- Supporté nativement par Keycloak.
- Risque : toute entité possédant le jeton peut l'utiliser (token replay).

Transmission :

- Toujours dans l'entête
- Toujours en https
- Le stockage en LocalStorage expose au XSS



DPoP

Le client prouve qu'il détient une clé privée associée au jeton.

Envoie un header DPoP contenant une preuve cryptographique signée.

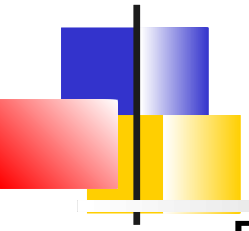
- Empêche l'usage d'un jeton volé par un tiers.
- Support dans les versions récentes de Keycloak

mTLS

Utilise une **authentification TLS mutuelle** (le client présente un certificat).

Le jeton est lié à l'empreinte du certificat.

- Très robuste, adapté aux cas réglementés (FAPI, banque).
- Supporté par Keycloak, mais nécessite :
  - Activation dans le client
  - Infrastructure compatible mTLS



Phantom token et Split token

Deux patterns d'architecture, non standardisés, mis en œuvre par une API gateway

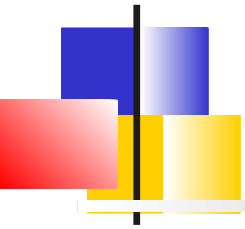
Objectif : ne pas exposer le contenu du jeton en dehors du système d'information

Phantom token : le client reçoit un jeton opaque

- La gateway l'échange contre un JWT par introspection (RFC 7662)
- Les API internes reçoivent le JWT

Split token : le client ne reçoit que la signature du JWT

- La gateway conserve le reste du jeton et le reconstitue



Keycloak et OpenID

Discovery endpoint
Authentification
Logout

Discovery Endpoint

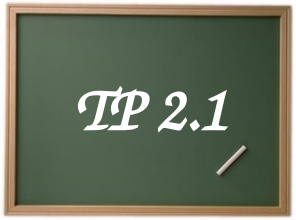
Le point de découverte de Keycloak est accessible à :

```
https://<server>/realms/<realm-name>/.well-known/openid-configuration
```

Permet à un client de découvrir toutes les informations de configuration utiles :

- Endpoints et URIs accessibles
- Types de grants et de réponses supportés
- Méthodes PKCE supportées (code_challenge_methods_supported)
- Mécanismes de logout supportés
- Algorithmes de signature et de chiffrement, scopes et claims supportés

Les librairies clientes s'auto-configurent à partir de ce document



Endpoints

```
"issuer":   # Identifiant du serveur (iss)
  "https://<server>/realms/<realm-name>",

"authorization_endpoint": # Obtention du code d'autorisation
  "https://<server>/realms/<realm-name>/protocol/openid-connect/auth",

"token_endpoint":   # Obtentiondu token
  "https://<server>/realms/<realm-name>/protocol/openid-connect/token",

"introspection_endpoint": # Permet la validation d'un token
  "https://<server>/realms/<realm-name>/protocol/openid-connect/token/introspect",

"userinfo_endpoint": # Info User
  "https://<server>/realms/<realm-name>/protocol/openid-connect/userinfo",

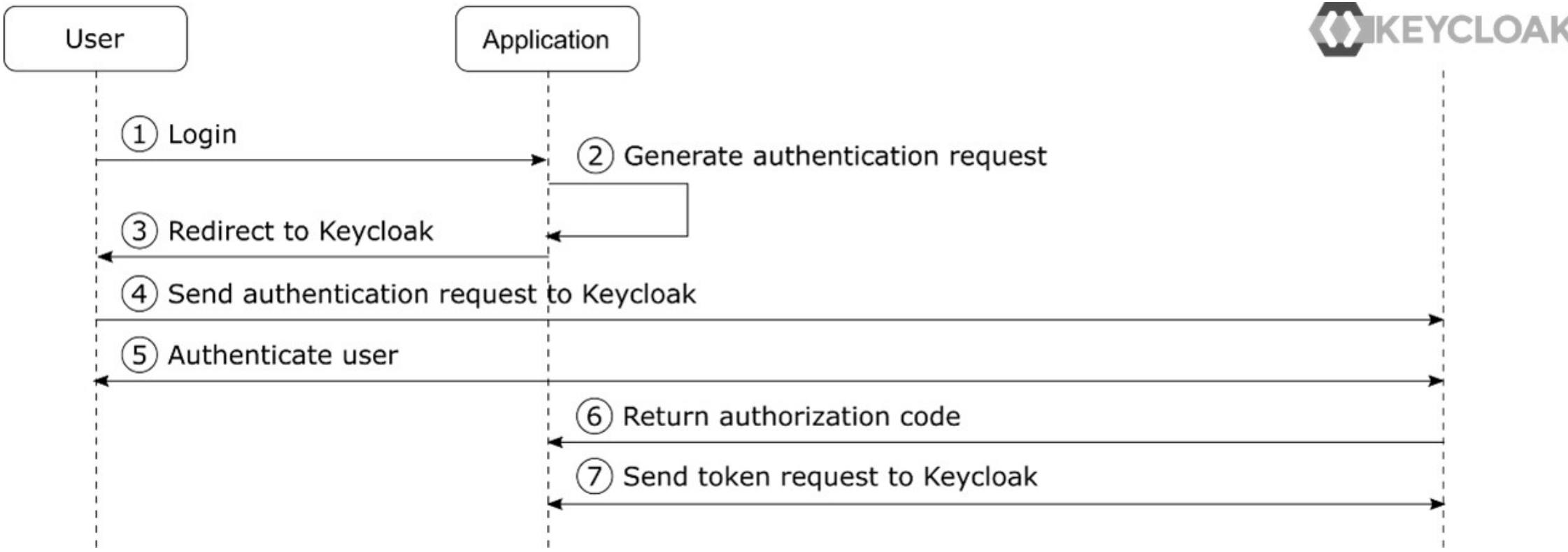
"end_session_endpoint": # Fermer le session SSO
  "https://<server>/realms/<realm-name>/protocol/openid-connect/logout",

"jwks_uri": # Clé publiques pour vérifier localement un jeton
  "https://<server>/realms/<realm-name>/protocol/openid-connect/certs",
```

Authentification avec OpenID

Discovery endpoint
Authentification
Logout

Authorization Code Flow



Requête d'autorisation

Paramètres obligatoires :

- ***client_id***, ***redirect_uri*** (déclarée dans le client), ***response_type=code***
- ***scope*** : doit contenir *openid*, d'autres scopes peuvent être ajoutés

Paramètres de sécurité :

- ***state*** : valeur aléatoire vérifiée au retour (protection CSRF)
- ***nonce*** : recopié dans l'ID token (protection contre le rejeu)
- PKCE : ***code_challenge*** et ***code_challenge_method=S256***[1], obligatoire selon le RFC 9700

Paramètres optionnels : ***prompt*** (none, login, consent), ***max_age***, ***login_hint***, ***acr_values***, ***kc_idp_hint*** [2]

La réponse est un code d'autorisation à usage unique, valable 1 min par défaut

1. BASE64URL(SHA-256(code_verifier))

2. Paged de login, dernière date d'authentification acceptée pour le sso, le champ login renseigné, niveau d'authentification demandé, identifiant d'un idp fédéré

Échange du code

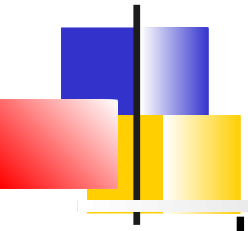
Requête POST sur le token endpoint :

- ***grant_type=authorization_code***
- ***code*** : le code reçu
- ***redirect_uri*** : identique à celle de la requête d'autorisation (vérification)
- ***code_verifier*** : secret PKCE dont le code_challenge est dérivé

Authentification d'un client confidentiel :

- ***client_id*** et ***client_secret*** (en Basic ou dans le corps)
- Ou JWT signé (private_key_jwt), certificat mTLS

Un client public n'envoie que son client_id



Réponse jetons

La réponse JSON contient :

- ***access_token*** et ***expires_in*** : durée de validité en secondes
- ***refresh_token*** et ***refresh_expires_in***
- ***id_token*** : jeton d'identification
- ***token_type*** : Bearer, ou DPoP si le jeton est lié à une clé du client
- ***scope*** : scopes effectivement accordés, ils peuvent différer de la demande
- ***session_state*** : identifiant de la session Keycloak

Rafraîchissement

L’access token a une courte durée de vie pour limiter l’impact d’une fuite

- L’application n’a pas à ré-authentifier l’utilisateur

Le refresh token, de durée plus longue, permet d’obtenir de nouveaux jetons :

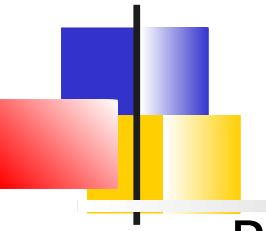
- ***grant_type=refresh_token*** sur le token endpoint
- Nouvel access token, nouvel ID token et nouveau refresh token

Par défaut, l’ancien refresh token reste réutilisable

- Option « *Revoke Refresh Token* » du realm pour un usage unique

Sa validité est bornée par la session SSO (SSO Session Idle et Max)

Offline token (scope *offline_access*) : survit à la fin de la session SSO



Jeton d’identification

Preuve de l’authentification, destinée au client (l’application)

- Ne doit pas servir à appeler une API : c’est le rôle de l’access token

JWT signé par la clé du realm :

- <En-tête>.<Charge utile>.<Signature>, encodés en Base64URL

Le client doit le valider :

- Signature, avec les clés publiées sur *jwks_uri*
- *iss, aud, exp*
- *nonce* identique à celui de la requête

Revendications

iss : émetteur, URL du realm

sub : identifiant unique de l'utilisateur

aud : les audiences, client_id pour lequel ce jeton est destiné;

azp : client_id du client qui a demandé le jeton

exp, ***iat***, ***auth_time*** : expiration, émission, dernière authentification

nonce : valeur transmise dans la requête d'autorisation

acr : niveau d'authentification ;

sid : identifiant de session

at_hash : empreinte de l'access token

Les autres claims décrivent l'utilisateur et se configurent via les client scopes et mappers



UserInfo endpoint

Retourne les informations de l'utilisateur authentifié

```
GET https://<server>/realms/<realm-name>/protocol/openid-connect/userinfo
Authorization: Bearer <access_token>
```

L'access token doit avoir été obtenu avec le scope *openid*

Configuration :

- Option « *Add to userinfo* » de chaque mapper
- *Client → Advanced → User info signed response algorithm* : réponse JWT signée

Intérêt : garder un ID token léger et obtenir des informations à jour

User Profile

- Depuis la version 24, le profil utilisateur est déclaratif et toujours activé
- *Realm settings → User profile*
  - Liste des attributs gérés et groupes d’attributs
  - Éditeur de formulaire ou JSON
- Un même schéma pilote :
  - La console d’administration et la console du compte
  - Les pages d’inscription et de mise à jour du profil
  - L’API d’administration

1. Avant la version 24, les attributs n’étaient pas gérés de cette façon. Le champ unmanaged attributes permet de spécifier le comportement vis à vis des attrinuts non déclarés dont les anciens (disabled par défaut)



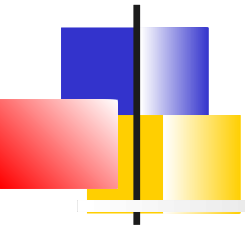
Définition d'un attribut

Propriétés d'un attribut :

- Nom, libellé (internationalisable), mono ou multi-valué
- Obligatoire pour les utilisateurs et/ou les admins, éventuellement selon les scopes demandés
- Permissions de lecture et d'écriture : user, admin
- Validations : length, email, pattern, options, integer, uri…
- Annotations : type de champ du formulaire (select, textarea…)

Exposition dans les jetons :

- Mapper « *User Attribute* » dans un client scope dédié
- Nom du claim et jetons cibles : ID token, access token, UserInfo



Authentification avec OpenID

Discovery endpoint
Authentification
Logout



Introduction

Gérer la déconnexion dans une expérience SSO est délicat, surtout si l'on veut une déconnexion instantanée de toutes les applications

Après un login, trois éléments vivent indépendamment :

- La session SSO dans Keycloak (cookie sur le domaine de Keycloak)
- La session locale de chaque application (son propre cookie)
- Les jetons déjà émis : access, refresh et ID tokens

Un logout peut être initié par l'expiration de la session ou par une action de l'utilisateur

Selon le type de client, la déconnexion est répercutée de façon différente

La déconnexion doit aussi rendre inutilisables les jetons de l'utilisateur

Scénario de logout

1. L’utilisateur clique sur Déconnexion dans une application
2. L’application nettoie son contexte et redirige vers l’*end_session_endpoint* avec :
   - *id_token_hint* ou *client_id* : identification du client
   - *post_logout_redirect_uri* : URI de retour, déclarée dans le client
   - *state, ui_locales* (langue de l’écran de déconnexion)
3. Keycloak ferme la session SSO : les refresh tokens sont invalidés
   - Un access token JWT reste valide jusqu’à son expiration pour une API qui le valide localement (sauf introspection)
4. Si configuré, Keycloak notifie les autres clients (front ou back-channel)
5. L’utilisateur est redirigé vers post_logout_redirect_uri
6. Reste à prévenir les autres applications

Propagation du logout

Chaque application garde sa session locale : elle doit apprendre que la session SSO est fermée

Deux questions : qui prend l'initiative, et par quel canal ?

	L'application interroge (pull)	**Keycloak notifie (push)**
Par le serveur	Expiration / échec du refresh simple, mais pas immédiat	**Back-channel fiable, immédiat : recommandé**
Par le navigateur	Session Management cookies tiers : legacy	Front-channel cookies tiers : fragile

Le back-channel est le seul mécanisme à la fois côté serveur et à l'initiative de Keycloak. La ligne navigateur dépend des cookies tiers, la colonne pull n'est jamais immédiate.

Détection par expiration

L’application se rend compte de la déconnexion quand un token n’est plus utilisable.

- Les tokens d’accès et ID tokens ont une durée de vie courte (ex. 5 min).

L’application essaie de rafraîchir le token via le *refresh_token*.

Si ce dernier est révoqué ou expiré, l’utilisateur doit se reconnecter.

Avantages : simple, ne nécessite aucune configuration spéciale.

Limite : la déconnexion n’est pas immédiate pour les autres clients.

OIDC Session Management

L’application surveille l’état de la session SSO via une iframe

- Iframe chargeant /realms/{realm}/protocol/openid-connect/login-status-iframe.html
- Polling régulier, logout local si la session est fermée

Avantage : pas de traitement côté serveur

Limite majeure : l’iframe est cross-site et dépend des cookies tiers

- Bloqués par Safari et Firefox, de plus en plus restreints par Chrome

Mécanisme à considérer comme legacy

OIDC Back-Channel Logout

Keycloak notifie directement le backend des applications

- Chaque client déclare une Backchannel logout URL
- Keycloak envoie un POST contenant un logout token (JWT signé)
- L'application valide le jeton et invalide la session correspondante (sid ou sub)

Option « Backchannel logout session required » : inclut le sid

Mécanisme recommandé : fiable, indépendant du navigateur, fonctionne même navigateur fermé

Limites :

- Keycloak doit pouvoir joindre le backend (réseau, Docker, proxy)
- L'application doit retrouver sa session à partir du *sid*

OIDC Front-Channel Logout

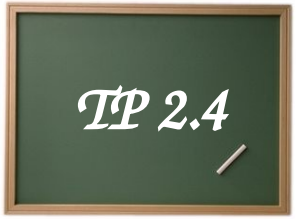
Propagation du logout via des iframes client dans la page de logout Keycloak.

- Chaque client enregistre une URI de logout front-channel.
- Lors du logout, Keycloak affiche une page contenant une iframe par client.
- Ces iframes appellent la page de logout du client pour qu'il nettoie la session.

Avantages : pas de backend requis.

Limites :

- Iframes cross-site : dépend des cookies tiers, de plus en plus bloqués par les navigateurs
- Fragile, nécessite une configuration soignée



Logout : recommandations

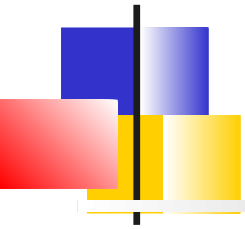
Toujours : jetons d'accès courts, filet de sécurité commun à tous

Application avec backend : back-channel, avec le sid pour retrouver la session

SPA sans backend : jetons courts, ou pattern BFF (cas précédent)

Front-channel et Session Management : en dernier recours (cookies tiers)

API : JWT valide jusqu'à expiration ; introspection si la révocation doit être immédiate



Autoriser les accès avec oAuth2

Jeton d’accès et consentement

Limitations des accès

Personnalisation du jeton

Validation du jeton

D’OIDC à OAuth 2.0

OpenID Connect : qui est l’utilisateur ? → ID token, destiné à l’application

OAuth 2.0 : que peut faire l’application en son nom ? → access token, destiné aux API

Même flux, même requête : OIDC est une couche au-dessus d’OAuth 2.0

- scope *openid* → identité ; autres scopes → droits délégués

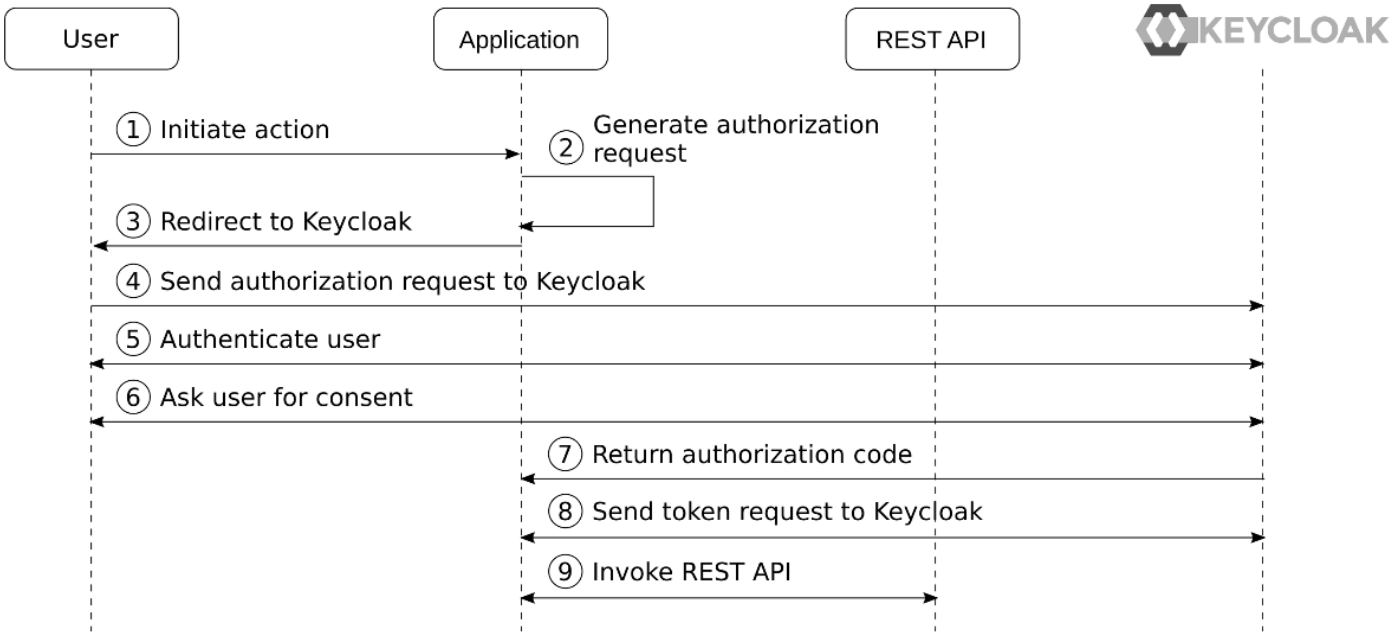
Oauth est né pour la délégation il ajoute **le consentement** : l’utilisateur autorise une application à agir sur ses données sans lui donner son mot de passe

- Le consentement rend cette délégation explicite ; il est inutile pour une application interne de confiance

Obtenir un jeton d'accès

- L'***Authorization-flow*** est le moyen le plus courant d'obtenir un jeton d'accès

Même flux qu'en OIDC : ici, ce sont les étapes 6 (consentement) et 9 (appel de l'API) qui comptent



Le consentement de l'utilisateur est optionnel et désactivé par défaut dans Keycloak

Contenu du jeton d'accès

Claims standards : ***iss, sub, exp, iat***

aud : services auxquels le jeton est destiné

azp : client qui a obtenu le jeton

scope : scopes accordés, séparés par des espaces

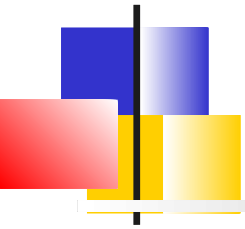
Claims propres à Keycloak :

- **realm_access.roles** : rôles realm, intersection des rôles de l'utilisateur et de ceux autorisés pour le client
- **resource_access** : rôles client, regroupés par client



Consentement de l’utilisateur

- Le consentement se configure par client : option *Consent required*
  - Désactivé par défaut : adapté aux applications internes de confiance
  - À activer pour les applications tierces ou externes
- Les scopes demandés déterminent les privilèges présentés à l’utilisateur
  - Texte affiché paramétrable par client scope (Display on consent screen, Consent screen text)
  - Scopes pouvant être demandés progressivement par l’application
- Keycloak conserve les consentements ; l’utilisateur peut les révoquer dans la console du compte (Applications)



Autoriser les accès avec oAuth2

Jeton d’accès et consentement

Limitations des accès

Personnalisation du jeton

Validation du jeton

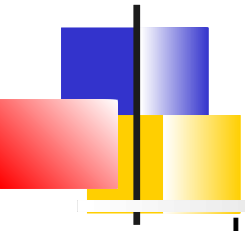
Limiter les accès du jeton

Les accès du jeton peuvent être limités selon 3 axes principaux :

- **Audience** : permet de lister les fournisseurs de ressources qui doivent accepter un jeton d'accès.
- **Rôles** : Les ACLs sur les ressources sont exprimées en fonction des rôles utilisateur.
- **Scopes**[1] : Les ACLs sur les ressources sont exprimées en fonction des scopes clients.

1. Seul mécanisme natif d'OAuth 2.0 ; dans Keycloak, chaque scope demandé correspond au client scope du même nom.

Client scopes



Un **client scope** regroupe une configuration réutilisable par plusieurs clients :

- Des mappers : claims ajoutés aux jetons
- Un role scope mapping : rôles que le client peut recevoir

Liaison avec un client (*Client → Client scopes*) :

- **Default** : toujours appliqué
- **Optional** : appliqué seulement s'il est demandé dans le paramètre scope

Chaque client possède un scope dédié (<client>-dedicated), toujours appliqué

- Il porte l'option Full scope allowed : si elle est activée (défaut), le client reçoit tous les rôles de l'utilisateur, sans filtrage par rapport au client
- Dépréciée (26.8) : la laisser désactivée et déclarer les rôles utiles pour ce client dans l'onglet Scope



Configuration des audiences

Le claim ***aud*** liste les services destinataires du jeton : chaque API doit vérifier qu'elle y figure

2 manières d'ajouter un client à l'audience :

- Manuelle : mapper **Audience**, par exemple dans le scope dédié du client
- Automatique : mapper **Audience Resolve** (scope roles), qui ajoute les clients dont le jeton contient des rôles client

Souvent, aud contient account : les rôles par défaut des utilisateurs (manage-account, view-profile) passent dans le jeton, et Audience Resolve ajoute leur client. Ce n'est plus le cas si Full scope allowed est désactivé.



Configuration des rôles

2 questions distinctes : quels rôles possède l'utilisateur ? quels rôles le client peut-il recevoir ?

Rôles inclus dans le jeton = intersection entre :

- **Les rôles de l'utilisateur** (attribués directement, par groupe ou par défaut)
- **Les rôles autorisés pour le client** : ses propres rôles client, et ceux du scope dédié et des client scopes appliqués

Pourquoi filtrer ? Moindre privilège : le jeton d'une application ne porte que les rôles utiles à ses API

- Par défaut, Full scope allowed : le client reçoit tous les rôles de l'utilisateur (option dépréciée)
- Pour limiter : désactiver Full scope allowed , puis choisir les rôles autorisés



ACLs sur les scopes

Mécanisme standard d'OAuth 2.0 pour limiter les autorisations d'un jeton d'accès

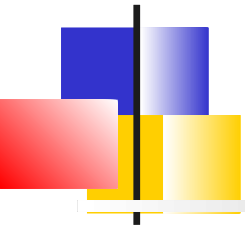
Dans Keycloak, un scope OAuth 2.0 correspond à un client scope

- Pour un simple scope d'ACL : client scope sans mapper ni rôle
- Généralement déclaré optionnel et demandé par l'application si nécessaire

L'API contrôle le claim scope du jeton d'accès

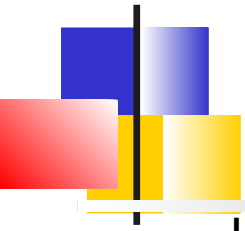
Nommage : préfixer par le nom ou l'URL du service

- albums:view, https://api.acme.org/albums.view, https://www.googleapis.com/auth/calendar.events



Autoriser les accès avec oAuth2

Jeton d’accès et consentement
Limitations des accès
Personnalisation du jeton
Validation du jeton



Introduction

Les différents clients d'une application n'ont pas nécessairement besoin des mêmes informations dans les jetons.

Dans OAuth2 / OIDC, les *scopes* permettent de définir les droits ou informations qu'un client peut demander (ex. openid, email, profile).

Keycloak implémente ce mécanisme via 2 composants clés :

- ***Client Scopes*** : ensembles configurables regroupant des paramètres (notamment des mappers) associés à un ou plusieurs scopes OAuth2/OIDC.
  Ils peuvent être automatiquement attachés ou activés explicitement lors de l'authentification.
- ***Protocol Mappers*** : définissent les informations (claims) ajoutées dans les tokens.

En combinant client scopes et mappers, Keycloak adapte dynamiquement les jetons en fonction des scopes OAuth2 demandés.

Protocol Mappers

Les ***Mappers*** permettent de personnaliser les revendications des jetons fournies à un client:

- Informations statiques,
- Attributs d'utilisateur
- Rôles.

Ils s'appliquent aux tokens ID, tokens d'accès, et à l'endpoint *userinfo* selon la configuration

Keycloak fournit des mappers prédéfinis.

Les mappers peuvent être définis au niveau d'un client ou d'un client scope

Client scopes prédéfinis

profile, email, address, phone : scopes OpenID Connect (mappers, pas de rôle)

offline_access : obtention d'un offline token, pour des applications inactives longtemps

basic : claims sub et auth_time (depuis la version 25)

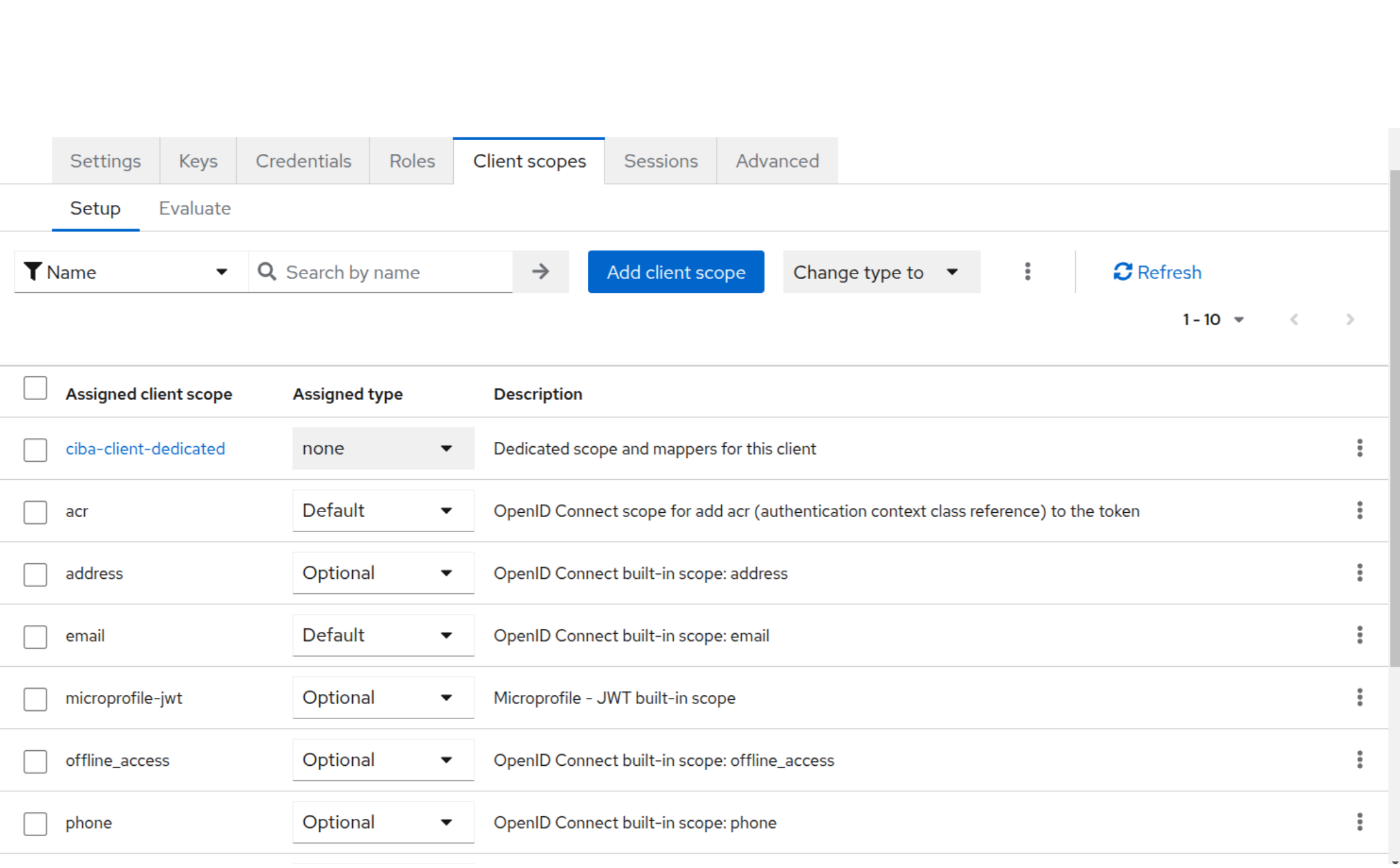
roles : rôles dans le jeton d'accès, mapper Audience Resolve

web-origins : origines autorisées (allowed-origins), pour CORS

acr : niveau d'authentification

microprofile-jwt, **organization** (version 26) : usages spécifiques

basic, *roles*, *web-origins* et *acr* sont appliqués par défaut mais n'apparaissent pas dans le claim scope


Settings
Keys
Credentials
Roles
Client scopes
Sessions
Advanced
Setup
Evaluate
Name
Search by name
Add client scope
Change type to
Refresh
1 - 10
Assigned client scope
Assigned type
Description
ciba-client-dedicated
none
Dedicated scope and mappers for this client
acr
Default
OpenID Connect scope for add acr (authentication context class reference) to the token
address
Optional
OpenID Connect built-in scope: address
email
Default
OpenID Connect built-in scope: email
microprofile-jwt
Optional
Microprofile - JWT built-in scope
offline_access
Optional
OpenID Connect built-in scope: offline_access
phone
Optional
OpenID Connect built-in scope: phone

Évaluer les client scopes

Il est possible d’évaluer les mappeurs effectifs et les mapping de rôle qui seront générés.

Client Scopes → Evaluate

Choisir ensuite les scopes optionnels éventuels

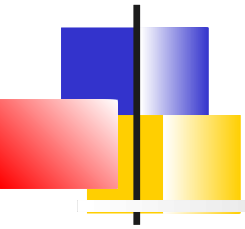


Permissions des client scopes

A l'émission d'un jeton, les client scopes ne sont appliqués que si l'utilisateur y est autorisé.

Au niveau d'un client scope, on peut également limiter les rôles pouvant le demander (Onglet *Scope*)

- Lorsqu'aucun rôle mapping n'est défini, Tous les utilisateurs peuvent utiliser tous les scopes.
- Quand des mappings sont définis, seuls les utilisateurs possédant au moins un de ces rôles peuvent utiliser ce scope.



Autoriser les accès avec oAuth2

Jeton d’accès et consentement
Limitations des accès
Personnalisation du jeton
Validation du jeton



Validation du jeton

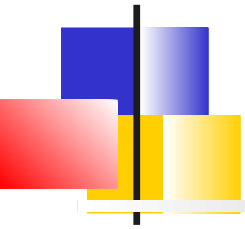
Introspection (RFC 7662) : appel du endpoint d'introspection

- Réservée à un client authentifié (confidentiel)
- Détecte en temps réel un jeton révoqué ou une session fermée
- Coût : une requête par validation (cache possible)

Validation locale du JWT : signature et claims (voir Validation d'un JWT)

- Aucun appel à Keycloak, très performant
- Le jeton reste valide jusqu'à son expiration, même après un logout

En pratique : validation locale et durées de vie courtes ; introspection pour les opérations sensibles ou au niveau d'une gateway



Sécurisation des différents types d’application

Application web

Application native ou mobile

REST APIs et services

Types d’application web

La sécurisation d’une application web dépend de l'architecture de l'application, on peut distinguer :

- Traditionnelle : L'application Web s'exécute entièrement à l'intérieur d'un serveur « backend ».
- SPA avec API dédiée : L'application s'exécute dans le navigateur et appelle uniquement une API dédiée sous le même domaine.
- SPA avec API intermédiaire : L'application SPA appelle des API externes via une API intermédiaire hébergée sous le même domaine
- SPA avec API externe : L'application SPA appelle des API hébergées sous un domaine différent.

Quelque soit l’architecture le meilleur mécanisme d’authentification d’un utilisateur reste l’*Authorization Code Flow*

Application traditionnelle

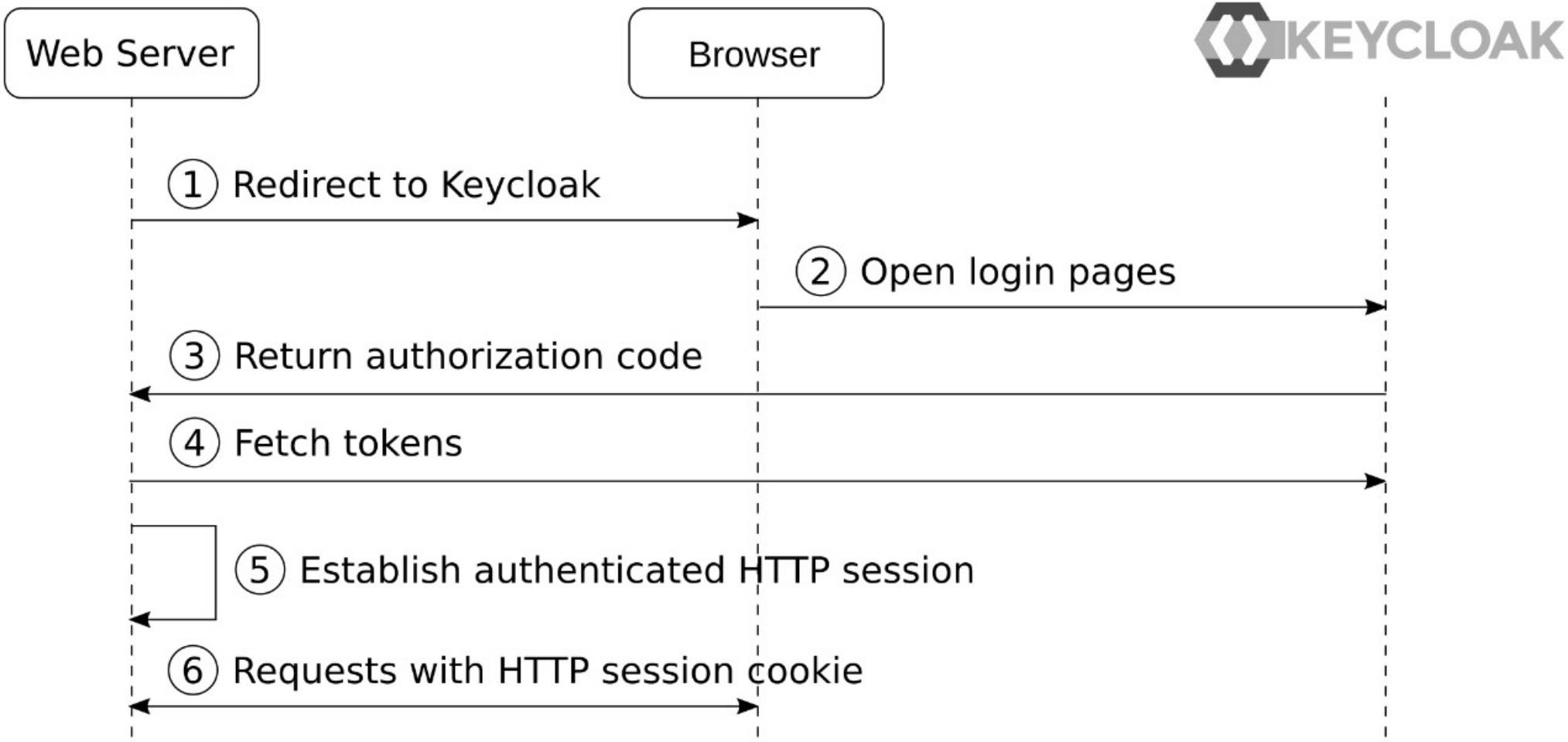
Avec ce type d’application, seul le jeton d'identification est utilisé pour établir une session HTTP.

On le configure afin qu’il contienne les informations nécessaires pour les ACLs de l’application.

Recommandations :

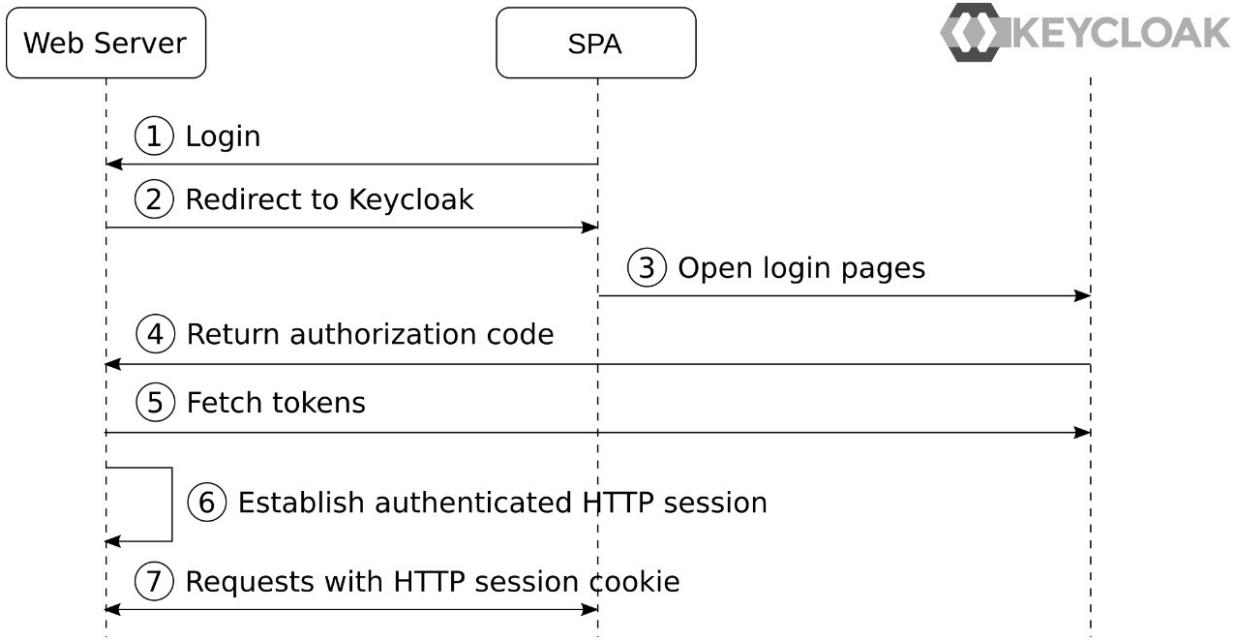
- Enregistrer un client confidentiel
  Pour obtenir le jeton, il faut alors le code + le secret du client stocké dans le back-end
- *PKCE* est recommandé pour plus de sécurité
- Donner des URLs de redirection strictes

Application traditionnelle

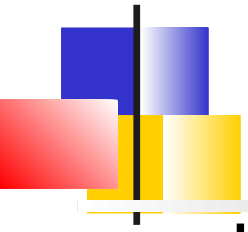


SPA avec API dédiée

- Identique au cas d'une appli web traditionnelle. Le backend fait le code Flow, et le SPA ne reçoit que le cookie.
- Scalabilité : session partagée (Redis…) ou cookie chiffré contenant l'intégralité de la session ; les API restent stateless



SPA avec API intermédiaire dédiée



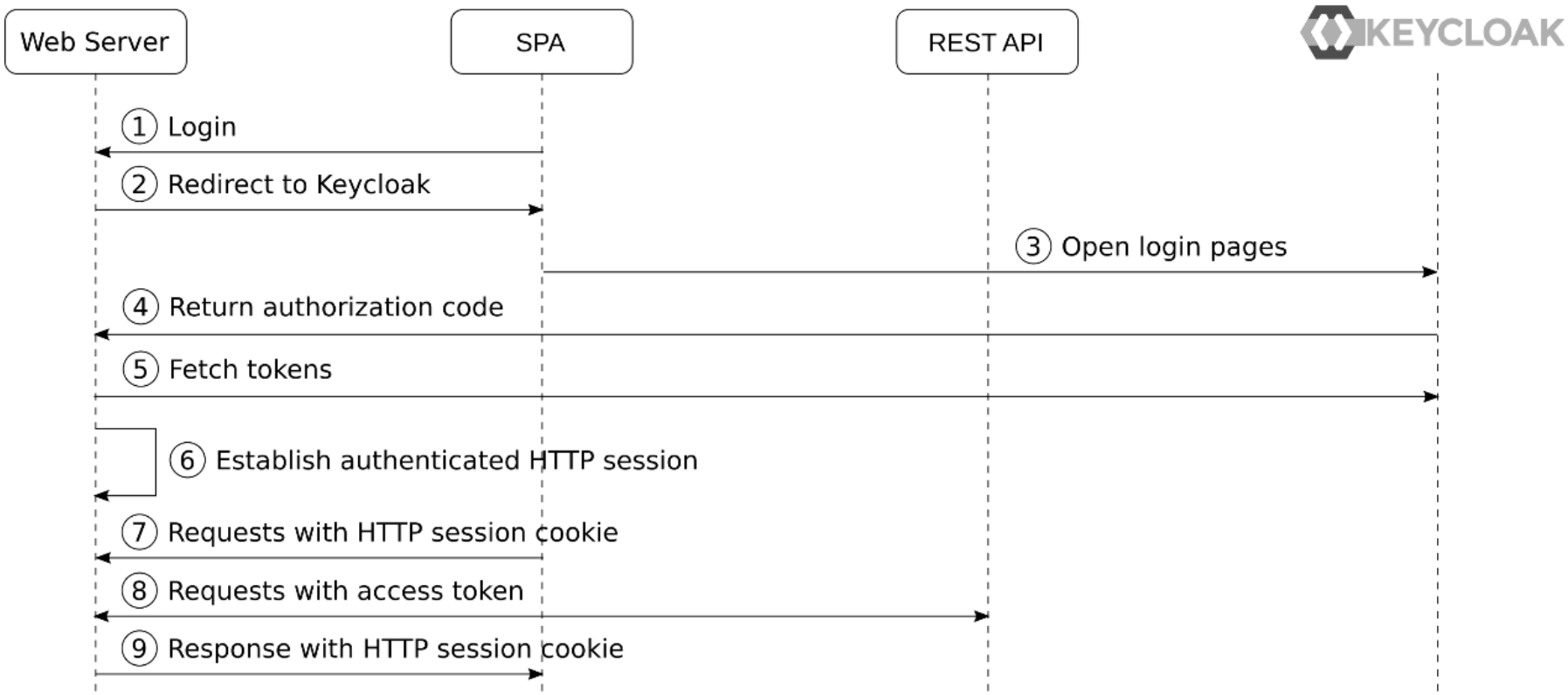
Le moyen le plus sûr d'invoquer des API externes à partir d'un SPA consiste à utiliser une API intermédiaire hébergée sur le même domaine que le SPA[1].

- Possibilité d’utiliser un client confidentiel et les jetons ne sont pas directement accessibles dans le navigateur.
- Pas besoin de mettre en place du CORS

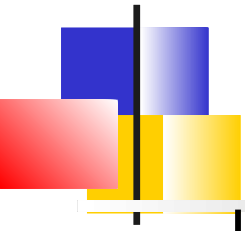
Les appels vers les APIs externes utilisent le jeton d’accès

1. Design patterns : Gateway, Backend for frontend

SPA avec API intermédiaire dédiée



SPA avec API externe



- Le moyen le plus simple est d'effectuer le flux *authorization-code* depuis le SPA avec un client public enregistré dans Keycloak.
  - L'approche est moins sécurisée car les jetons, y compris le jeton d'actualisation, sont exposés directement au navigateur.

=> Avoir une courte expiration pour le jeton d'actualisation

=> Révoquer les jetons d'actualisation après leur 1ère utilisation.

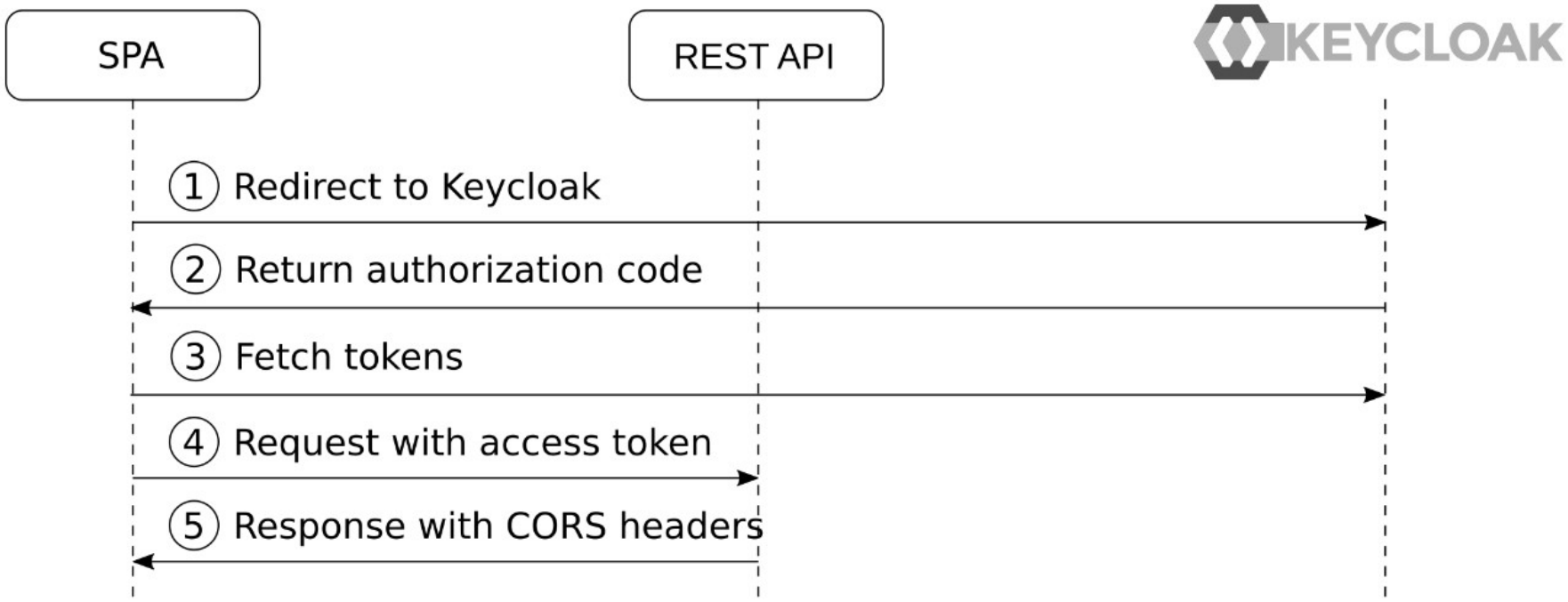
=> Utiliser l'extension PKCE

=> Stocker les jetons dans l'état de la fenêtre ou la session de stockage HTML5 n'est pas recommandée. On garde les jetons en mémoire

=> Protéger le SPA contre le Cross-Site Scripting (XSS)[1]

1. Injection de script malicieux

SPA avec API externe

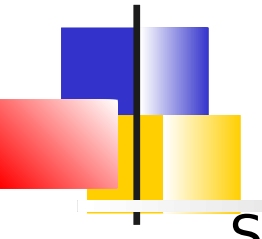


Sécurisation des différents types d’application

Application web

Application native ou mobile

REST APIs et services



Introduction

Sécuriser une application web avec Keycloak est plus simple que de sécuriser une application native ou mobile.

- Les pages de connexion étant affichées dans le navigateur.

On pourrait être tenté d'implémenter une page de connexion dans l'application et utiliser le password grant flow, mais mauvaise idée :

- L'application ne doit pas avoir accès aux mots de passe
- Certaines fonctionnalités de Keycloak ne seraient plus disponibles comme le MFA, le SSO ou la gestion de session

=> Le flow recommandé est donc **Authorization Code flow avec l'extension PKCE**

=> Sur un appareil sans navigateur (TV, CLI sur un serveur distant) : **Device Code Flow**

Authentification



Dans une application mobile ou native, l'utilisateur s'authentifie toujours via un navigateur (RFC 8252, OAuth 2.0 for Native Apps)

Vue Web intégrée (WebView) : à éviter

- Facile à intégrer mais peu sûre : l'application peut intercepter les identifiants
- Pas de SSO : cookies non partagés avec le navigateur

Navigateur intégré sécurisé (in-app browser) : recommandé

- Custom Tabs (Android), ASWebAuthenticationSession (iOS)
- Isolé de l'application, partage la session du navigateur système : SSO possible

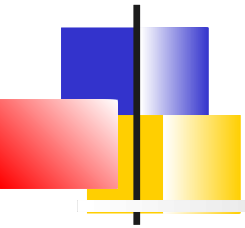
Navigateur externe (navigateur par défaut du système)

- Très sûr : authentification hors de l'application
- Moins fluide : bascule visible vers le navigateur

Authorization Code avec PKCE

Authorization Code avec PKCE est aujourd'hui le standard recommandé pour les applications mobiles ou desktop.

1) L'application génère un code_verifier aléatoire et en dérive le code_challenge.
2) Elle ouvre le navigateur sécurisé sur Keycloak, en envoyant le code_challenge dans la requête d'autorisation.
3) L'utilisateur s'authentifie sur la page de connexion de Keycloak.
4) Keycloak redirige vers l'application (claimed HTTPS, schéma personnalisé ou loopback) avec un code d'autorisation.
5) L'application échange ce code contre les jetons, en envoyant le code_verifier.
6) Keycloak vérifie que le code_verifier correspond au code_challenge reçu à l'étape 2.
7) Keycloak renvoie les jetons à l'application.



Renvoi du code : mobile

Pour renvoyer le code d'autorisation à l'application, OAuth 2.0 définit des URI de redirection adaptées ; le choix dépend de la plateforme

Claimed HTTPS scheme : App Links (Android) / Universal Links (iOS)

- L'app revendique un domaine HTTPS : l'OS ouvre l'URI dans l'application au lieu du navigateur système
- **Pourquoi la plus sûre ?** L'OS vérifie que l'app appartient au propriétaire du domaine (fichier publié sur le domaine)

Schéma d'URI personnalisé : enregistré avec l'application, qui reçoit la redirection de Keycloak

- Inverse d'un domaine du fournisseur, par exemple org.acme.app://oauth2/provider-name
- Plus simple à configurer mais moins sécurisé : une autre app peut revendiquer le même schéma et intercepter le code ; PKCE rend ce code inutilisable



Renvoi du code : desktop

Interface de loopback : l'application ouvre un serveur Web temporaire sur l'interface de loopback

- URI de redirection enregistrée : http://127.0.0.1/oauth2/provider-name
- Port choisi à l'exécution : Keycloak ignore le port d'une URI de loopback
- Recommandé pour les applications desktop et CLI

URI de redirection spéciale urn:ietf:wg:oauth:2.0:oob : le code est affiché par Keycloak, l'utilisateur le copie/colle dans l'application

- Déconseillé : méthode obsolète et non recommandée par les spécifications récentes d'OAuth

Et sans navigateur sur l'appareil ? Aucune redirection n'est possible : c'est le cas du Device Code Flow

Device Code Flow

Pourquoi ? Pas de navigateur sur l'appareil, ou une saisie pénible : le navigateur est sur un second appareil (voir Flows sans navigateur)

Le ***Device Code Flow*** permet à une application sans navigateur intégré (ex. Smart TV, CLI) d'obtenir des jetons via un second appareil avec navigateur.

1) L'application effectue un POST sur le *device_authorization_endpoint*. Keycloak répond avec un ***code utilisateur*** et un ***code device.***

2) L'utilisateur saisit le code utilisateur sur un endpoint de keycloak (*verification_uri*) en utilisant un autre device avec navigateur

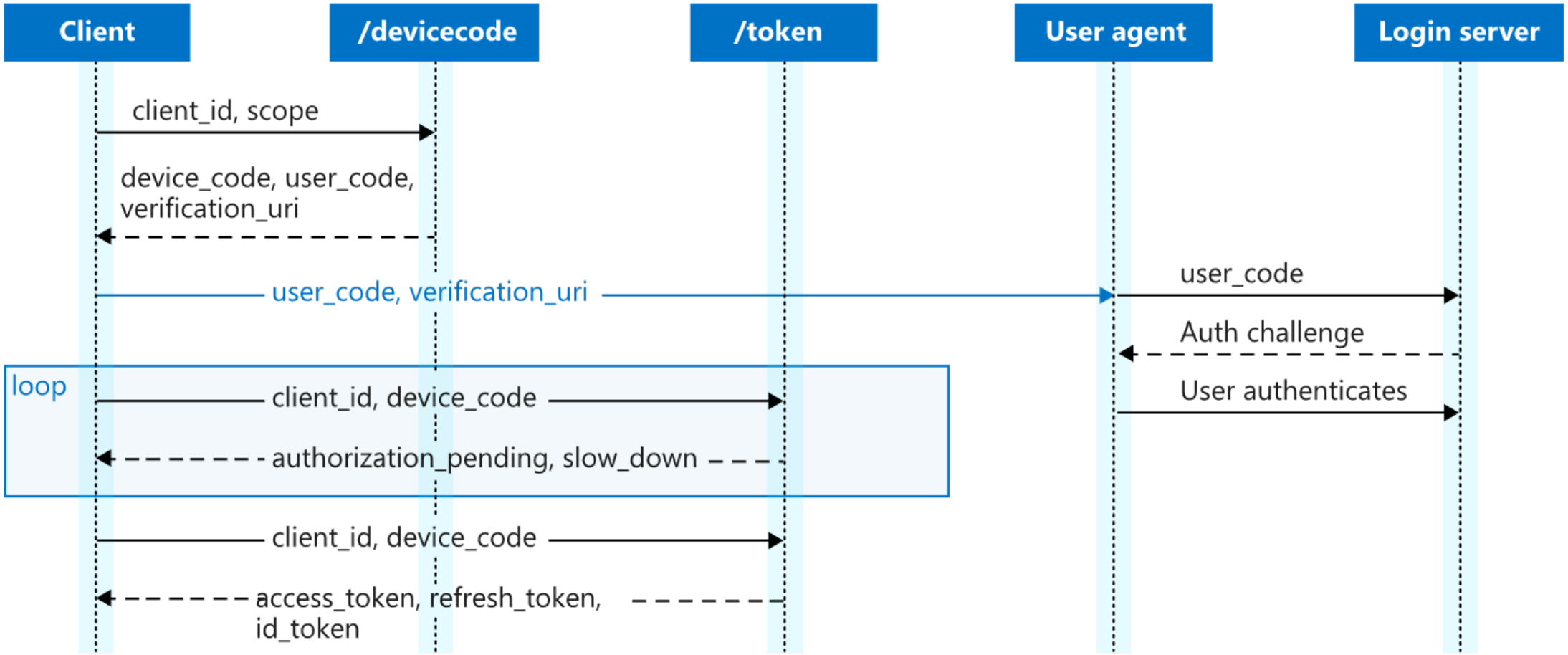
3) Après avoir entré le code, l'utilisateur s'authentifie et consent

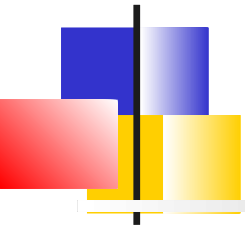
4) L'application est ensuite capable de récupérer les jetons en utilisant son device code

Ce flow n'utilise pas de redirection et ne supporte pas le SSO.

TP 4.1.2

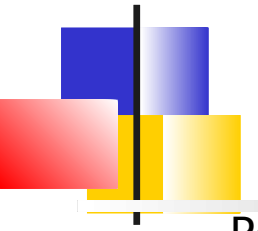
Device Code Flow





Sécurisation des différents types d’application

Application web
Application native ou mobile
REST APIs et services



Introduction

Pour appeler une API protégée par OAuth 2.0, l'application obtient un jeton d'accès auprès de Keycloak, puis l'envoie dans l'en-tête Authorization

Dans une architecture micro-services, 3 approches pour les appels entre services :

Propagation du jeton : le même jeton est relayé de service en service

- Simple, l'identité de l'utilisateur est conservée
- Un seul contexte de sécurité : le jeton doit être accepté par tous les services (audience large)

Client credentials : chaque service obtient son propre jeton

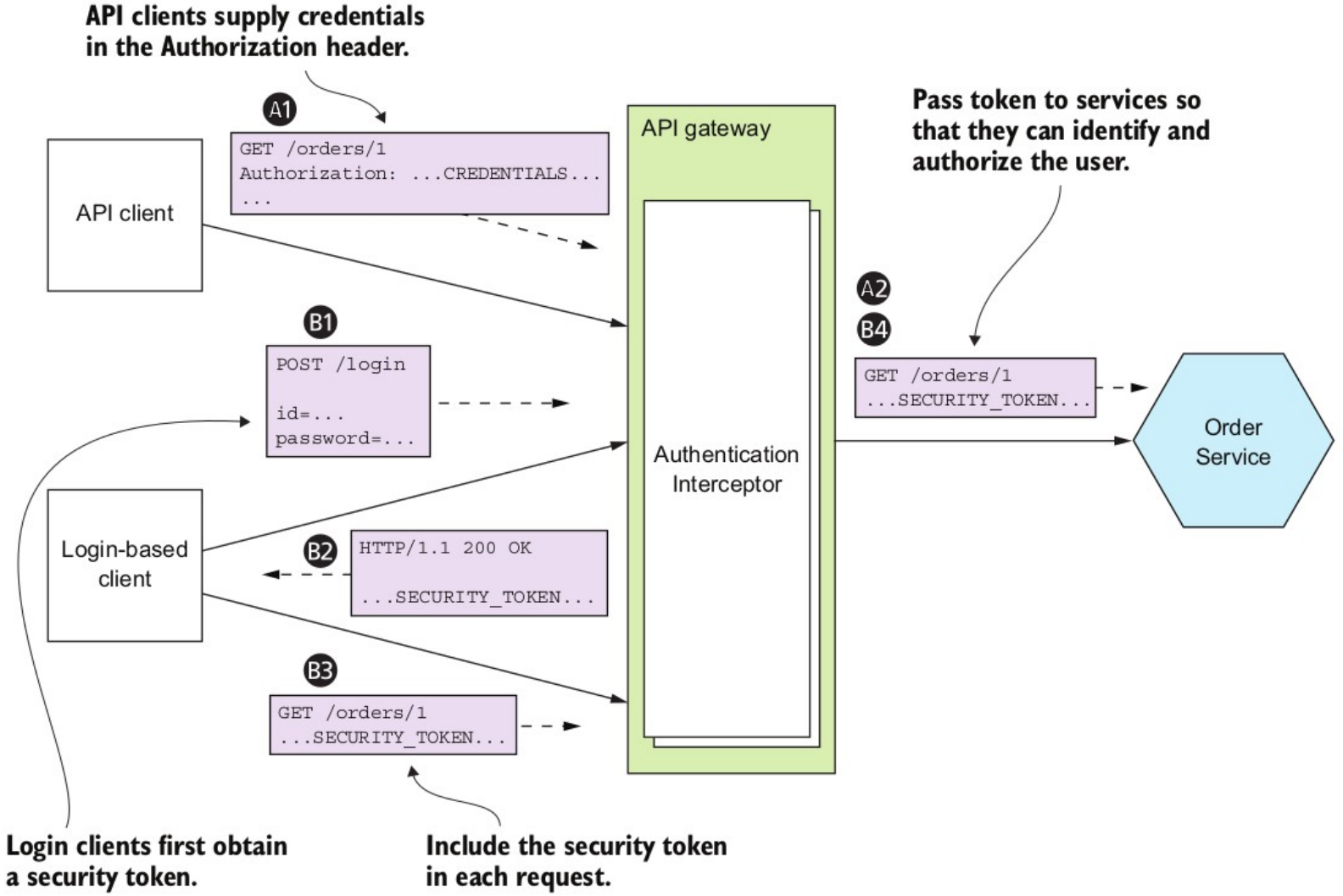
- Chaque interaction a son propre contexte de sécurité
- L'identité de l'utilisateur est perdue

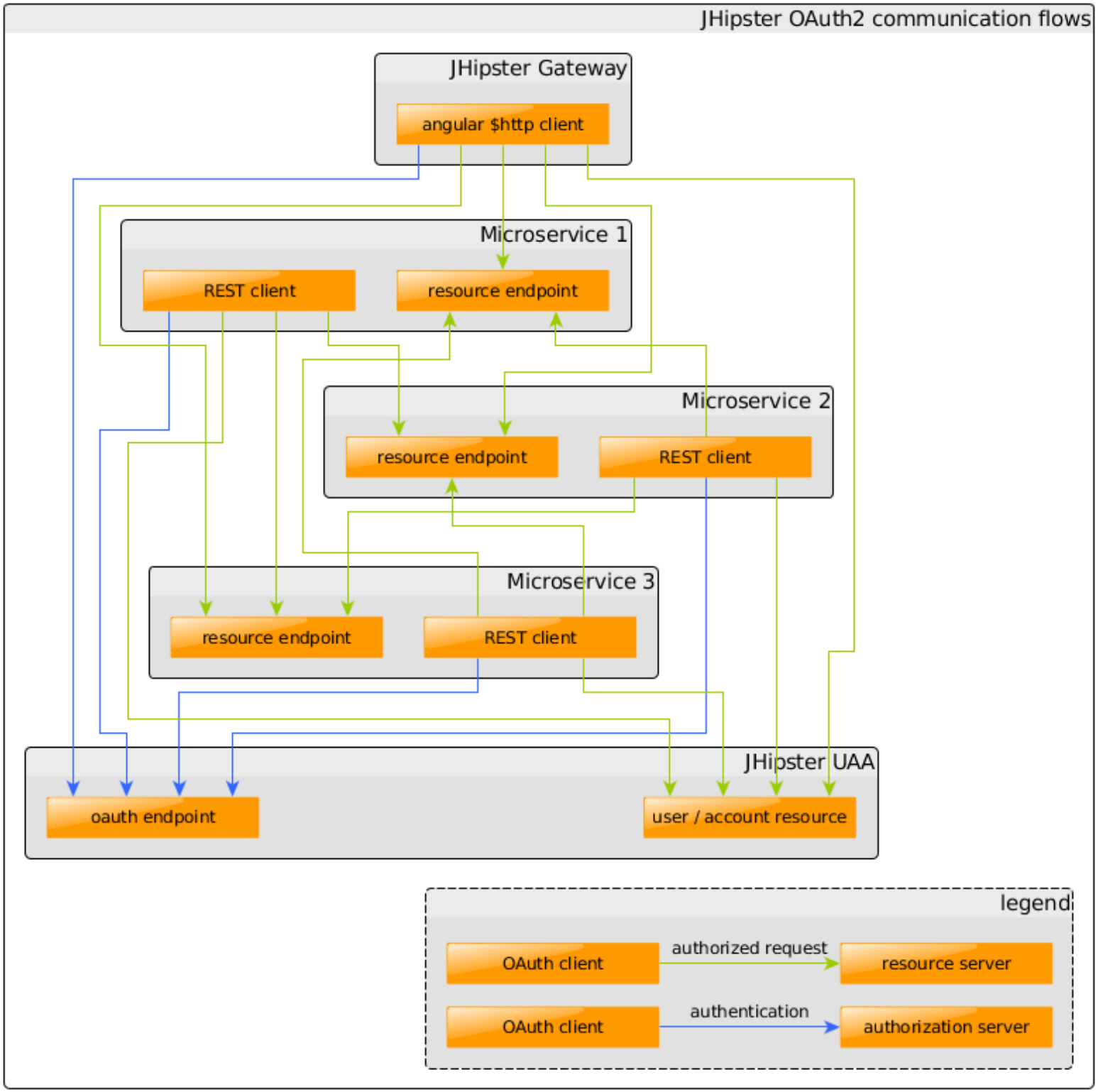
Token exchange (RFC 8693) : le service échange le jeton reçu contre un jeton destiné au service appelé

- Conserve l'identité de l'utilisateur avec une audience restreinte
- Supporté en standard depuis Keycloak 26.2

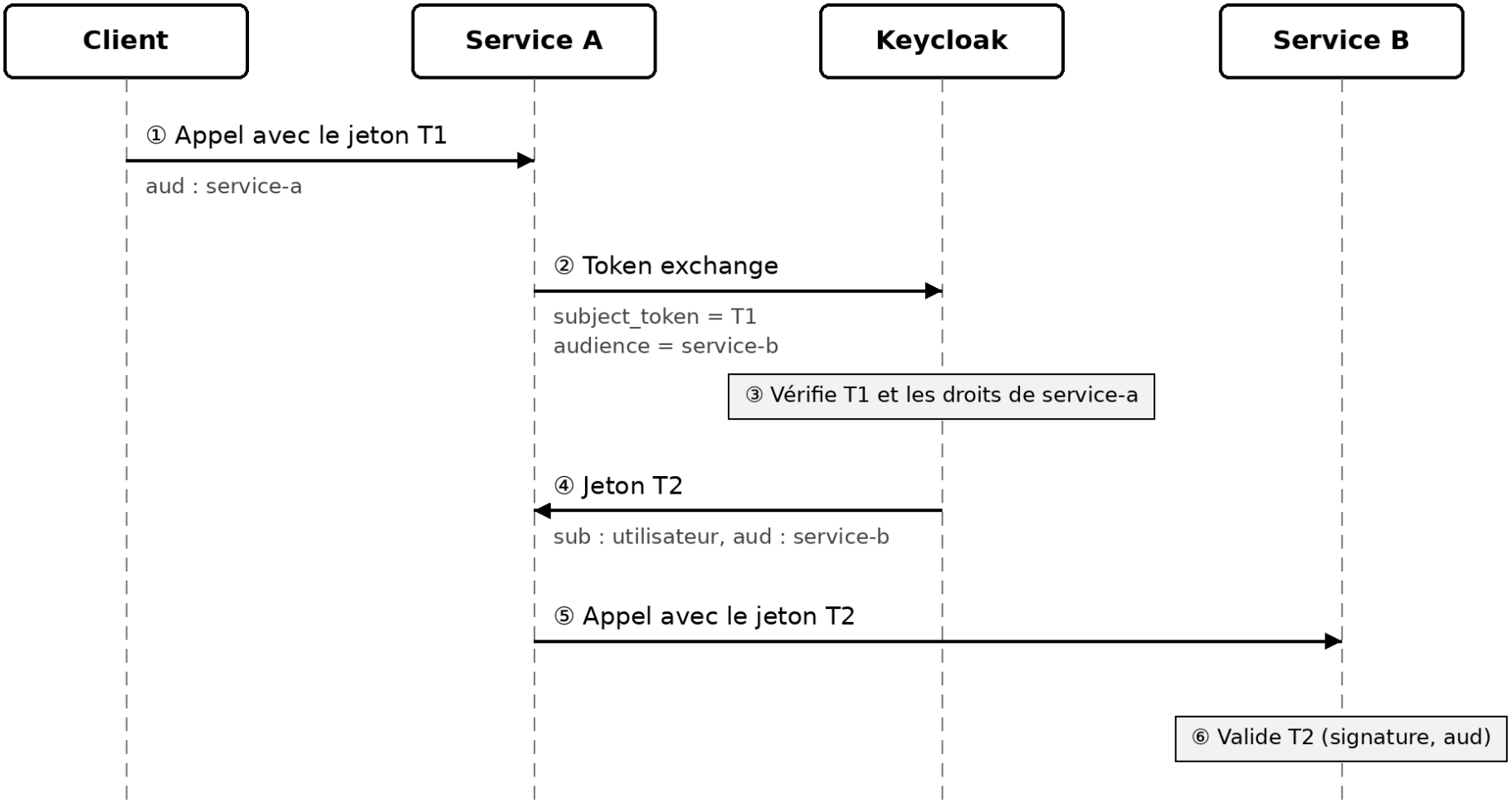
Access Token Pattern

La gateway relaie le jeton d'accès




JHipster OAuth2 communication flows
JHipster Gateway
angular $http client
Microservice 1
REST client
resource endpoint
Microservice 2
resource endpoint
REST client
Microservice 3
resource endpoint
REST client
JHipster UAA
oauth endpoint
user / account resource
legend
OAuth client
authorized request
resource server
OAuth client
authentication
authorization server

Token Exchange



Token Exchange dans Keycloak

- Standard Token Exchange (RFC 8693), supporté depuis Keycloak 26.2
- Le service A échange le jeton reçu contre un jeton destiné au service B :
  - Client service-a : confidentiel, capability Standard Token Exchange
  - Le jeton reçu doit contenir service-a dans son audience

```
POST /realms/{realm}/protocol/openid-connect/token
grant_type=urn:ietf:params:oauth:grant-type:token-exchange
&client_id=service-a&client_secret=…
&subject_token={jeton reçu}
&subject_token_type=urn:ietf:params:oauth:token-type:access_token
&audience=service-b
```

- Le nouveau jeton conserve l’utilisateur (sub), avec une audience restreinte à service-b

JWT Authorization Grant

RFC 7523 : obtenir un jeton Keycloak en présentant un JWT signé par un émetteur externe de confiance, sans interaction de l'utilisateur »

- Cas d'usage : propagation d'identité entre domaines, combiné au Token Exchange

Configuration :

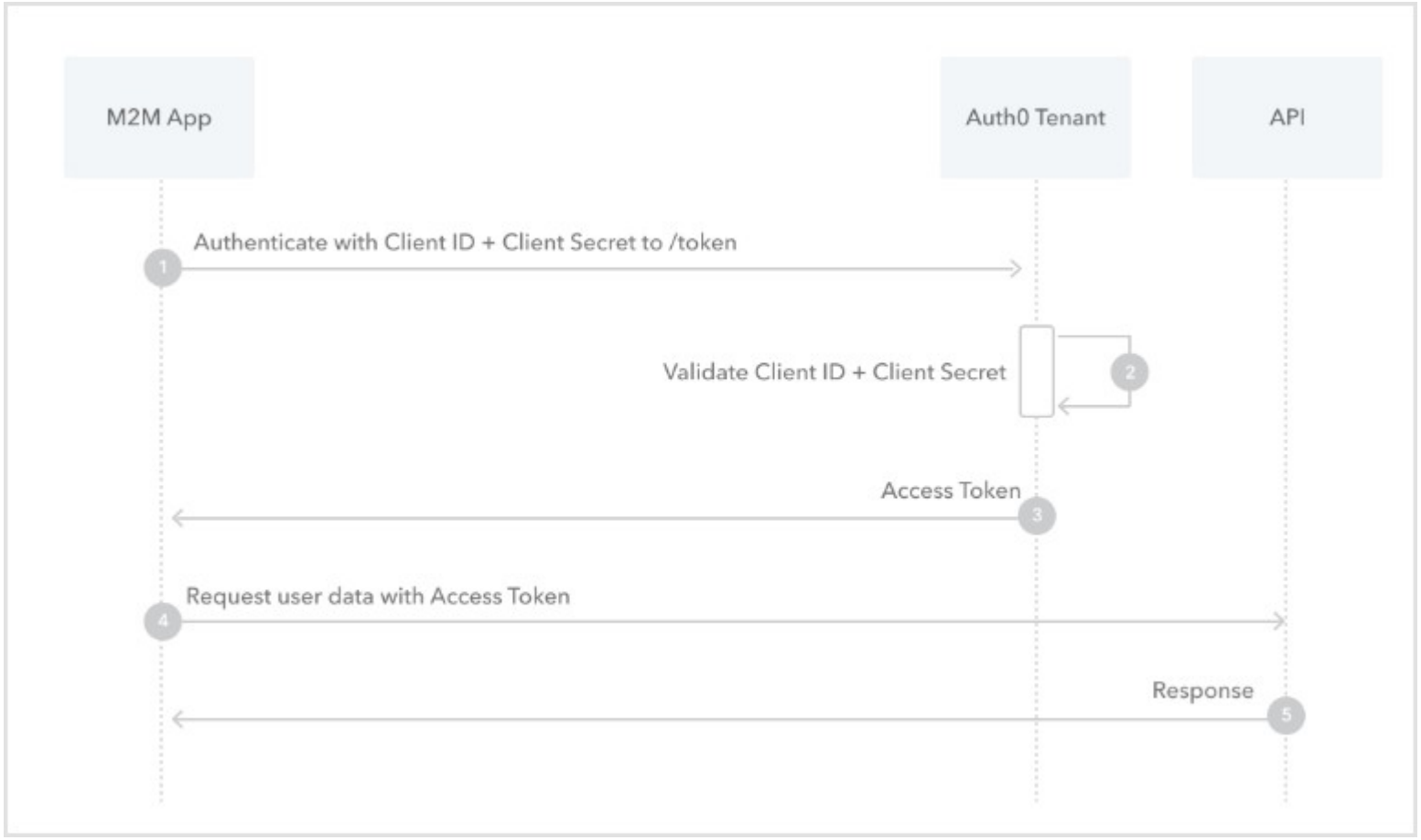
- *Identity providers → fournisseur OIDC → section Authorization Grant Settings*
- Client confidentiel, capability JWT Authorization Grant

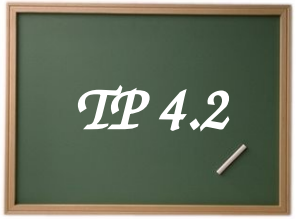
```
grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
&client_id=partner-app&client_secret=…
&assertion={JWT émis par le fournisseur externe représentant l'utilisateur}
```

Keycloak vérifie la signature, l'émetteur, l'audience et l'expiration du JWT

Introduit en 26.5, recommandé à la place du token exchange externe V1

Client Credentials Flow avec secret





Client credentials avec certificat X509

Dans ce cas, une application backend obtient sont jeton via le flow client credentials

Pour s'authentifier au lieu d'utiliser un secret il utilise son certificat X509

Configuration Keycloak

- Déclarer un client avec Client authentification et Service Account Roles
- Dans l'onglet credentials, définir X509 et indiquer le Subject DN complet du certificat

Client credentials avec JWT-signé

Permet à un client de s'authentifier en utilisant un JWT signé avec une clé privée

Génération du JWT

- Le client génère un JWT contenant des revendications (claims) spécifiques. généralement client_id, iss, sub, aud et iat.
- Le JWT est signé avec une clé privée détenue par le client. Le serveur vérifie l'intégrité et l'authenticité avec la clé publique correspondante.

Envoi du JWT :

- Le client envoie le JWT signé dans le cadre d'une requête d'authentification en utilisant généralement le champ client_assertion dans une requête de token.

Validation du JWT :

- Le serveur valide le JWT en vérifiant la signature avec la clé publique du client.
- Il vérifie également les revendications pour s'assurer que le JWT est valide et n'a pas expiré.
- Si le JWT est valide, le serveur d'autorisation procède à l'authentification du client et peut émettre un token d'accès.

Mise en œuvre dans Keycloak

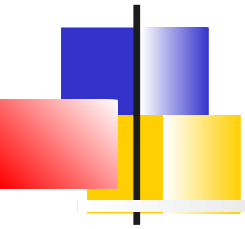
Dans l'onglet *Credentials*, sélectionner ***Signed JWT*** ou *Signed JWT with secret*

Dans l'onglet *Keys*, ou peut générer ou importer une paire clé privé/clé publique.

Dans l'application cliente, générer un JWT et le signer avec la clé privée puis :

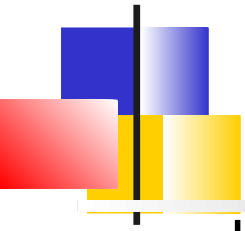
```
POST /realms/{realm-name}/protocol/openid-connect/token HTTP/1.1
Host: keycloak.example.com
Content-Type: application/x-www-form-urlencoded


grant_type=client_credentials
&client_id={client-id}
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion={signed-jwt}
```



Intégration KeyCloak

Introduction
Keycloak JS
Spring Boot et Quarkus
Python
Reverse Proxy
Débogage



Introduction

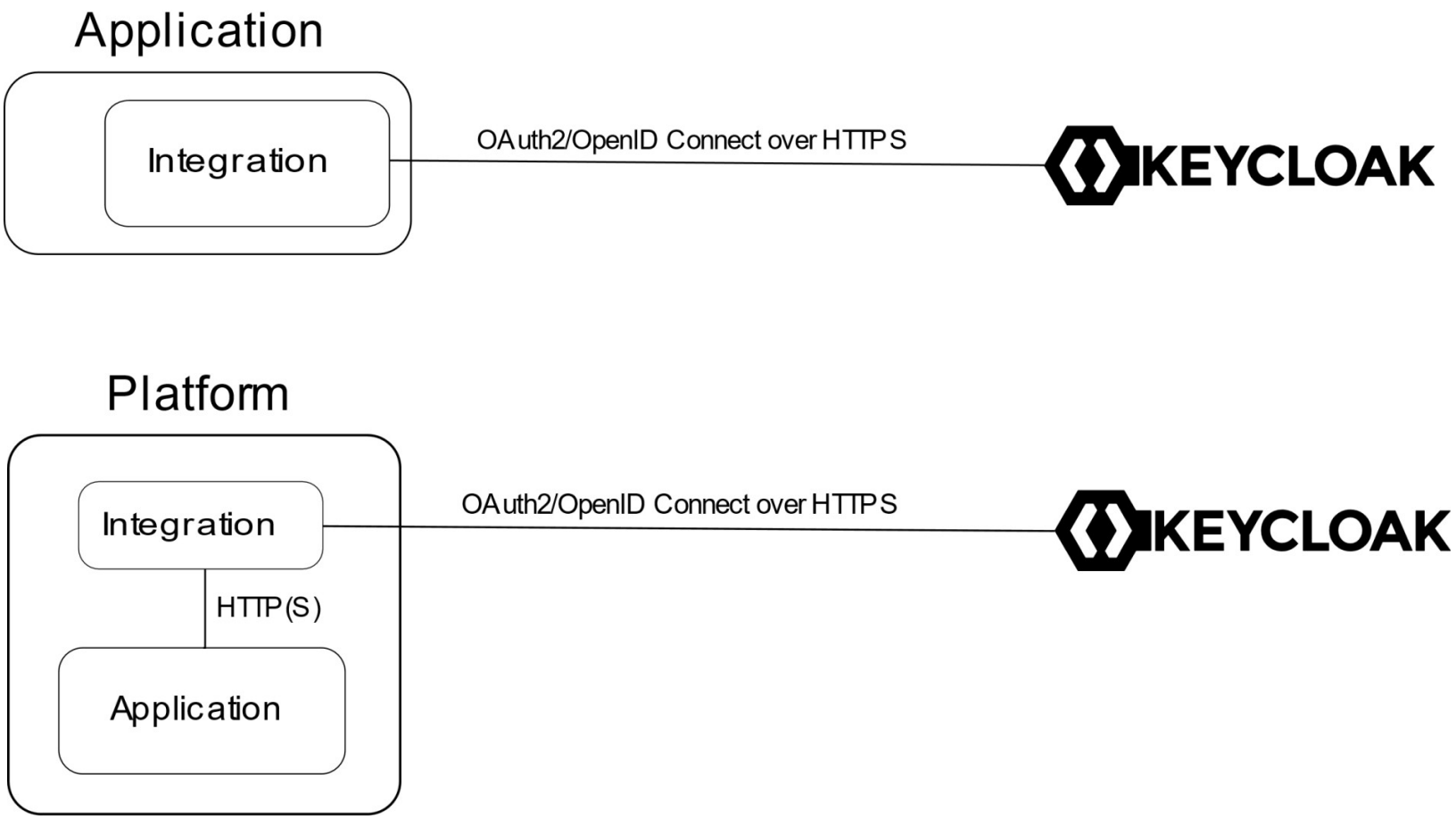
L’application délègue l’authentification à Keycloak ; 3 acteurs interagissent :

- Le navigateur de l’utilisateur : redirigé vers Keycloak pour s’authentifier
- Le code d’intégration : échange le code contre les jetons, valide les jetons
- Keycloak : authentifie l’utilisateur et émet les jetons

2 styles d’intégration, selon l’emplacement de ce code :

- **Embarqué** : librairie du langage ou du framework, dans l’application
- **Proxy** : reverse proxy devant l’application, seul à dialoguer avec Keycloak (code legacy, gateway micro-services)

Embarqué ou Proxy





Librairies et adaptateurs

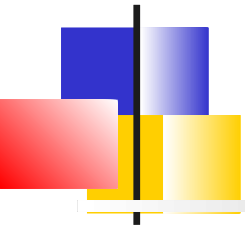
OpenID Connect et OAuth 2.0 sont supportés nativement par la plupart des frameworks :

- Java : *Spring Security, Quarkus OIDC, Nimbus*
- JavaScript : *openid-client, oauth4webapi, oidc-client-ts*
- Python : *Authlib* ; .NET : *ASP.NET Core OpenIdConnect* ; Go : *go-oidc*
- Apache : *mod_auth_openidc*

Librairies certifiées : https://openid.net/developers/certified/

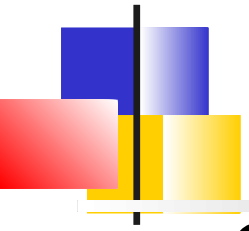
Adaptateurs Keycloak :

- Java et Node.js : plus maintenus, remplacés par le support natif des frameworks
- *keycloak-js* : seul adaptateur maintenu, publié comme projet indépendant



Intégration KeyCloak

Introduction
Keycloak JS
Spring Boot et Quarkus
Python
Reverse Proxy
Débogage



Adaptateur JavaScript

- Cas d’usage : applications exécutées dans le navigateur (SPA React, Angular, Vue…) sans backend chargé de l’authentification
  - Client public : pas de secret, Authorization Code Flow avec PKCE
  - Les jetons sont détenus par le navigateur
- Installation : **npm install keycloak-js**

```
import Keycloak from "keycloak-js";
const keycloak = new Keycloak({
  url: "https://auth.example.com", realm: "myrealm", clientId: "myapp"
});
await keycloak.init({ onLoad: "login-required", pkceMethod: "S256" });
```

Utilisation de l’adaptateur

Options de *init()* :

- *onLoad* : login-required (connexion forcée) ou check-sso (vérifie une session existante)
- *checkLoginIframe* : détection des logouts SSO, inopérante si les cookies tiers sont bloqués

updateToken(minValidity) : si le jeton expire dans moins de minValidity secondes, le rafraîchit avec le refresh token (échoue si la session Keycloak est fermée)

```
await keycloak.updateToken(30);
fetch("/api/data", {
  headers: { Authorization: "Bearer " + keycloak.token } });
```

Autres méthodes : *login(), logout(), hasRealmRole(), loadUserProfile()*

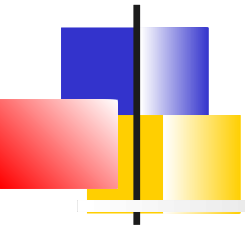


Exemple complet

```
import Keycloak from "keycloak-js";
const keycloak = new Keycloak({
  url: "https://auth.example.com", realm: "myrealm", clientId: "spa"
});
const authenticated = await keycloak.init({
  onLoad: "check-sso", pkceMethod: "S256" });
if (authenticated) {
  user.textContent = keycloak.tokenParsed.preferred_username;
  logoutBtn.onclick = () => keycloak.logout({ redirectUri: location.origin });
} else {
  loginBtn.onclick = () => keycloak.login();
}

async function callApi() {
  await keycloak.updateToken(30);
  const res = await fetch("/api/data", {
    headers: { Authorization: "Bearer " + keycloak.token } });
  return res.json();
}
```

Keycloak : client public, redirect URI et web origins de la SPA



Intégration KeyCloak

Introduction
Keycloak JS
Spring Boot et Quarkus
Python
Reverse Proxy
Débogage



Spring Boot 4 et Quarkus

Deux rôles possibles pour une application :

- **Application web** : client OIDC, Authorization Code Flow, session côté serveur
- **Resource server** (API) : validation du jeton Bearer

Starters Spring Boot 4 (Spring Security 7) :

- ***spring-boot-starter-security-oauth2-client*** : application web
- ***spring-boot-starter-security-oauth2-resource-server*** : resource server

Extensions Quarkus :

- ***quarkus-oidc*** : application web (web-app) ou resource server (service)
- ***quarkus-oidc-client, quarkus-rest-client-oidc-filter, quarkus-rest-client-oidc-token-propagation*** : appels sortants

Application web Spring

Configuration à partir de l'issuer (discovery) :

```
spring.security.oauth2.client:
  provider.keycloak.issuer-uri: https://auth.example.com/realms/myrealm
  registration.keycloak:
    client-id: spring-app
    client-secret: ${CLIENT_SECRET}
    scope: openid, profile, email
```

Redirect URI par défaut : {baseUrl}/login/oauth2/code/keycloak

Chaîne de sécurité Spring

```
@Bean
SecurityFilterChain web(HttpSecurity http) throws Exception {
  return http
    .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
    .oauth2Login(withDefaults())
    .build();
}
```

Déclenchement : une requête sans session sur une ressource protégée est redirigée vers /oauth2/authorization/keycloak, qui lance l'Authorization Code Flow

Accès à l'utilisateur connecté :

```
@GetMapping("/me")
String me(@AuthenticationPrincipal OidcUser user) {
  return user.getPreferredUsername();
}
```

Application web Quarkus

```
quarkus.oidc.auth-server-url=https://auth.example.com/realms/myrealm
quarkus.oidc.client-id=quarkus-app
quarkus.oidc.credentials.secret=${CLIENT_SECRET}
quarkus.oidc.application-type=web-app
```

Déclenchement : un accès sans session à une ressource protégée (*@Authenticated*, *@RolesAllowed* ou permission HTTP) redirige vers Keycloak

```
@Inject @IdToken JsonWebToken idToken;

@GET @Authenticated
public String me() {
  return idToken.getClaim("preferred_username");
}
```

Logout : mécanisme

Le logout d'une application web combine 3 étapes :

- **Logout local** : l'application invalide sa session HTTP
- **RP-Initiated Logout** : redirection du navigateur vers l'*end_session_endpoint* avec *id_token_hint* ; Keycloak ferme la session SSO puis redirige vers *post_logout_redirect_uri*
- **Back-channel** : Keycloak notifie les autres applications de la session SSO, qui invalident la session associée au sid

Un appel serveur à l'*end_session_endpoint* ne suffit pas : le cookie de session Keycloak du navigateur ne serait pas supprimé

Logout : mise en œuvre

Spring Boot 4

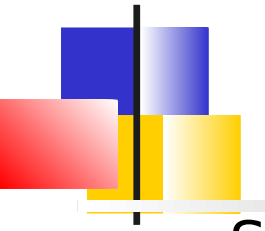
```
http.logout(l -> l.logoutSuccessHandler(oidcLogoutSuccessHandler))
    .oidcLogout(l -> l.backChannel(withDefaults()));
```

oidcLogoutSuccessHandler : OidcClientInitiatedLogoutSuccessHandler ; back-channel sur /logout/connect/back-channel/{registrationId}

Quarkus

```
quarkus.oidc.logout.path=/logout
quarkus.oidc.logout.post-logout-path=/
quarkus.oidc.logout.backchannel.path=/back-channel-logout
```

Dans les deux cas, déclarer l'URL de back-channel dans le client Keycloak



Resource server

Spring Boot 4 :

```
spring.security.oauth2.resourceserver.jwt:
  issuer-uri: https://auth.example.com/realms/myrealm
  audiences: my-api
```

Quarkus :

```
quarkus.oidc.auth-server-url=https://auth.example.com/realms/myrealm
quarkus.oidc.application-type=service
quarkus.oidc.token.audience=my-api
```

Signature, iss et exp toujours vérifiés ; audience vérifiée seulement si configurée

- Optionnelle mais recommandée : sinon, un jeton émis pour un autre service est accepté
- Quarkus : propriété au singulier, qui accepte une liste de valeurs

Mapping des rôles

Les rôles Keycloak (realm_access.roles) doivent être traduits dans le modèle de sécurité du framework

Spring : autorités SCOPE_xxx par défaut, il faut écrire un convertisseur

```
@Bean
JwtAuthenticationConverter jwtAuthenticationConverter() {
  var converter = new JwtAuthenticationConverter();
  converter.setJwtGrantedAuthoritiesConverter(jwt -> {
    Map<String, List<String>> realm = jwt.getClaim("realm_access");
    return realm.get("roles").stream()
        .map(r -> (GrantedAuthority) new SimpleGrantedAuthority("ROLE_" + r))
        .toList();
  });
  return converter;
}
```

Quarkus : intégration naturelle, rôles lus dans groups puis realm_access/roles

- Configurable : quarkus.oidc.roles.role-claim-path

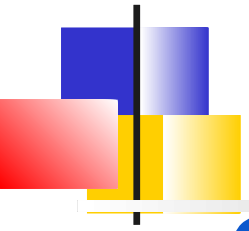
ACLs sur les endpoints

- Une fois les rôles mappés, les ACLs s’expriment avec le modèle standard du framework
- Spring Boot 4 :

```
http.authorizeHttpRequests(auth -> auth
      .requestMatchers(GET, "/api/**").hasAuthority("SCOPE_read")
      .requestMatchers("/admin/**").hasRole("admin")
      .anyRequest().authenticated())
    .oauth2ResourceServer(o -> o.jwt(withDefaults()));
```

- Quarkus : annotations Jakarta (@RolesAllowed, @Authenticated, @PermitAll)

```
@GET @RolesAllowed("admin")
public String admin() { return identity.getPrincipal().getName(); }
```



Appels sortants : Quarkus

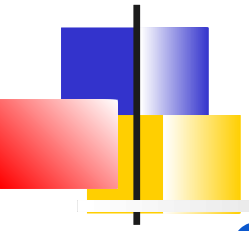
Client credentials : le service obtient son propre jeton (quarkus-rest-client-oidc-filter)

```
quarkus.oidc-client.auth-server-url=https://auth.example.com/realms/myrealm
quarkus.oidc-client.client-id=order-service
quarkus.oidc-client.credentials.secret=${CLIENT_SECRET}

@RegisterRestClient @OidcClientFilter
public interface StockService { @GET String stock(); }
```

Propagation : le jeton reçu est transmis (quarkus-rest-client-oidc-token-propagation)

```
@RegisterRestClient @AccessToken
public interface StockService { @GET String stock(); }
```



Appels sortants : Spring

Client credentials : enregistrement d'un client OAuth2 dédié

```
spring.security.oauth2.client.registration.stock:
  provider: keycloak
  client-id: order-service
  client-secret: ${CLIENT_SECRET}
  authorization-grant-type: client_credentials
```

RestClient avec l'intercepteur OAuth2 :

```
var interceptor = new OAuth2ClientHttpRequestInterceptor(authorizedClientManager);
interceptor.setClientRegistrationIdResolver(request -> "stock");
RestClient client = builder.requestInterceptor(interceptor).build();
```

Propagation : récupérer le jeton reçu (JwtAuthenticationToken) et le placer dans l'en-tête Authorization

TP
5.2 /5.3

DevServices et *KeyCloak*

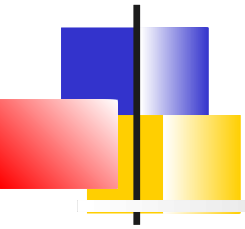
Quarkus propose une auto-configuration pour *KeyCloak*,
=> Pas besoin d'installer Keycloak sur son poste

Activé si

- *quarkus-oidc*
- dev et test mode
- La propriété *quarkus.oidc.auth-server-url* n'est pas configuré

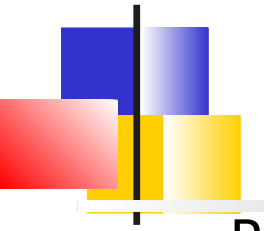
Le service démarre un container *Keycloak* et l'initialise en important ou créant un realm.
Le realm peut-être spécifié par :
`quarkus.keycloak.devservices.realm-path=quarkus-realm.json`

De plus, la Dev UI (*/q/dev*) permet d'acquérir les jetons de Keycloak et de tester l'application Quarkus.



Intégration KeyCloak

Introduction
Keycloak JS
Spring Boot et Quarkus
Python
Reverse Proxy
Débogage



Écosystème Python

Pas d'adaptateur officiel : on utilise des librairies OpenID / OAuth2 standards

Authlib : client OAuth2 / OIDC, intégrations Flask, Django, Starlette / FastAPI

PyJWT : décodage et validation de JWT, client JWKS intégré

mozilla-django-oidc : authentification OIDC pour Django

python-keycloak : accès aux endpoints OIDC et à l'API d'administration

Mêmes principes qu'en Java :

- Application web : Authorization Code Flow, session côté serveur
- API : validation du jeton Bearer (signature, iss, aud, exp)

Application web Flask avec Authlib

```
oauth = OAuth(app)
oauth.register("keycloak",
  client_id="flask-app", client_secret="...",
  server_metadata_url="https://kc/realms/myrealm/.well-known/openid-configuration",
  client_kwargs={"scope": "openid profile email"})

@app.route("/login")
def login():
  return oauth.keycloak.authorize_redirect(url_for("auth", _external=True))

@app.route("/auth")
def auth():
  token = oauth.keycloak.authorize_access_token()
  session["user"] = token["userinfo"]
  return redirect("/")
```

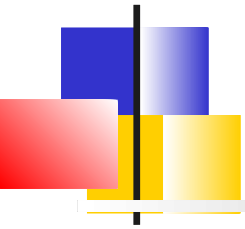


API FastAPI : validation du jeton

```
import jwt
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

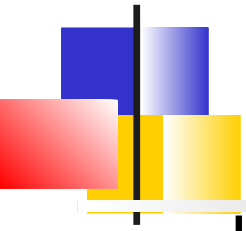
ISSUER = "https://kc/realms/myrealm"
jwks = jwt.PyJWKClient(ISSUER + "/protocol/openid-connect/certs")

def current_user(cred = Depends(HTTPBearer())):
  try:
    key = jwks.get_signing_key_from_jwt(cred.credentials)
    return jwt.decode(cred.credentials, key.key, algorithms=["RS256"],
                      audience="my-api", issuer=ISSUER)
  except jwt.PyJWTError:
    raise HTTPException(status_code=401)
```



Intégration KeyCloak

Introduction
Keycloak JS
Spring Boot et Quarkus
Python
Reverse Proxy
Débogage



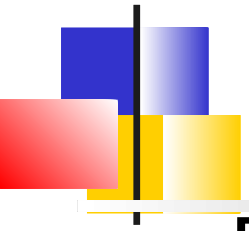
Reverse proxy

Le proxy gère l'Authorization Code Flow et la session ; l'application reçoit l'identité dans des en-têtes HTTP

Solutions :

- **Apache** : module mod_auth_openidc
- **Nginx** : pas de module OIDC en open source (natif dans NGINX Plus) : auth_request + oauth2-proxy
- **Traefik** : middleware ForwardAuth + oauth2-proxy (OIDC natif dans Traefik Hub)
- **oauth2-proxy** : proxy d'authentification dédié, provider keycloak-oidc

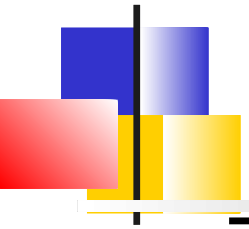
Côté Keycloak : client confidentiel, redirect URI du proxy, mapper Audience



Nginx

- Pas de module OIDC dans Nginx open source (module natif réservé à NGINX Plus)
- Solution courante : auth_request + oauth2-proxy

```
location /oauth2/ { proxy_pass http://oauth2-proxy:4180; }
location = /oauth2/auth {
  proxy_pass http://oauth2-proxy:4180;
  proxy_pass_request_body off;
  proxy_set_header Content-Length "";
}
location / {
  auth_request /oauth2/auth;
  error_page 401 =403 /oauth2/sign_in;
  auth_request_set $user $upstream_http_x_auth_request_user;
  proxy_set_header X-User $user;
  proxy_pass http://backend:8080;
}
```

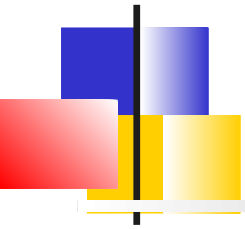


Traefik

- Traefik Proxy n’intègre pas OIDC : délégation via le middleware ForwardAuth

```
labels:
  - traefik.http.middlewares.oidc.forwardauth.address=
      http://oauth2-proxy:4180/oauth2/auth
  - traefik.http.middlewares.oidc.forwardauth.authResponseHeaders=
      X-Auth-Request-User,X-Auth-Request-Email
  - traefik.http.middlewares.oidc-err.errors.status=401
  - traefik.http.middlewares.oidc-err.errors.service=oauth2-proxy
  - traefik.http.middlewares.oidc-err.errors.query=/oauth2/sign_in
  - traefik.http.routers.app.middlewares=oidc-err,oidc
```

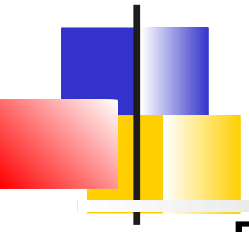
- Alternatives : plugins OIDC communautaires, middleware OIDC de Traefik Hub



Intégration KeyCloak

Introduction
Keycloak JS
Spring Boot et Quarkus
Python
Reverse Proxy
Débogage

Outils de diagnostic



Décoder les jetons : jwt.io ou en ligne de commande

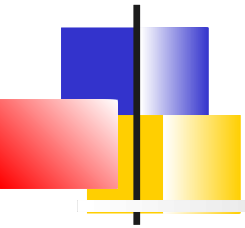
Client scopes → Evaluate : aperçu de l'access token, de l'ID token et de UserInfo générés pour un utilisateur

Events de login : Realm settings → Events → User events

- Type d'événement, client, IP et code d'erreur

Outils de développement du navigateur : paramètres de la redirection, code, state, cookies

curl sur les endpoints token, userinfo et introspect



Logs d’intégration

Côté Keycloak :

```
kc.sh start --log-level="INFO,org.keycloak.events:debug"
kc.sh start --log-level="INFO,org.keycloak.authentication:debug"
```

Le listener jboss-logging trace les erreurs de login en WARN et les succès en DEBUG

Côté applications :

```
# Spring Boot
logging.level.org.springframework.security=DEBUG
# Quarkus
quarkus.log.category."io.quarkus.oidc".level=DEBUG
# Python
logging.getLogger("authlib").setLevel(logging.DEBUG)
```

Erreurs fréquentes

Invalid parameter: redirect_uri : URI non déclarée dans le client (correspondance exacte)

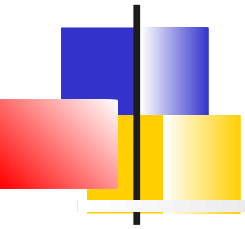
invalid_client / unauthorized_client : secret erroné, client public ou confidentiel, flow non activé

401 sur l'API :

- Claim aud absent : ajouter un mapper Audience
- iss différent : hostname vu par le navigateur ≠ hostname vu par le back-end (localhost vs nom de conteneur)
- Clé inconnue (kid) après rotation : cache JWKS à rafraîchir
- Horloges désynchronisées : exp, iat, nbf

Erreurs CORS : Web origins du client

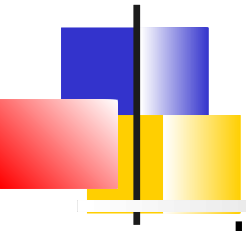
Boucle de redirection : cookies bloqués (SameSite, cookies tiers)



Stratégies d’autorisation

RBAC, GBAC, ABAC, scopes OAuth2

Authorization service



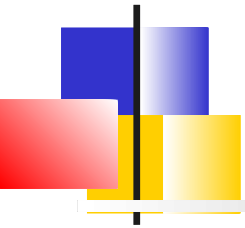
Introduction

Les données relatives à l'autorisation (l'utilisateur peut-il accéder à une ressource) sont extraites du jeton fourni par Keycloak : les revendications

- Cela peut être n'importe quelle revendication
- En général, on utilise les rôles, les groupes ou les scopes

2 stratégies pour implémenter l'autorisation :

- Implémenter le contrôle d'accès au niveau de l'application déclarativement ou programmatiquement
- Déléguer les décisions d'accès à un service externe



RBAC

RBAC (Role-Based Access Control) : protéger les ressources selon les rôles de l'utilisateur

Keycloak gère les rôles et les propage dans les jetons :

- Realm roles : rôle dans l'organisation (realm_access.roles)
- Client roles : rôle dans une application (resource_access.<client>.roles)

Pour éviter l'explosion du nombre de rôles, ne pas utiliser les rôles pour une autorisation fine

- Un rôle par action gonfle les jetons et l'administration : le fin relève d'ABAC ou de l'Authorization Service

Mécanismes facilitant la gestion :

- Rôle affecté à un groupe : tous les membres du groupe ont le rôle
- Rôle composite : englobe plusieurs rôles
- Rôles par défaut (default-roles-<realm>) : attribués à tout nouvel utilisateur



Groupes

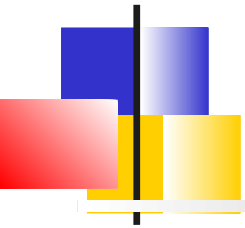
GBAC (Group-Based Access Control) : ACLs définies par rapport aux groupes

Les groupes représentent généralement le service d'un utilisateur ; hiérarchiques, ils reflètent l'organigramme

Un groupe peut porter des rôles et des attributs, hérités par ses membres

Groupes absents des jetons par défaut : mapper **Group Membership**

- Full group path activé : chemin complet, ex. /human resource/manager
- Désactivé : nom seul, ex. manager



Scopes OAuth2

Rôles : ce que peut l'utilisateur ; scopes : ce que peut le client en son nom

Objectif premier : protéger les données de l'utilisateur vis-à-vis des clients

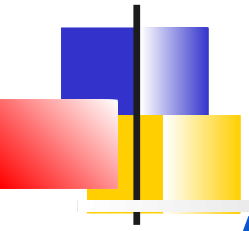
Les API peuvent donc définir des ACLs selon le client qui les appelle :

- Client interne ou externe
- Outil de monitoring ou service applicatif

Un outil de reporting lit, le back-office écrit : mêmes utilisateurs, droits différents

Combiner : un scope réservé à un rôle (Permissions des client scopes)

ABAC et stratégies personnalisées



ABAC (Attribute-Based Access Control) : décision fondée sur des attributs de l'utilisateur, de la ressource ou du contexte

Côté jeton : attributs utilisateur exposés par un mapper User Attribute (déclarés dans le User Profile)

Côté application : règles sur les claims

```
@PreAuthorize("principal.claims['department'] == 'hr'") // Spring
```

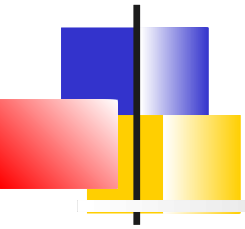
Côté Authorization Service de Keycloak : policies Regex, Time ou JavaScript

Quel modèle choisir ?

Les modèles se combinent : chacun répond à une question différente

Modèle	Décide selon	Granularité	Administré par	Limite
RBAC	les rôles de l'utilisateur	grossière, métier	admin Keycloak	explosion des rôles si trop fin
GBAC	l'appartenance aux groupes	organisationnelle	admin ou annuaire (LDAP)	dépend de l'organigramme
Scopes	les droits délégués au client	par API ou par action	développeurs, consentement	ne dit rien des droits de l'utilisateur
ABAC	attributs utilisateur, ressource, contexte	fine	code ou policies	règles dispersées dans le code

En pratique : RBAC ou GBAC pour le grossier, scopes pour le client appelant, ABAC ou un service externe pour le fin.



Stratégies d’autorisation

RBAC, GBAC, ABAC, scopes OAuth2

Authorization service

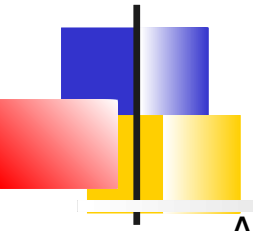
Introduction

L'autorisation centralisée permet d'externaliser la gestion des accès à l'aide d'un service d'autorisation externe.

Elle permet d'utiliser plusieurs mécanismes de contrôle d'accès sans y coupler l’application.

Keycloak peut agir comme un service d'autorisation centralisé via une fonctionnalité appelée ***Authorization Services***.

Introduction



Approche RBAC classique :

```
If (User.hasRole("manager") {
// can access the protected resource
}
```

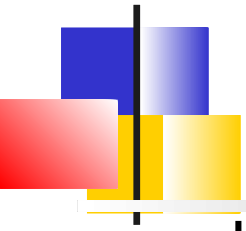
=> Si l'on veut changer les règles d'accès (ajout d'un nouveau rôle par exemple), il faut modifier le code et redéployer

Approche Autorisation centralisée

```
If (User.canAccess("Manager Resource") {
  // can access the protected resource
}
```

=> aucune référence à un mécanisme de contrôle d'accès spécifique ; le contrôle d'accès est basé sur la ressource.

=> Les modifications apportées à l'accès à *Manager Resource* se définissent dans Keycloak et n'ont pas d'impact sur le code applicatif



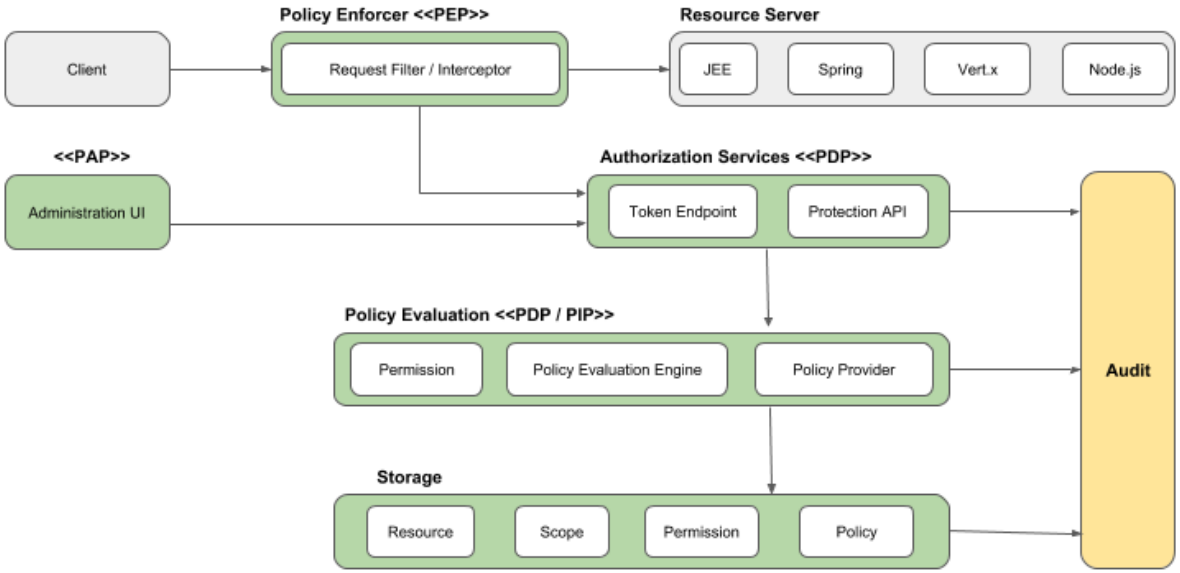
Types de policies

L'Authorization Service combine plusieurs mécanismes de contrôle d'accès via des policies :

- **Role**, **Group**, **User**, **Client**, **Client scope** : RBAC, GBAC, accès par utilisateur ou par client
- **Time** : contrôle par date ou plage horaire
- **Regex** : condition sur un attribut ou un claim
- **JavaScript** : règle libre, déployée sous forme de jar dans providers/ (feature scripts)
- **Aggregated** : combinaison de plusieurs policies

Stratégie de décision d'une permission : unanime, affirmative ou consensus

Architecture



Configuration

1) Définition des ressources (console ou API) :
   - **Resource server** = client avec Authorization activé
   - **Ressource** = motif d'URI ; scope (optionnel) = action sur la ressource
2) Définition des policies et des permissions :
   - **Policy** : condition à satisfaire
   - **Permission** : associe une ressource (et éventuellement des scopes) à des policies
3) Application des ACLs par un PEP (Policy Enforcement Point) dans le serveur de ressources :
   - Java : librairie keycloak-policy-enforcer ; Quar kus : quarkus-keycloak-authorization
   - Autres langages : appel du token endpoint avec le grant uma-ticket.



Policy Enforcement Point

Filtre ou intercepteur de l'application qui vérifie les ACLs auprès de Keycloak

Grant urn:ietf:params:oauth:grant-type:uma-ticket sur le token endpoint :

- À partir du jeton d'accès de l'utilisateur, avec l'audience et les permissions demandées
- Réponse : un RPT (Requesting Party Token), jeton d'accès enrichi du claim authorization.permissions
- Ou une simple décision (response_mode=decision)

Le PEP peut exiger un RPT ou évaluer les permissions à chaque requête

Ce grant vient de UMA 2.0 (User-Managed Access)

Limites et alternatives

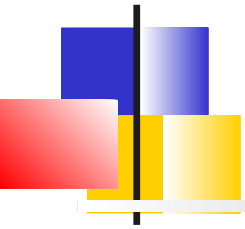
Limites de l'Authorization Service :

- Latence : appel à Keycloak à chaque décision, sauf cache ou RPT
- Configuration complexe, difficile à versionner et à tester
- Couplage fort à Keycloak

Alternatives d'autorisation externalisée :

- **OPA** (Open Policy Agent) : policies en langage Rego
- **OpenFGA** : modèle de relations inspiré de Google Zanzibar
- **Cedar** : langage de policies (AWS Verified Permissions)

Keycloak reste le fournisseur d'identité et des claims utilisés par les policies



Exploitation courante

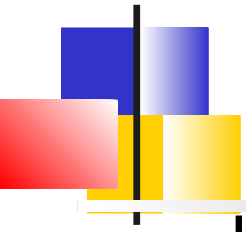
Gestion des utilisateurs

Mots de passe et MFA

Sessions et jetons

Événements et audit

Utilisateurs locaux et externes

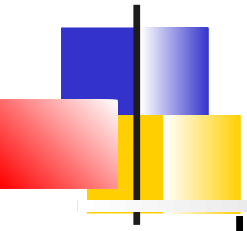


- Keycloak gère une base d'utilisateurs dans sa base de données. Les opérations de gestion concernent :
  - La création des utilisateurs et la gestion de leurs credentials
  - Les actions requises
  - L'éventuelle self-registration
  - Les attributs, définis par le User Profile
  - L'appartenance aux groupes, qui transmettent leurs rôles et attributs (voir stratégies d'autorisation)
- Keycloak peut aussi s'intégrer avec des systèmes externes, LDAP ou fournisseurs d'identité : fédération d'utilisateurs

Required User Actions

- Actions imposées à l'utilisateur lors de sa prochaine authentification
  - Verify Email, Update Password, Update Profile, Update Email
  - Configure OTP, WebAuthn Register
  - Terms and Conditions : acceptation des conditions d'utilisation
  - Verify Profile : complétion du profil quand le User Profile l'exige
- Configuration : *Authentication → Required actions*
  - Activation de chaque action, action par défaut pour les nouveaux utilisateurs
- Affectation manuelle par l'administrateur sur un utilisateur

Self-registration



- Les utilisateurs peuvent créer eux-mêmes leur compte
  - *Realm settings → Login → User registration*
- Formulaire construit à partir du User Profile (attributs, validations)
- Options associées :
  - Vérification de l'email
    Realm settings → Login → Verify email
  - Acceptation des conditions d'utilisation
    required action Terms and Conditions
  - reCAPTCHA dans le flow registration

Récupération de compte

Mot de passe oublié : *Realm settings → Login → Forgot password*

- Nécessite un serveur SMTP : *Realm settings → Email*
- Parcours défini par le flow « reset credentials »
- Lien envoyé par mail à durée limitée :
  Realm settings → Tokens → action tokens

Perte du second facteur :

- Codes de récupération (Recovery Authentication Codes) générés à l'enrôlement
- Ou suppression de l'OTP par un administrateur puis ré-enrôlement

Verrouillage après échecs répétés :
Realm settings → Security defenses → Brute force detection



Actions de l'administrateur

Onglet ***Credentials*** d'un utilisateur :

- Réinitialiser le mot de passe, éventuellement temporaire
- Supprimer un credential (OTP, WebAuthn)

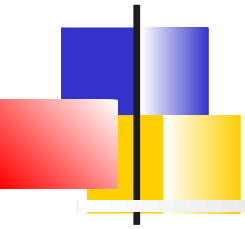
Envoi d'un mail d'actions : *Credentials → Credential reset*

- UPDATE_PASSWORD, CONFIGURE_TOTP, VERIFY_EMAIL, UPDATE_PROFILE

Required actions positionnées manuellement, exécutées au prochain login

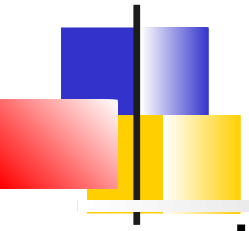
Déverrouiller un compte bloqué par la détection brute force

Désactiver un compte (Enabled = off) sans le supprimer



Exploitation courante

Gestion des utilisateurs
Mots de passe et MFA
Sessions et jetons
Événements et audit



Introduction

- Keycloak permet de configurer l’authentification des utilisateurs :
  - Politique sur les mots de passe
  - Utilisation de l’OTP (One Time Password)
  - Utilisation de WebAuthn (Web Authentication)
- La configuration des séquences d’authentification (flows) est traitée plus tard



Politique des mots de passe

Authentication → Policies → Password policy

- Algorithme de hachage : Argon2 par défaut depuis la version 24 (PBKDF2 en mode FIPS)
- Itérations de hachage : impact sur les performances
- Longueur minimale et maximale, chiffres, minuscules, majuscules, caractères spéciaux
- Not username, Not email, expression régulière
- Historique (Not recently used), expiration, liste noire

Recommandations actuelles (NIST) : privilégier la longueur à la complexité, éviter l'expiration forcée

Protéger aussi contre la force brute :
Realm settings → Security defenses

OTP



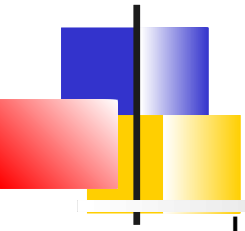
En plus de fournir un mot de passe, les utilisateurs doivent fournir une 2ème preuve de leur identité

- Un code fourni par les applications FreeOTP ou Google Authenticator disponible sur mobile
- Soit l'utilisateur, soit l'administrateur peut décider de l'utilisation de OTP

Différentes stratégies peuvent être configurées pour l'OTP :

- **Time-Based One-Time Password (TOTP)** : Le code est valable pendant une certaine période (30 secondes par défaut)
- **HMAC-Based One-Time Password (HOTP)** : Le code est valable tant qu'il n'a pas été utilisé

Authentication → Policies → OTP Policy



OTP en détail

Lors de l’enrôlement :

- Keycloak génère une clé secrète (la graine) et l’affiche en QR code
- L’application mobile la scanne : le même secret est stocké des deux côtés

Lors de l’authentification :

- Les deux côtés calculent un code à partir du secret et de l’heure (TOTP) ou d’un compteur (HOTP)
- L’utilisateur saisit le code, Keycloak le compare au sien

Risques liés au secret partagé :

- Fuite de la base Keycloak : les graines permettent de générer les codes de tous les utilisateurs
- Copie du QR code à l’enrôlement (capture d’écran, partage)
- Phishing en temps réel : le code saisi sur une fausse page est rejoué aussitôt sur la vraie
- Perte du téléphone : réenrôlement par l’administrateur



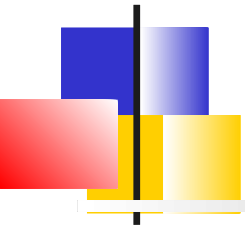
Mise en place de l'OTP

L'utilisateur active l'OTP dans sa console de compte

- *Account security → Signing in → Set up authenticator application*
- QR code représentant la clé partagée qui sert à générer les codes

Rendre l'OTP obligatoire :

- Pour un utilisateur : required action Configure OTP
- Pour tous : required action Configure OTP par défaut, ou étape OTP du flow browser à Required



WebAuthn

- Norme du W3C (FIDO2) conçue pour remplacer ou compléter les mots de passe :
  - Clés de sécurité matérielles (type YubiKey)
  - Biométrie ou code PIN d’un appareil (empreinte, reconnaissance faciale)
  - Passkeys : identifiants WebAuthn synchronisés entre appareils, supportées dans les versions récentes
- Cryptographie à clé publique : aucun secret partagé, contrairement à l’OTP
- Résistant au phishing : la clé est liée à l’origine du site

WebAuthn en détail

Lors de l’enregistrement :

- L’authenticator génère une paire de clés (publique/privée).
- La clé publique est envoyée au serveur (ici Keycloak).
- La clé privée est stockée dans l’authenticator (clé matérielle, OS, etc.).

Lors de l’authentification :

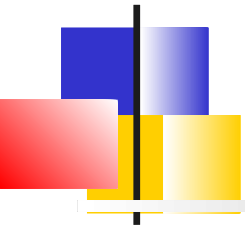
- Le serveur envoie un challenge, différent à chaque fois.
- Le navigateur le transmet à l’authenticator avec l’origine du site ; l’authenticator le signe avec la clé privée.
- Le serveur vérifie la signature avec la clé publique.

Avantages face aux risques de l’OTP :

- Fuite de la base : sans effet, une clé publique ne permet pas de signer
- Aucun secret ne transite, ni à l’enregistrement ni au login
- Phishing : une fausse page sur un autre domaine n’obtient aucune signature valable
- Limite : perte de l’authenticator ; en enregistrer deux, ou utiliser des passkeys synchronisées (Clés copié dans un compte cloud)

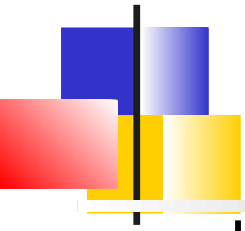
Mise en place de WebAuthn

- Ajouter WebAuthn Authenticator (2nd facteur) ou WebAuthn Passwordless dans un flow
- L'utilisateur enregistre son authenticator dans sa console de compte
  - Ou via la required action WebAuthn Register
- Policies dédiées :
  Authentication → Policies → WebAuthn Policy



Exploitation courante

Gestion des utilisateurs
Mots de passe et MFA
Sessions et jetons
Événements et audit



Introduction

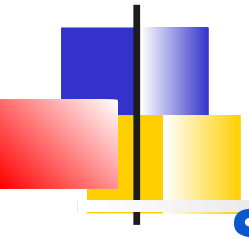
Les sessions permettent à Keycloak de savoir si un utilisateur est authentifié, pour combien de temps, et quand le ré-authentifier

Depuis la version 26, les sessions utilisateur sont persistées en base de données, avec un cache en mémoire

- Elles survivent au redémarrage ; leur volume impacte la base

L’exploitant peut :

- Configurer les timeouts
- Visualiser les sessions actives
- Forcer la fermeture de sessions



Types de sessions

- **Session SSO** (utilisateur) : suit l'utilisateur quel que soit le client
  - Ses timeouts contrôlent la fréquence des ré-authentifications
  - Son expiration entraîne celle de toutes les sessions client associées
- **Session client** : une par application accédée
  - Porte les jetons et les scopes accordés
  - Expire avec la session SSO, ou quand le refresh token expire ou est révoqué
- **Session offline** : créée par le scope *offline_access*
  - Survit à la fin de la session SSO, avec ses propres timeouts

Timeouts

Realm settings → Sessions

- **SSO Session Idle** : expiration sans activité ni rafraîchissement (30 min par défaut)
- **SSO Session Max** : durée maximale d'une session (10 h par défaut)
- Équivalents pour les sessions offline et l'option Remember me

Au niveau d'un client :

Client → Advanced → Advanced settings

- Client Session Idle, Client Session Max, Access Token Lifespan

Gestion des sessions actives

- Les administrateurs ont une visibilité des sessions actives au niveau
  - Du realm
  - D’un client
  - D’un utilisateur
- Il peut forcer un logout à tous les niveaux

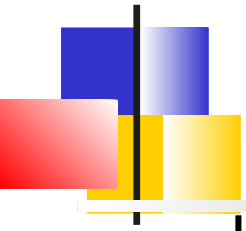
Cookies

Après authentification, Keycloak positionne plusieurs cookies :

- ***KEYCLOAK_IDENTITY*** : identité de la session SSO, HttpOnly
- **KEYCLOAK_SESSION** : identifiant de session, lisible en JavaScript (session management)
- **AUTH_SESSION_ID** : session d'authentification en cours, porte la route du nœud (affinité)

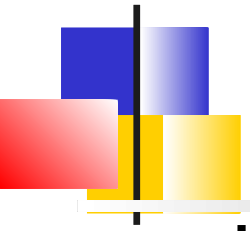
Expiration alignée sur le Max time de la session

Attribut Secure en HTTPS, SameSite=None pour les usages cross-site



Jetons

- Les jetons sont généralement liés à des sessions. Par conséquent, la validité des jetons (pas nécessairement leur durée de vie) dépend des sessions.
- Les jetons ont leur propre durée de vie et la durée pendant laquelle ils sont considérés comme valides dépend de la façon dont ils sont validés
- Avec JWT, les serveurs de ressources valident indépendamment de Keycloak un jeton
  => Pendant sa durée de vie, la session peut avoir expirée
  => Si l'on ne veut pas, il faut se rabattre sur une validation en utilisant l'endpoint d'introspection



Durée de vie des jetons

- Jetons d’accès et d’identification : même durée de vie, courte, configurée par ***Access Token Lifespan***
  - Limite l’impact d’un jeton volé, qui reste valide jusqu’à expiration
- Jetons de rafraîchissement : durée plus longue, bornée par les sessions
  - Leur validité suit les timeouts idle et max des sessions SSO et client

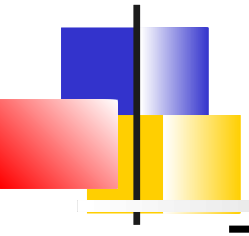
Configuration des jetons

La configuration s'effectue dans

Realm Settings → Tokens

Les valeurs par défaut sont 5 minutes pour les jetons d'accès et d'identification

On peut configurer globalement ou par clients



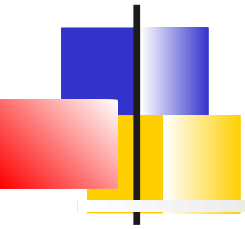
Jetons de rafraîchissement

- Toujours liés à une session client
- Valides tant que les sessions utilisateur et client associées n’ont pas expiré
- Offrent une surface d’attaque plus longue que les jetons d’accès
  - Un client confidentiel a généralement des sessions plus longues qu’un client public



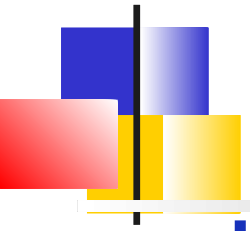
Rotation des jetons de rafraîchissement

- Keycloak émet un nouveau refresh token à chaque rafraîchissement
- Revoke Refresh Token (Realm settings → Tokens) : l'ancien jeton est invalidé
  - Détecte une fuite : l'attaquant ou le client légitime doit se ré-authentifier
- Refresh Token Max Reuse : nombre de réutilisations tolérées (0 par défaut : usage unique)



Exploitation courante

Gestion des utilisateurs
Mots de passe et MFA
Sessions et jetons
Événements et audit



Événements Keycloak

User events : actions des utilisateurs

- LOGIN, LOGIN_ERROR, LOGOUT, REGISTER, UPDATE_PASSWORD
- CODE_TO_TOKEN, REFRESH_TOKEN, CLIENT_LOGIN, …

Admin events : opérations d'administration

- CREATE, UPDATE, DELETE, ACTION sur une ressource (user, client, rôle…)
- Via la console ou l'API REST

Les événements sont transmis à des event listeners :

- jboss-logging (actif par défaut), email, listeners personnalisés via SPI

Configuration des événements

Realm settings → Events

User events settings :

- Save events : stockage en base de données
- Expiration : purge automatique des événements anciens
- Choix des types d'événements enregistrés

Admin events settings : Save events

- Include representation : contenu JSON de la ressource modifiée (attention aux données sensibles)

Event listeners : ajout des listeners actifs pour le realm



Consultation et audit

Menu ***Events*** : onglets ***User events*** et ***Admin events***

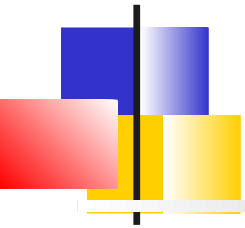
- Filtres : utilisateur, type, client, IP, période
- Admin events : realm, chemin de la ressource, opération, auteur

Accès via l'API d'administration :

```
GET /admin/realms/{realm}/events?type=LOGIN_ERROR
GET /admin/realms/{realm}/admin-events?operationTypes=DELETE
```

Usages : diagnostic des connexions, traçabilité des actions admin

La base Keycloak n'est pas un outil d'audit à long terme : centralisation dans un SIEM (voir supervision)



Adaptation et personnalisation

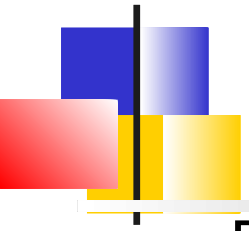
Organisation et administration

Fédération d’identités

Flows d’authentification

Thèmes

SPI



Introduction

Deux questions d’organisation, dans cet ordre :

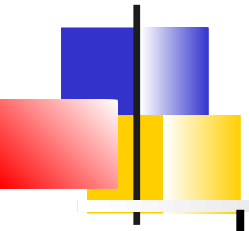
Comment découper ? Realms, clients et organisations

- Un découpage qui suit les populations et leurs politiques de sécurité

Qui administre quoi ? Super-administrateurs, administrateurs de realm, permissions fines

- Une fois le découpage fait, on délègue l’administration de chaque partie

Les droits d’administration s’attribuent par realm, puis par groupe : ils dépendent donc du découpage



Organisation

- Une organisation classique de l’administration :
  - 1 ou plusieurs super-administrateurs (realm master)
  - Des administrateurs dédiés à un realm
  - Éventuellement, des administrateurs aux permissions limitées (ex. gérer uniquement les utilisateurs d’un groupe particulier)
- Outils : Rôles du client ***realm-management***, puis permissions fines (FGAP v2)

Attribution des droits d'administration

Chaque realm dispose de son propre client ***realm-management***, qui contient des rôles d'administration

Un utilisateur devient admin d'un realm lorsqu'il reçoit un ou plusieurs rôles de ce client :

- ***realm-admin*** : Admin complet du realm
- ***manage-users*** : gérer les utilisateurs
- ***view-clients*** : lire les clients
- ***manage-realm***, ***manage-clients***, ***view-events***…

L'admin d'un realm se connecte à la console de son realm : ***/admin/{realm}/console***

- Ex. : ***http://localhost:8080/admin/formation/console***

Permissions fines



Fine-grained admin permissions v2, supportées depuis la 26.2

Activation :

Realm settings → Admin Permissions

Menu Permissions : une permission associe

- un type de ressource (Users, Groups, Clients, Roles), tout ou partie
- des actions : ***view***, ***manage***, ***manage-members***…
- des policies : à qui elle s'applique (utilisateur, groupe, rôle…)

Ex. : ***manage-members*** sur le groupe ***paris*** pour le groupe ***support-paris***

Organiser realms et clients

Realm master réservé à l'administration du serveur

Un realm = un domaine de sécurité : utilisateurs, SSO, politiques

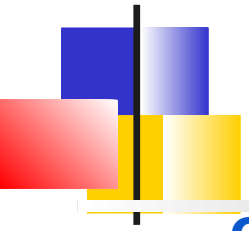
- Un realm par population (employés, clients, partenaires) plutôt que par application
- Pas de SSO entre realms

Environnements (dev, recette, prod) : instances séparées de préférence

Éviter la multiplication des realms (démarrage, mémoire, console)

Clients :

- Convention de nommage des *client_id*
- Client scopes partagés plutôt que mappers dupliqués



Organisations

Organizations répond au besoin d'une application B2B qui sert plusieurs entreprises clientes. Avant Keycloak 26, il y avait deux approches, toutes deux insatisfaisantes :

- un realm par client : lourd à gérer, sans SSO entre realms, et la mémoire et le démarrage se dégradent quand les realms se multiplient ;
- un seul realm : on bricole avec des groupes et des attributs.

Fonctionnalité supportée depuis Keycloak 26

Une organisation est un sous-ensemble d'un realm. Elle a trois composantes :

- Des membres (gérés ou issus d'un IdP)
- Des domaines de messagerie
- Des fournisseurs d'identité associés

Organisations

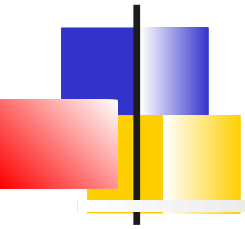
L’authentification devient « ***identity-first*** ».

Keycloak demande d’abord l’email seul, en déduit l’organisation à partir du domaine, puis redirige vers l’IdP de l’entreprise.
(Etape Organization du flow browser)

Côté application le Client scope organization ajoute le claim *organization* dans les jetons

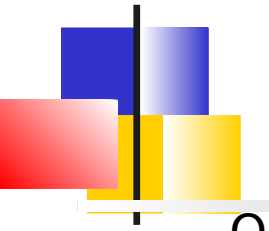
On reste dans un seul realm : mêmes clients, mêmes flows et SSO partagé. L’isolation est logique et non physique.

Activation :
Realm settings → Organizations



Adaptation et personnalisation

Organisation
Fédération d’identités
Flows d’authentification
Thèmes
SPI



Fédération d’identités

Objectif : s’appuyer sur des identités gérées ailleurs plutôt que de les recréer dans Keycloak

User federation : Keycloak consulte un référentiel d’utilisateurs externe

- LDAP / Active Directory, Kerberos, base de données spécifique (User Storage SPI)
- La saisie des identifiants se fait sur la page de login de Keycloak

Identity brokering : Keycloak délègue l’authentification à un autre fournisseur

- OpenID Connect, SAML 2.0, fournisseurs sociaux (Google, GitHub, Microsoft…)
- L’utilisateur s’authentifie chez le fournisseur, Keycloak reçoit des jetons ou une assertion

Organizations : rattachent domaines et IdP à des organisations clientes

Dans tous les cas, les applications ne connaissent que Keycloak

Intégration LDAP / Active Directory

User federation → Add LDAP providers

- Connexion : URL, bind DN, users DN, filtre
- Edit mode : READ_ONLY, WRITABLE, UNSYNCED
- Import users : copie locale ou consultation à la volée

La vérification du mot de passe reste déléguée à l'annuaire

Plusieurs annuaires possibles dans un realm, avec un ordre de priorité

Synchronisation périodique : complète ou utilisateurs modifiés

Mappers LDAP : ***user-attribute***, ***full-name***, ***group-ldap-mapper***, ***role-ldap-mapper***, ***msad-user-account-control*** (AD)

Kerberos (SPNEGO) optionnel pour le SSO poste de travail. Keycloak réutilise le ticket Kerberos obtenu lors de l'authentification Windows

LDAP : importer ou non

Import users activé (par défaut) : copie des utilisateurs dans la base Keycloak

- Atout :
  - Recherches rapides, moins de requêtes vers l'annuaire
  - Données locales possibles : rôles, attributs, mode UNSYNCED
- Limite :
  - Données dupliquées, à synchroniser (périodiquement ou à la connexion)
  - Utilisateurs supprimés de l'annuaire à purger dans Keycloak

Import users désactivé : lecture à la volée dans l'annuaire (avec cache)

- Atout :
  - Pas de duplication, données toujours à jour
- Limite :
  - Dépendance forte à l'annuaire : disponibilité, latence, charge
  - Pas de mode UNSYNCED, recherches limitées dans la console

Dans les deux cas, le mot de passe est vérifié par l'annuaire (bind LDAP)



LDAP : modes d’édition

READ_ONLY : l’annuaire reste la seule source de vérité

- Atout : Aucune écriture dans l’annuaire, compte de bind en lecture seule
- Limite : Profil et mot de passe non modifiables depuis Keycloak

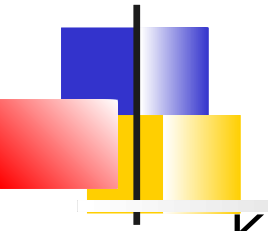
WRITABLE : les modifications faites dans Keycloak sont écrites dans l’annuaire

- Atout : Gestion centralisée : profil, mot de passe, création d’utilisateurs
- Limite : Compte de bind avec droits d’écriture, conflits avec les autres outils de gestion

UNSYNCED : modifications conservées dans Keycloak, jamais écrites dans l’annuaire

- Atout : Enrichissement local sans toucher à l’annuaire
- Limite : Divergence progressive entre Keycloak et l’annuaire

Cas courant : Active Directory d’entreprise en READ_ONLY avec import et synchronisation des utilisateurs modifiés



Identity brokering

- Keycloak délègue l'authentification à un fournisseur externe puis émet ses propres jetons
  - Applications inchangées : elles ne connaissent que Keycloak
  - Utilisateur importé dans Keycloak à sa 1ère connexion : il bénéficie ensuite des rôles, sessions et politiques du realm
- Fournisseurs : OpenID Connect, SAML 2.0, sociaux (Google, GitHub, Microsoft…)
- Identity providers → Add provider
  - Alias et redirect URI : ***/realms/{realm}/broker/{alias}/endpoint***
  - Trust email, Hide on login page, Sync mode (Import, Legacy, Force)
- Paramètre ***kc_idp_hint*** : redirection directe vers un IdP

First broker login

Flow exécuté lors de la 1ère connexion via un IdP externe

- Review profile : complétion des informations manquantes
- Création de l'utilisateur local
- Ou liaison avec un compte existant : confirmation, vérification par email ou ré-authentification

Post login flow : étapes supplémentaires à chaque connexion (ex. OTP)

Comptes liés visibles dans la console du compte utilisateur

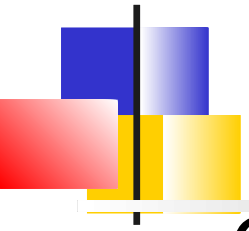
Flows personnalisables comme les autres flows d'authentification



Mappers des fournisseurs d'identité

- Transforment les informations reçues de l'IdP
  - Attribute importer :
    claim ou attribut SAML → attribut utilisateur
  - Username template importer :
    construction du username
  - Claim to role, Advanced claim to role, SAML attribute to role
  - Hardcoded role, Hardcoded attribute, Hardcoded group
- Sync mode par mapper :
  Appliqué à la création ou à chaque connexion (risque d'écrasement de modifications locales)
- Les rôles importés peuvent ensuite être exposés aux applications via les mappers des clients

Intégration avec une base de données spécifique

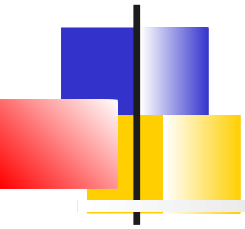


Objectif : conserver les utilisateurs dans son propre schéma relationnel

Deux alternatives :

- Implémenter un provider Java (User Storage SPI, interface ***UserStorageProvider***) et le déployer dans Keycloak : les utilisateurs restent dans la base externe[1]
- Utiliser l'API REST d'administration pour synchroniser les utilisateurs vers Keycloak

1. https://github.com/keycloak/keycloak-quickstarts/tree/latest/extension/user-storage-jpa



Adaptation et personnalisation

Organisation
Fédération d’identités
Flows d’authentification
Thèmes
SPI

Flux d’authentification

Un flow décrit toutes les étapes d’une authentification :
écrans de saisie, vérifications, actions

Flows prédéfinis :

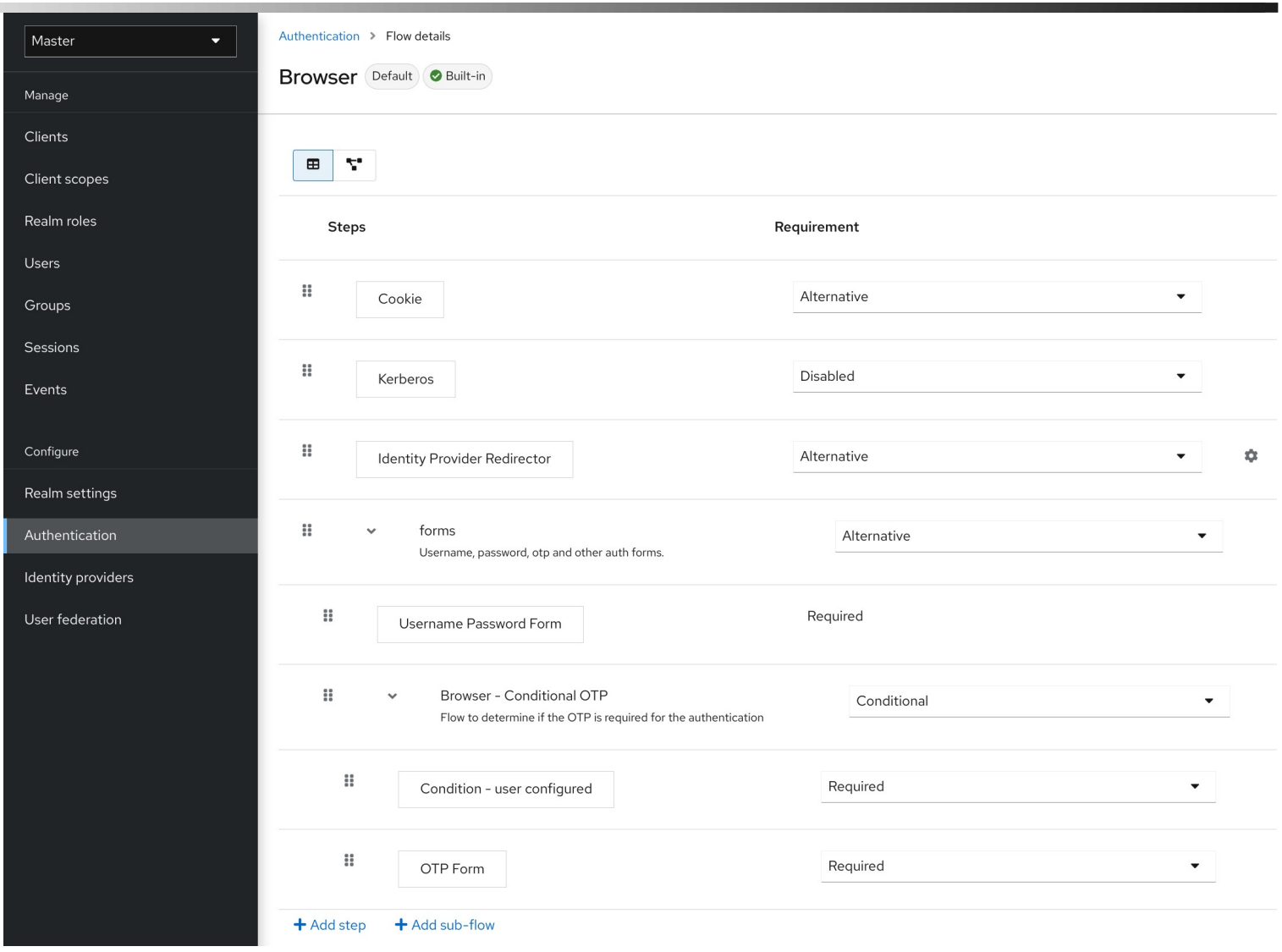
- on peut seulement changer l’exigence des étapes ; pour ajouter ou retirer des étapes, on duplique le flow

Association au realm : Un flow par usage
Action → Bind flow

Surcharge par client :
Client → Advanced → Authentication flow overrides

Sélection dynamique également possible via les *client policies*

Exemple Browser Flow



Attributs des étapes

Des radio buttons contrôlent l'exécution des étapes du flow :

- ***Required*** : Tous les éléments requis du flux doivent être exécutés séquentiellement avec succès.
  Le flux se termine si un élément requis échoue.
- ***Alternative*** : Un seul élément doit s'exécuter avec succès pour que le flux soit évalué comme réussi.
  => Tout élément alternative dans un flux contenant des éléments requis ne s'exécutera pas.
- ***Disabled*** : L'élément n'est pas pris en compte pour marquer un flux comme réussi.
- ***Conditional*** : Seulement sur des sous-flux
  - Contient des exécutions qui doivent correspondre à des instructions logiques.
  - Si toutes les exécutions sont évaluées comme vraies, le sous-flux conditionnel agit comme requis.
  - Si au moins un exécution est évaluée comme fausses, le sous-flux conditionnel agit comme désactivé.

Exemple Browser Flow

Cookie : si une session SSO existe déjà, l'étape réussit ; toutes les étapes de ce niveau étant ALTERNATIVE, l'utilisateur est connecté sans autre saisie

Kerberos : désactivé par défaut ; à passer en ALTERNATIVE pour le SSO Kerberos/SPNEGO, après configuration d'une fédération Kerberos ou LDAP avec Kerberos

Identity Provider Redirector : redirige directement vers un IdP (***kc_idp_hint*** ou IdP par défaut)

Organization (depuis la 26) : sous-flux conditionnel, actif si les Organisations sont activées ; identifie l'organisation par le domaine de l'email et redirige vers son IdP

Forms : Sous-flux ALTERNATIVE qui porte la saisie des identifiants et le second facteur

Sous-flux Form

Username Password Form : page de login et mot de passe, obligatoire (REQUIRED)

Browser - Conditional OTP ou ***Browser - Conditional 2FA*** selon la version : sous-flux CONDITIONAL

- ***Condition - user configured*** : vraie si l'utilisateur a configuré un des authentificateurs du sous-flux
- ***OTP Form*** : exécuté seulement si la condition est vraie

Conditional 2FA (realms créés depuis la 26.3) : OTP Form en ALTERNATIVE, WebAuthn et Recovery Authentication Code Form présents mais désactivés par défaut



Flows adaptatifs

Un sous-flux CONDITIONAL s'exécute si toutes ses conditions sont vraies

Conditions disponibles :

- ***Condition - user configured*** : l'utilisateur a configuré l'authentificateur
- ***Condition - user role*** / ***user attribute*** : rôle ou attribut de l'utilisateur (option Negate)
- ***Condition - client scope*** : le client demande un scope donné
- ***Condition - Level of Authentication*** : niveau d'authentification demandé (acr)
- ***Condition - sub-flow executed*** : un autre sous-flux a été exécuté (ou non)
- ***Condition - credential*** : type de credential utilisé (ex. passkey)

Exemple : OTP obligatoire uniquement pour les utilisateurs ayant le rôle admin

acr

acr (***Authentication Context Class Reference)*** est un claim standard qui indique le niveau ou le contexte de l'authentification utilisée pour connecter un utilisateur.

Avec Keycloak, on peut associer (au niveau realm ou au niveau client) des valeurs d’acr à des entiers : les ***LoA (Level Of Authentication)***

Il est alors possible de mettre au point un flow d’authentification avec une étape conditionnel dépendant du *LoA*

Scénario

1. Lors de la requête OIDC initiale, le client précise une valeur du paramètre acr_values. Ex :
   `acr_values=urn:myorg:mfa`
2. Keycloak mappe cette valeur à un LoA
3. Le flow d'authentification qui s'applique dépend du LoA
4. A l'issue de l'authentification, le claim ***acr*** est dans l'ID Token

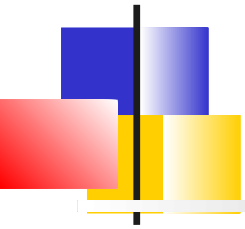
Cas d’usage : step-up

Exemple : portail RH, modifier un mot de passe administrateur exige un second facteur

- 1. Connexion par mot de passe : acr = urn:myorg:pwd
- 2. Action sensible : l’API lit acr et refuse (401)
- 3. L’application relance le login avec acr_values=urn:myorg:mfa
- 4. Keycloak ne demande que le second facteur : acr = urn:myorg:mfa
- 5. Max Age du niveau 2 : step-up valable quelques minutes

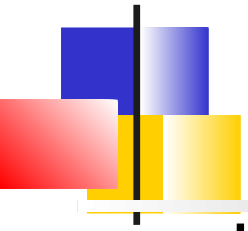
Pourquoi ? Second facteur seulement quand il est utile, et récent

Attention : c’est l’API qui contrôle acr, pas seulement le front



Adaptation et personnalisation

Organisation
Fédération d’identités
Flows d’authentification
Thèmes
SPI



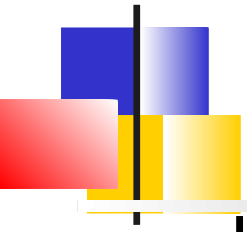
Introduction

Keycloak permet de personnaliser les pages utilisés par les utilisateurs via des thèmes :

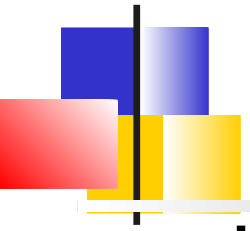
- Formulaires de login
- Pages de l’application account
- Email
- Pages d’accueil Keycloak
- Console d’administration

Il existe des thèmes prédéfinis et il est possible de développer son propre thème

Configuration



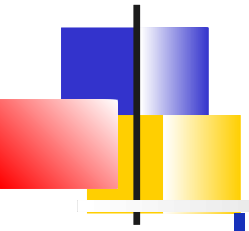
- La configuration s’effectue via la console d’admin : Realm settings → Themes (login, account, admin, email)
- Le thème de la page d’accueil se précise au démarrage du serveur :
  - ***bin/kc.[sh|bat] start --spi-theme--welcome-theme=custom-theme***
- Depuis la 26.3, les options SPI s’écrivent ***--spi-<spi>--<option>*** : l’ancien format à simple tiret reste accepté avec un warning « legacy format »



Eléments d’un thème

Un thème est constitué de :

- Gabarits FreeMarker (thèmes login et email)
- Images
- Messages localisés (bundles)
- Feuilles de style CSS
- Scripts
- Propriétés (***theme.properties***)



Types de thèmes

login et email : gabarits FreeMarker, entièrement surchargeables

welcome : page d'accueil du serveur

account et **admin** : applications React (keycloak.v3, keycloak.v2)

- Personnalisation à la marge : CSS, messages, logo, propriétés
- Pas de gabarits FreeMarker à surcharger

Refonte complète (y compris login en React) : Keycloakify

- Thèmes développés en React, packagés en jar à déposer dans providers/

Héritage

La création s’effectue généralement en étendant :

- Le thème ***Keycloak*** qui contient des images et les feuilles de style si l’on veut modifier les gabarits (ajout de lien par exemple)
- Ou le thème ***base*** qui contient les gabarits et les messages si l’on veut travailler le look and feel

Après avoir étendu un thème on peut surcharger ses éléments

Lors de la création de thème, il est recommandé de désactiver la fonctionnalité de cache pour éviter de redémarrer le serveur

Procédure

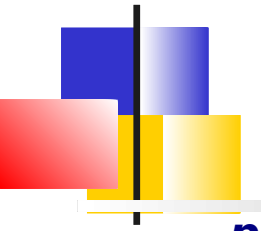
Désactivation du cache :

- ***bin/kc.[sh|bat] start --spi-theme--static-max-age=-1 --spi-theme--cache-themes=false --spi-theme--cache-templates=false***
- En ***start-dev***, le cache des thèmes est déjà désactivé

Création d'un sous-répertoire dans le répertoire ***themes***

Puis création d'un répertoire par type de thème (ex. ***login/*** pour la page de login)

Pour chaque type, création d'un fichier ***theme.properties***



Propriétés de thème

parent : Le thème parent à étendre

import : Importer des ressources d'un autre thème

common : Surcharge le chemin vers les ressources communes. (par défaut est *common/keycloak*) Utilisé dans les gabarits *Freemarker* via ${url.resourcesCommonPath}

styles/scripts : Listes de styles/script à inclure. Les fichiers sont placés dans */resources/css* | */resources/js* du répertoire du thème

locales : Liste des langues supportés

Des propriétés sont également utilisées pour changer les classes css de certains éléments

Il est également possible de définir des propriétés custom qui seront utilisées dans les gabarits et d'utiliser des propriétés JVM ou de l'environnement

Création de gabarit

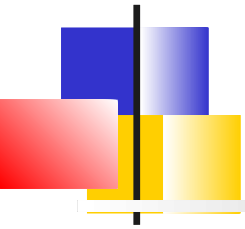
- Il est possible de surcharger un gabarit du parent en plaçant un gabarit FreeMarker à la racine du répertoire du type
- La liste des gabarits surchargeables est visible en ouvrant le jar de la distribution :
  - ***lib/lib/main/org.keycloak.keycloak-themes-<version>.jar***

Déploiement du thème

Les thèmes peuvent être déployés :

- En copiant le répertoire du thème dans le répertoire ***themes*** de la distribution
- En créant un jar contenant les thèmes (***theme/<nom>/<type>/…***) et un fichier ***META-INF/keycloak-themes.json*** qui déclare chaque thème et ses types
- Le jar se place dans le répertoire ***providers*** de la distribution





Adaptation et personnalisation

Organisation
Fédération d’identités
Flows d’authentification
Thèmes
SPI

Introduction

Les ***SPI (Service Provider Interfaces)*** sont des extensions personnalisées qui permettent aux développeurs d'étendre, de personnaliser et d'intégrer Keycloak avec d'autres systèmes ou services.

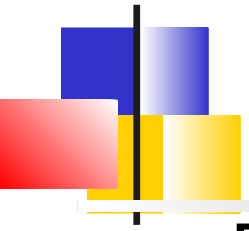
Les SPI offrent une flexibilité et une extensibilité considérables pour adapter Keycloak à des cas d'utilisation spécifiques

SPIs disponibles

Les principales SPIs sont :

- ***Authenticator SPI*** : Étendre les flows d'authentification
- ***User Storage SPI*** : Stocker les utilisateurs dans un système de persistance personnalisé
- ***Federated Identity Provider SPI*** : Pour intégrer des fournisseurs d'identité externes
- ***Protocol Mapper SPI*** : Pour ajouter des informations personnalisés dans les jetons
- ***Event Listener SPI*** : Pour intégrer des listeners des événements Keycloak
- ***Action Token Handler SPI*** : Des liens à usage unique permettant des actions côté Keycloak (Réintialiser les mots de passe, Confirmer une adresse e-mail, Executer des required action(s))
- ***Vault SPI :*** Connexion à une implémentation Vault
- ...

1. Des exemples sont disponibles à https://github.com/keycloak/keycloak-quickstarts/tree/latest/extension



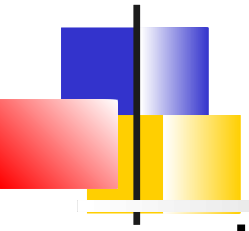
Implémentation

Pour implémenter une SPI, il faut :

- Implémenter 2 interfaces : ***ProviderFactory*** et ***Provider***
- Créer un fichier de configuration du service qui référence la factory

Attention : beaucoup de SPI reposent sur des API internes (***keycloak-server-spi-private***) sans garantie de compatibilité

- Recompiler et retester les providers à chaque montée de version (voir J5, migrations)



Interface factory

- Le serveur *Keycloak* instancie la factory comme singleton et appelle ses méthodes de callback *init()*, *postInit()*, *close()*.
- La responsabilité de la factory est de créer l'implémentation du provider lors de sa méthode *create()* et de fournir un identifiant :
  - Unique
  - Ou surchargeant un service existant

Exemple : Choix du thème en fonction du contexte (client ou autre)
ThemeSelectorProviderFactory

```
// Singleton créé au démarrage ; à déclarer dans
    META-INF/services/org.keycloak.theme.ThemeSelectorProviderFactory

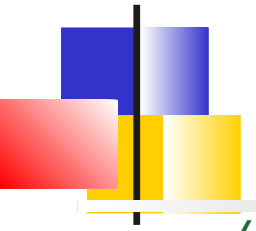
public class MyThemeSelectorProviderFactory implements ThemeSelectorProviderFactory {


  // Crée le provider qui choisit le thème selon le contexte (realm, client…)
    @Override
    public ThemeSelectorProvider create(KeycloakSession session) {
        return new MyThemeSelectorProvider(session);
    }
    // Appelée une fois au démarrage : lit la configuration (--spi-theme-selector)
    @Override
    public void init(Config.Scope config) { }

    @Override
    public void postInit(KeycloakSessionFactory factory) { }

    @Override
    public void close() { }

    // Identifiant du provider ; à activer avec --spi-theme-selector--provider=myThemeSelector
    @Override
    public String getId() {
        return "myThemeSelector";
    }
```

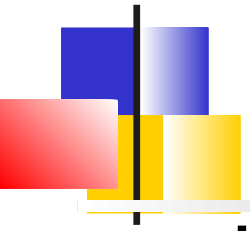


Exemple *ThemeSelectorProvider*

```
// Provider créé par la factory à chaque requête : décide du thème à
   utiliser
public class MyThemeSelectorProvider implements ThemeSelectorProvider {
    // Session de la requête courante : donne accès au realm, au client,
    // à l'utilisateur…
    private KeycloakSession session;


    public MyThemeSelectorProvider(KeycloakSession session) {
        this.session = session;
    }


   // Appelée pour chaque type de thème (LOGIN, ACCOUNT, ADMIN, EMAIL) :
    // renvoie le nom du thème à appliquer
    @Override
    public String getThemeName(Theme.Type type) {
        return _resolveTheme(type);
    }
}
```



Fichier de configuration

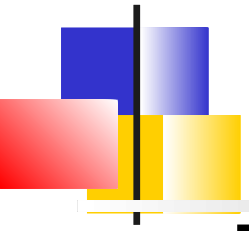
Le fichier de configuration se place ***META-INF/services/*** et se nomme en fonction de la SPI

Par ex :

```
META-INF/services/org.keycloak.theme.ThemeSelectorProviderFactory
```

Il référence l'implémentation :

```
com.acme.provider.MyThemeSelectorProviderFactory
```



Déploiement

Placer le jar dans le répertoire **providers/**

Reconstruire le serveur :

- Automatique au démarrage sans --optimized
- Explicite avec ./bin/kc.sh build pour une image optimisée

En conteneur : copier le jar dans /opt/keycloak/providers/ avant l'étape de build du Dockerfile

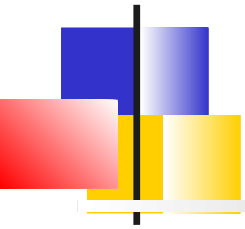
Affichage dans la console

Afin que la SPI développée s'affiche dans la page server info de la console d'administration, il faut implémenter l'interface ***ServerInfoAwareProviderFactory*** dans la classe factory

```
public class MyThemeSelectorProviderFactory implements
ThemeSelectorProviderFactory, ServerInfoAwareProviderFactory {
 ...


 @Override
 public Map<String, String> getOperationalInfo() {
     Map<String, String> ret = new LinkedHashMap<>();
     ret.put("theme-name", "my-theme");
     return ret;
 }
}
```



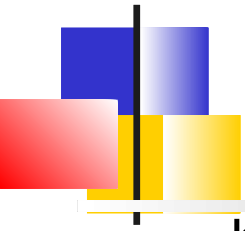


Automatisation et CI/CD

Scripts d’administration

CI/CD et GitOps

API REST d'administration



Keycloak propose une API REST d'administration : ***/admin/realms/…***

C'est elle qu'utilise la console : tout ce qui se fait dans la console peut se faire par l'API

L'API permet de :

- Manipuler les realms
- Manipuler les users / groupes / realm roles
- Les clients / client scopes / rôles associés aux clients
- Les mappers
- Mais également certains endpoints pour détecter les attaques brute force

Documentation :
https://www.keycloak.org/docs-api/latest/rest-api/

Obtention du jeton

Pour invoquer l'API, il faut un jeton d'accès portant les rôles d'administration adéquats

- Client ***admin-cli*** + identifiants d'un administrateur
  - Usage interactif, découverte de l'API
- Client confidentiel dédié, Service accounts enabled : usage dans les scripts
  - On attribue au service account les rôles minimaux du client ***realm-management*** (***manage-users***, ***view-realm***, ***manage-clients***…) selon les besoins[1]
  - Ni audience ni rôle admin supplémentaire nécessaires
  - Secret stocké dans un coffre (Vault, secrets CI/CD), rotation périodique

1. Onglet service account roles

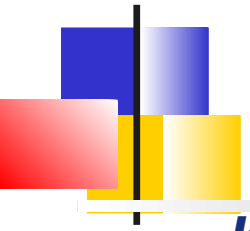
Appel de l’API avec curl

Obtention du jeton par client credentials, puis appel de l’API :

```
TOKEN=$(curl -s -d grant_type=client_credentials \
  -d client_id=automation -d client_secret="$KC_SECRET" \
  "$KC_URL/realms/master/protocol/openid-connect/token" | jq -r .access_token)
curl -s -H "Authorization: Bearer $TOKEN" \
  "$KC_URL/admin/realms/myrealm/users?briefRepresentation=true&max=100" \
  | jq -r ".[].username"
```

Avec ***admin-cli*** :

grant_type=password,
client_id=admin-cli, *username*, *password*



kcadm.sh

kcadm.sh est l'outil en ligne de commande officiel pour administrer Keycloak via son API REST

Livré avec chaque distribution Keycloak (répertoire ***bin***)

Avant utilisation, se connecter à l'instance Keycloak :

- En administrateur, pour un usage interactif
- Avec le client dédié, dans un script

```
kcadm.sh config credentials --server http://localhost:8080 \
  --realm master --user admin --password admin
```

```
kcadm.sh config credentials --server https://auth.example.com \
  --realm master --client automation --secret "$KC_SECRET"
```

Le jeton est conservé dans ***~/.keycloak/kcadm.config*** pour les commandes suivantes

Quelques commandes

```
# Lister les utilisateurs du realm 'myrealm'
kcadm.sh get users -r myrealm

# Créer un utilisateur
kcadm.sh create users -r myrealm \
  -s username=testuser -s enabled=true

# Ajouter un rôle à un utilisateur
kcadm.sh add-roles -r myrealm \
  --uusername testuser \
  --rolename user
```

Possibilité de chainer des appels et de manipuler des IDs via *jq* ou *grep*.

Fichiers json

```
# Importation d’un realm
kcadm.sh create realms -f realm-export.json

# Création d’un user
kcadm.sh create users -r monrealm -f user.json
kcadm.sh set-password -r monrealm --userid
  $USER_ID --new-password monpass123

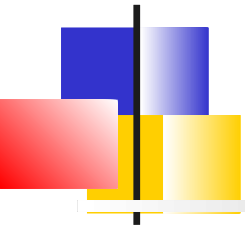
# Création d’un client
kcadm.sh create clients -r monrealm -f client.json
```

Exemple de script



Exemple : désactiver les comptes d'une liste de départs

```
#!/bin/bash
set -euo pipefail
kcadm.sh config credentials --server "$KC_URL" --realm master \
  --client automation --secret "$KC_SECRET"
while read -r u; do
  id=$(kcadm.sh get users -r myrealm -q username="$u" -q exact=true \
       --fields id --format csv --noquotes)
  [ -n "$id" ] && kcadm.sh update users/$id -r myrealm -s enabled=false
done < departs.txt
```



Automatisation et CI/CD

Scripts d’administration

CI/CD et GitOps



Configuration as code

Realms, clients, rôles, scopes, flows décrits dans Git

- Revue par merge request, historique, retour arrière
- Application idempotente par un pipeline

Séparer :

- La configuration (Git) des données (utilisateurs en base)
- Les valeurs par environnement (variables) du modèle commun
- Les secrets (coffre) des fichiers versionnés

Outils : keycloak-config-cli, Terraform, Ansible, Keycloak Operator

keycloak-config-cli

- Outil open source d'import idempotent de fichiers de realm JSON ou YAML
  - Crée, met à jour et peut supprimer les éléments
  - Substitution de variables : $(env:CLIENT_SECRET)

```
docker run --rm \
  -e KEYCLOAK_URL=https://auth.example.com \
  -e KEYCLOAK_USER=admin -e KEYCLOAK_PASSWORD="$PWD_ADMIN" \
  -e IMPORT_FILES_LOCATIONS="/config/*" \
  -e IMPORT_VARSUBSTITUTION_ENABLED=true \
  -v ./config:/config adorsys/keycloak-config-cli:<version>
```

- Image à choisir selon la version de Keycloak ciblée

Terraform

Provider keycloak/keycloak : realms, clients, rôles, IdP, flows…

```
provider "keycloak" {
  url = "https://auth.example.com"
  client_id = "terraform"
  client_secret = var.kc_secret
}
resource "keycloak_realm" "app" {
  realm = "app"
}
resource "keycloak_openid_client" "web" {
  realm_id = keycloak_realm.app.id
  client_id = "web-app"
  access_type = "CONFIDENTIAL"
  standard_flow_enabled = true
  valid_redirect_uris = ["https://app.example.com/callback"]
}
```

plan en revue, apply dans le pipeline, état stocké à distance

Ansible

Installation et configuration des serveurs : rôles et collections dédiés

Configuration des realms : modules
community.general.keycloak_*

```
- name: Client web-app
  community.general.keycloak_client:
    auth_keycloak_url: https://auth.example.com
    auth_realm: master
    auth_username: "{{ kc_admin }}"
    auth_password: "{{ kc_admin_password }}"
    realm: app
    client_id: web-app
    redirect_uris: ["https://app.example.com/callback"]
    state: present
```

Autres modules :
keycloak_realm, keycloak_role, keycloak_group, keycloak_user…

GitOps sur Kubernetes

Ressources ***Keycloak*** et ***KeycloakRealmImport*** versionnées, synchronisées par Argo CD ou Flux

Limite : *KeycloakRealmImport* crée un realm mais ne met pas à jour un realm existant

Pipeline type :

- Merge request → validation (plan, dry-run)
- Application en recette → test de login automatisé
- Promotion en production

Exemple Ressource Keycloak

Décrit le déploiement : l'Operator crée le StatefulSet, les services et applique les mises à jour

```
apiVersion: k8s.keycloak.org/v2alpha1
kind: Keycloak
metadata: { name: keycloak, namespace: iam }
spec:
  instances: 3
  image: registry.example.com/iam/keycloak:26.8.0
  startOptimized: true
  db:
    vendor: postgres
    host: postgres-rw
    usernameSecret: { name: kc-db, key: username }
    passwordSecret: { name: kc-db, key: password }
  http: { tlsSecret: keycloak-tls }
  hostname: { hostname: https://auth.example.com }
  proxy: { headers: xforwarded }
  update: { strategy: Auto }
```

Image optimisée construite par la CI, ***startOptimized*** : pas de build au démarrage

Stratégie ***Auto*** : rolling update quand update-compatibility le permet

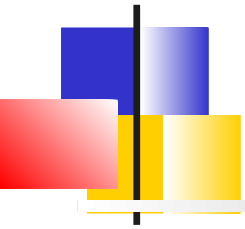
Exemple *KeycloakRealmImport*

Crée un realm à partir de sa représentation, importée par un Job de l'Operator

```
apiVersion: k8s.keycloak.org/v2alpha1
kind: KeycloakRealmImport
metadata: { name: realm-app, namespace: iam }
spec:
  keycloakCRName: keycloak
  realm:
    realm: app
    enabled: true
    clients:
      - clientId: web-app
        secret: ${WEB_APP_SECRET}
        redirectUris: [ "https://app.example.com/callback" ]
  placeholders:
    WEB_APP_SECRET:
      secret: { name: web-app, key: client-secret }
```

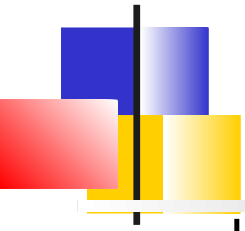
Les ***placeholders*** injectent des secrets Kubernetes : aucun secret dans Git

Un realm déjà existant n'est pas mis à jour



Architecture et fonctionnement

Composants et dépendances
Cycle de vie et modes d’exécution
Configuration du serveur
Image optimisée et conteneurs



Composants

Le serveur Keycloak : application Java construite sur Quarkus

- Il expose les pages de login, les consoles admin et account, et l'API REST
- Sans état propre hors ses caches : tous les nœuds sont interchangeables

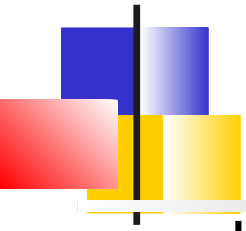
La base de données : la source de vérité

- Realms, clients, utilisateurs, credentials, clés et, depuis la 26, sessions
- Sa perte est fatale : sauvegarde et haute disponibilité obligatoires

Les caches Infinispan, embarqués dans chaque nœud

- Ils évitent de relire la base à chaque requête et partagent les sessions entre nœuds

Autour : load balancer, annuaires, IdP externes, serveur SMTP

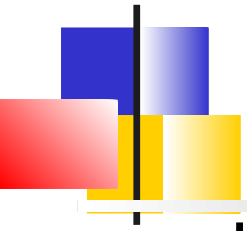


Briques techniques

Keycloak assemble des projets open source : les connaître aide à lire les logs

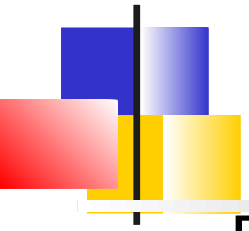
- Quarkus : démarrage rapide grâce à la phase de build
- Vert.x : couche HTTP
- Hibernate, Agroal, Liquibase : base, pool, schéma
- Infinispan et JGroups : caches et échanges entre nœuds
- Micrometer, SmallRye Health : métriques et sondes

Jusqu'à la 16 : serveur WildFly, attention aux documentations anciennes



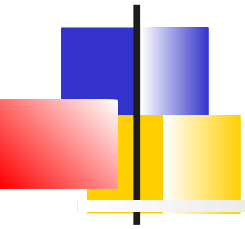
Caches locaux

- Keycloak distingue deux familles de caches, selon la nature des données
- Caches locaux : données lues souvent, modifiées rarement
  - ***realms***, ***users***, ***authorization***, ***keys***
  - Chaque nœud garde sa propre copie
- Après une modification, un message d'invalidation part vers les autres nœuds (cache work)
  - Ils relisent la donnée à jour en base à la requête suivante



Caches distribués

- Données créées en permanence et partagées entre les nœuds
  - ***sessions***, ***clientSessions***, ***authenticationSessions***, ***offlineSessions***
  - ***loginFailures***, ***actionTokens***
- Chaque entrée est stockée sur 2 nœuds (owners)
  - La perte d'un nœud ne fait pas perdre de données
- Depuis Keycloak 26, les sessions sont aussi écrites en base
  - Le cache n'en garde qu'une partie : ***--cache-embedded-sessions-max-count***



Architecture et fonctionnement

Composants et dépendances

Cycle de vie et modes d’exécution

Configuration du serveur

Image optimisée et conteneurs

Build et exécution

Comme toute application Quarkus, Keycloak sépare la construction du serveur de son exécution

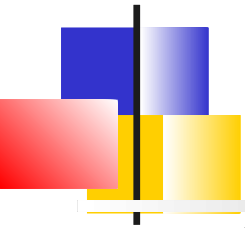
kc.sh build : phase d'augmentation

- Les options de build (type de bd, features, health, metrics…) sont figées pour accélérer le démarrage

kc.sh start : exécution avec les options de runtime (URL de la base, hostname, secrets…)

- Sans ***--optimized***, Keycloak vérifie les options de build et se reconstruit si besoin

Autres commandes : ***export***, ***import***, ***show-config***, ***bootstrap-admin***, ***update-compatibility***



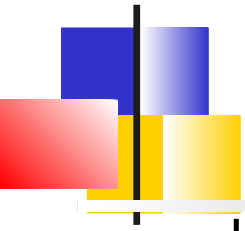
Démarrage et arrêt

Au démarrage d’un nœud :

- Connexion à la base
- Migration du schéma si nécessaire : un seul nœud à la fois, grâce à un verrou
- Entrée dans le cluster
- Le nœud se déclare prêt (/health/ready) : le load balancer peut lui envoyer du trafic

À l’arrêt :

- Le nœud se déclare non prêt et quitte le cluster
- Il transmet ses données de cache aux autres nœuds



Modes d’exécution

Une même distribution couvre tous les usages : le mode choisi détermine les valeurs par défaut

start-dev : poste de développement

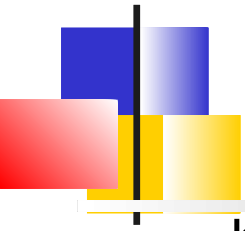
- Cache local, HTTP, base H2 embarquée : jamais en production

start sur un seul nœud (standalone) : configuration de production, sans haute disponibilité

start sur plusieurs nœuds : cluster, haute disponibilité et montée en charge

Conteneur : image officielle (Docker, Podman), Kubernetes avec l’Operator

Multi-site : deux clusters synchronisés, pour résister à la perte d’un datacenter



Démarrage

Keycloak peut être démarré en 2 modes :

Démarrage en mode dev

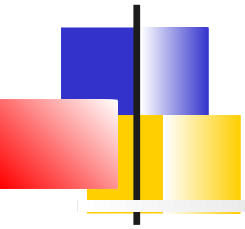
```
bin/kc.[sh|bat] start-dev
```

HTTP est activé
La résolution stricte du nom d'hôte est désactivée
Le cache est défini sur *local*
La mise en cache des thèmes et des modèles est désactivée (utile pour le développement de thème)

Démarrage en mode production

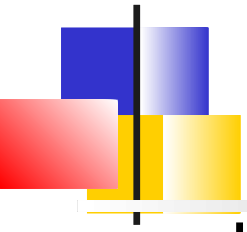
```
bin/kc.[sh|bat] start
```

HTTP est désactivé
Configuration du Hostname requise
HTTPS/TLS requis



Architecture et fonctionnement

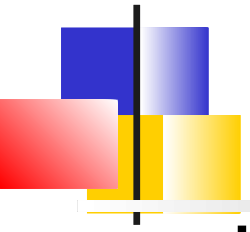
Composants et dépendances
Cycle de vie et modes d’exécution
Configuration du serveur
Image optimisée et conteneurs



Sources de configuration

Keycloak charge sa configuration à partir de quatre sources, par ordre de précédence :

- Paramètres de ligne de commande
  `--<key-with-dashes>=<value>`
  ex : `--db-url=<value>`
- Variables d'environnement
  `KC_<key_with_underscores>=<value>`
  ex : `KC_DB_URL=<value>`
- Fichier *.conf* créé par l'utilisateur
- Fichier *keycloak.conf* situé dans le répertoire conf.
  `<key-with-dashes>=<value>`
  ex : `db-url=<value>`



Fichiers de configuration

Les fichiers de configuration peuvent faire référence à des variables d’environnement et à des valeurs par défaut :

```
db-url-host=${MY_DB_HOST:mydb}
```

Un fichier de configuration dédié peut être fourni au démarrage :

```
bin/kc.[sh|bat] --config-file=/path/to/myconfig.conf start
```

TP 10.1

Utilisateurs admin

Au 1er démarrage, création d'un administrateur temporaire :

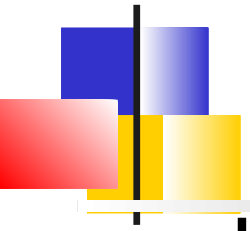
```
KC_BOOTSTRAP_ADMIN_USERNAME=admin
KC_BOOTSTRAP_ADMIN_PASSWORD=change_me
```

Les anciennes variables KEYCLOAK_ADMIN / KEYCLOAK_ADMIN_PASSWORD sont dépréciées

La console signale le caractère temporaire du compte

Récupération d'accès :
./bin/kc.sh bootstrap-admin user

Ensuite, créer des comptes nominatifs via la console ou kcadm et supprimer le compte temporaire



Configuration de production

La configuration de production nécessite :

- De configurer **TLS** pour les communications http
- Configurer le **hostname**
- Mettre en place un **reverse proxy**
- Mettre en place le **clustering**
- Configurer une **base de données de production**

Ports réseau

Keycloak sépare trois types de trafic sur des ports distincts

Trafic applicatif : navigateurs et applications

- ***8443*** HTTPS ; ***8080*** HTTP, refusé en production sauf TLS terminé au proxy

Trafic d'exploitation : interface de management, port ***9000***

- ***/health*** et ***/metrics*** séparés des pages de login, pour ne pas être exposés

Trafic interne au cluster : ***7800*** (JGroups) et ***57800*** (détection de défaillance)

Le préfixe ***/auth*** des anciennes versions a disparu en 17 (***--http-relative-path*** pour le recréer)

Flux réseau à ouvrir

Load balancer → nœuds :
8080 ou 8443, et 9000 pour les health checks

Nœud ↔ nœud : 7800 et 57800

- Uniquement entre les nœuds, jamais exposés

Nœuds → base de données

Nœuds → systèmes externes : LDAP, SMTP, IdP

- Et vers les applications pour le back-channel logout

Supervision → port 9000 : Prometheus, sondes

TLS

Pour charger les certificat, Keycloak s'appuie sur des fichiers PEM ou des keystore

PEM

```
bin/kc.[sh|bat] start --https-certificate-file=/path/to/certfile.pem
    --https-certificate-key-file=/path/to/keyfile.pem
```

Keystore

L'emplacement par défaut est conf/keystore

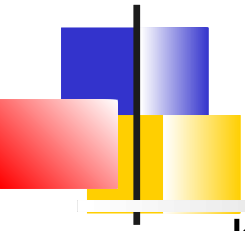
```
bin/kc.[sh|bat] start --https-key-store-password=<value>
```

Keycloak utilise le port 8443, cela peut être changé avec :

```
bin/kc.[sh|bat] start --https-port=<port>
```

L'authentification du client est également supportée :

```
bin/kc.[sh|bat] start --https-trust-store-file=/path/to/file
    --https-trust-store-password=<value>
    --https-client-auth=<none|request|required>
```



Certificats

Keycloak manipule plusieurs catégories de certificats, à ne pas confondre

Certificat HTTPS du serveur : présenté aux navigateurs et aux applications

- PEM ou PKCS12, rechargé automatiquement après renouvellement

Autorités de confiance des connexions sortantes (LDAPS, IdP, SMTP)

- Indispensables pour un annuaire signé par une autorité interne : ***--truststore-paths***

Certificats entre nœuds : mTLS JGroups, générés et renouvelés automatiquement

Clés de signature des jetons : propres à chaque realm (Realm settings → Keys), sans lien avec HTTPS

Types d’endpoints et hostname



Keycloak expose différents type d’endpoints :

- **Frontend** : Accessibles via un domaine public et généralement liés aux flux qui passent par le canal front.
  Par exemple, le *authorization_endpoint*
  Clé de configuration : ***hostname***
- **Backend** : Accessible via un domaine public ou un réseau privé. Ils sont utilisés pour une communication directe entre le serveur et les clients.
  Par exemple : *Token introspection, User info, Token endpoint, JWKS*
  Clé de configuration : ***hostname-backchannel-dynamic*** :
  - ***false (défaut) :*** Ces endpoints utilisent l’adresse publique (hostname)
  - ***true :*** Ces endpoints utilisent l’adresse par laquelle ils sont appelés, par exemple une adresse privée (hostname doit alors être une URL complète)
- **Console d'administration** : Généralement la console d’administration n’est pas accessible publiquement.
  Clé de configuration : ***hostname-admin***
- Diagnostic : --hostname-debug=true puis /realms/master/hostname-debug



Reverse proxy

Problème : derrière un proxy, Keycloak voit l'IP du proxy et une URL interne ; il a besoin de l'URL publique et de l'IP réelle du client

– Le proxy les transmet dans des en-têtes (X-Forwarded-* ou Forwarded)

--proxy-headers=xforwarded|forwarded : Keycloak fait confiance à ces en-têtes (ignorés par défaut)

– --proxy-trusted-addresses : seulement s'ils viennent du proxy, pour éviter qu'un client les forge

Selon l'emplacement du TLS :

– Terminé au proxy : **proxy-headers + --http-enabled=true**

– Ré-établi vers Keycloak : **proxy-headers + certificat sur Keycloak**

– Passthrough : **pas de proxy-headers** (le proxy ne voit pas la requête)

Note : L'ancienne option proxy (edge, reencrypt, passthrough) est supprimée

Configuration BD

Options basiques :

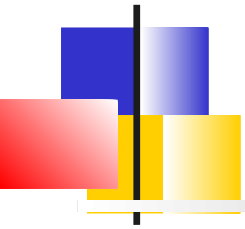
```
bin/kc.[sh|bat] start
--db postgres --db-url-host mypostgres
--db-username myuser --db-password change_me
```

Par défaut, le schéma par défaut de l'utilisateur de la base (public pour PostgreSQL), modifiable via *db-schema*.

Bases supportées : *mariadb, mssql, mysql, oracle, postgres*

Pilotes JDBC fournis, sauf pour Oracle : ajouter le pilote dans providers/

Transactions XA : --transaction-xa-enabled (désactivé par défaut, utile si une extension met à jour un ressource transactionnel qui supporte XA)



Architecture et fonctionnement

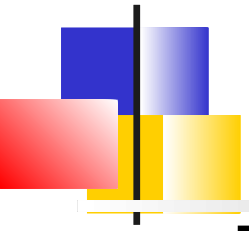
Composants et dépendances
Cycle de vie et modes d’exécution
Configuration du serveur
Image optimisée et conteneurs

Optimisation du démarrage

Il est recommandé d'optimiser Keycloak pour de meilleurs temps de démarrage et une meilleure consommation de mémoire avant de déployer dans des environnements de production.

Lors de l'utilisation des commandes *start* ou *start-dev*, Keycloak[1] exécute une commande de **build** qui peut prendre du temps.
=> Pour optimiser, exécuter cette commande de build avant le démarrage en fixant la plupart des valeurs de configuration, mettre au point un fichier *keycloak.conf* contenant les clé de configuration définies au runtime puis démarrer avec l'option ***optimized***.

1. Fonctionnement du framework Quarkus : la configuration est résolue lors d'une phase de build nommé augmentation



Exemple optimisation

Phase de build

```
bin/kc.[sh|bat] build --db=postgres
```

Keycloak.conf :

```
db-url-host=keycloak-postgres
db-username=keycloak
db-password=change_me
hostname=https://mykeycloak.acme.com
https-certificate-file=/opt/keycloak/conf/server.crt.pem
https-certificate-key-file=/opt/keycloak/conf/server.key.pem
```

Démarrage du serveur

```
bin/kc.[sh|bat] start --optimized
```

Cas d’un container

L'image du conteneur Keycloak par défaut est livrée prête à être configurée et optimisée.

On peut donc inclure la phase de build de Keycloak durant la construction du conteneur

Exemple Dockerfile

```
FROM quay.io/keycloak/keycloak:26.8.0 as builder

# Enable health and metrics support
ENV KC_HEALTH_ENABLED=true
ENV KC_METRICS_ENABLED=true

# Configure a database vendor
ENV KC_DB=postgres

WORKDIR /opt/keycloak
# for demonstration purposes only, please make sure to use proper certificates in production instead
RUN keytool -genkeypair -storepass password -storetype PKCS12 -keyalg RSA -keysize 2048 -dname "CN=server" -alias server -
     ext "SAN:c=DNS:localhost,IP:127.0.0.1" -keystore conf/server.keystore
RUN /opt/keycloak/bin/kc.sh build

FROM quay.io/keycloak/keycloak:26.8.0
COPY --from=builder /opt/keycloak/ /opt/keycloak/

# change these values to point to a running postgres instance
ENV KC_DB_URL=<DBURL>
ENV KC_DB_USERNAME=<DBUSERNAME>
ENV KC_DB_PASSWORD=<DBPASSWORD>
ENV KC_HOSTNAME=localhost
ENTRYPOINT ["/opt/keycloak/bin/kc.sh"]
```

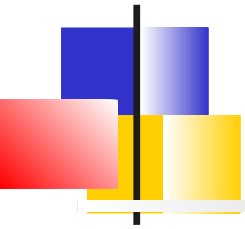


Build et démarrage

Après avoir mis au point le fichier *Dockerfile*

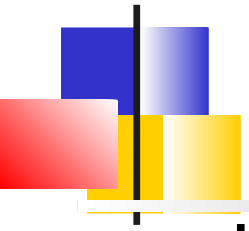
```
podman|docker build . -t mykeycloak
```

```
podman|docker run --name mykeycloak -p 8443:8443 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e
   KC_BOOTSTRAP_ADMIN_PASSWORD=change_me \
        mykeycloak \
        start --optimized
```



Haute disponibilité et clustering

Concepts
Configuration du cluster
Load balancing
Résilience et Kubernetes

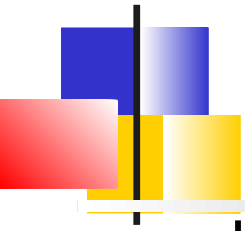


Introduction

Le clustering apporte :

- Haute disponibilité (HA)
- Scalabilité horizontale
- Réplication de sessions
- Synchronisation du cache utilisateur, tokens, sessions

Keycloak quarkus s’appuie sur 2 sous-projet Redhat indépendant ***JGroups*** et ***Infinispan***



JGroups

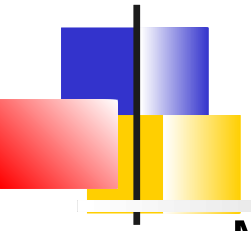
Librairie Java de communication réseau peer-to-peer

Utilisée pour la découverte des nœuds et le transport des messages

S’appuie sur différents protocoles de transport : UDP, TCP, TCPPING, KUBE_PING, JDBC-PING etc.

Fonctions clés :

- Formation du cluster et découverte des membres
- Détection de partition réseau
- Réplication des messages de cache



Infinispan – Cache distribué

Moteur de cache mémoire hautes performances

Fournit un cache distribué et répliqué

Utilisé par Keycloak pour :

- Sessions utilisateur
- Tokens
- Métadonnées clients et utilisateurs

Propose différents types de cache :

- local (non répliqué avec possibilité d’éviction)
- répliqué (synchronisation entre tous les nœuds)
- distribué (partitionné entre nœuds)

Flux de synchronisation dans un cluster

1) L’utilisateur s’authentifie sur un nœud
2) La session est stockée dans le cache Infinispan et, depuis Keycloak 26, persistée en base
3) *JGroups* transmet la mise à jour aux 2 nœuds propriétaires
4) Les autres nœuds peuvent répondre à l’utilisateur sans redirection

La ***Sticky session*** n’est pas requise si le cache est bien configuré. Elle reste recommandée : elle limite les échanges entre nœuds.

Flux de synchronisation lors d’un logout

1) L’application redirige le navigateur vers l’endpoint de logout : la requête arrive sur l’un des nœuds

2) Ce nœud retrouve la session utilisateur dans le cache distribué ou, à défaut, en base

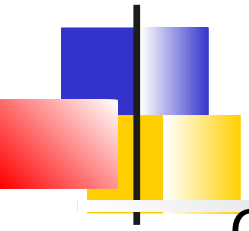
3) Il notifie les clients de la session : back-channel (appel serveur à serveur avec un logout token) ou front-channel (via le navigateur)

4) La session est supprimée du cache et de la base ; JGroups propage la suppression aux nœuds qui en détenaient une copie

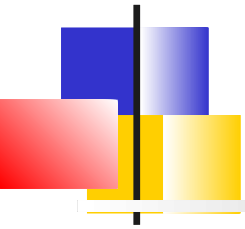
5) Sur tous les nœuds, le cookie SSO n’est plus valide, le refresh token est refusé et l’introspection répond inactive

Les access tokens déjà émis restent valides jusqu’à leur expiration : seule l’introspection voit la déconnexion immédiatement.

Bascule et persistance des sessions



- Que devient la session d’un utilisateur quand son nœud s’arrête ?
- Depuis Keycloak 26, les sessions en ligne sont persistées en base
  - Elles survivent à l’arrêt d’un nœud et au redémarrage complet du cluster
  - Le cache n’en garde qu’une partie, les autres sont relues en base
- Les logins en cours (sessions d’authentification) restent en cache seulement
  - Un login en cours sur un nœud arrêté peut être à recommencer
- Contrepartie : chaque login et chaque rafraîchissement écrit en base
  - Dimensionner la base pour la charge des connexions



Haute disponibilité et clustering

Concepts
Configuration du cluster
Load balancing
Résilience et Kubernetes

Configurer un cluster

Peu de configuration : les valeurs par défaut sont prévues pour le cluster

Même image, même configuration et même base sur tous les nœuds

--cache=ispn (défaut en production) : caches partagés

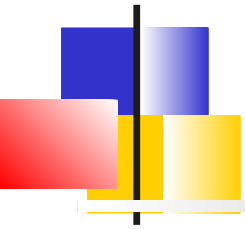
--cache-stack=jdbc-ping (défaut) : découverte des nœuds par la base

- Chaque nœud s'inscrit dans la table ***JGROUPS_PING*** : ni multicast ni DNS ; les autres stacks sont dépréciées

--cache-embedded-network-* : adresse et port JGroups, si le nœud a plusieurs interfaces

--cache-config-file : personnalisation avancée (copies, tailles)

Vérification : le log ***ISPN000094*** liste les membres du cluster



Haute disponibilité et clustering

Concepts
Configuration du cluster
Load balancing
Résilience et Kubernetes

Load balancer

Répartition devant les nœuds Keycloak :

- Sessions affines recommandées :
  - soit un cookie posé par le load balancer
  - soit la route du nœud que Keycloak ajoute au cookie AUTH_SESSION_ID
- Health checks sur **/health/ready** (port de management 9000)

Côté Keycloak :

- ***--proxy-headers=xforwarded*** ou ***forwarded***
- ***--hostname*** avec l'URL publique
- ***--http-enabled=true*** si TLS terminé au load balancer
- ***--proxy-trusted-addresses=<IP de HAProxy>***

N'exposer publiquement que les chemins nécessaires : */realms/*, */resources/*

- */admin/* réservé au réseau interne



Exemple HAProxy

```
frontend fe_keycloak
  bind *:443 ssl crt /etc/haproxy/certs/auth.pem
  option forwardfor
  http-request set-header X-Forwarded-Proto https
  http-request deny if { path_beg /admin }
  default_backend be_keycloak

backend be_keycloak
  balance roundrobin
  cookie KC_ROUTE insert indirect nocache
  option httpchk GET /health/ready
  server kc1 10.0.0.11:8080 check port 9000 cookie kc1
  server kc2 10.0.0.12:8080 check port 9000 cookie kc2
```

Keycloak démarré avec --proxy-headers=xforwarded --http-enabled=true --health-enabled=true

Arrêt gracieux



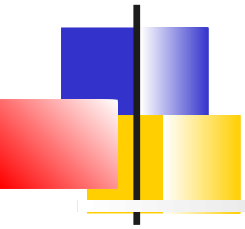
Objectif : retirer un nœud sans faire échouer de requête

Le load balancer interroge */health/ready* et retire un nœud qui n'est plus prêt

À la réception de SIGTERM, Keycloak :

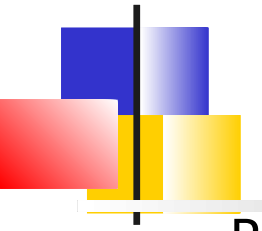
- Se déclare non prêt puis attend ***--shutdown-delay***, le temps que le load balancer le retire
- Termine les requêtes en cours (***--shutdown-timeout***)
- Quitte le cluster ; Infinispan recopie ses données sur les autres nœuds

Arrêt brutal (kill -9, panne) : détecté en quelques secondes, requêtes en cours perdues, sessions retrouvées en base



Haute disponibilité et clustering

Concepts
Configuration du cluster
Load balancing
Résilience et Kubernetes



Pannes de nœuds

Perte d’un nœud : exclu du cluster en quelques secondes

- Sessions retrouvées en base, logins en cours à recommencer

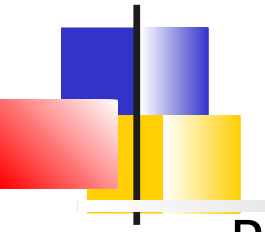
Perte de deux nœuds en même temps

- Les entrées dont les 2 copies étaient sur ces nœuds sont perdues : logins en cours, jetons d’action
- Les sessions survivent grâce à la base

Redémarrage complet du cluster : sessions conservées depuis Keycloak 26, caches reconstruits depuis la base

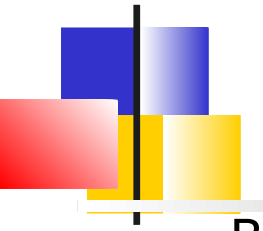
Partition réseau (split brain) : deux demi-clusters fonctionnent séparément

- Au retour du réseau, les vues fusionnent (***ISPN000093***) ; la base reste la référence



Pannes de l’infrastructure

- Perte de la base de données
  - Plus aucun login ni rafraîchissement : /health/ready passe à DOWN sur tous les nœuds
  - Les applications continuent de valider localement les access tokens déjà émis
- Perte du load balancer : interruption totale s’il est unique
  - Le doubler : keepalived (adresse IP flottante) ou load balancer managé
- Perte d’une dépendance externe (LDAP, IdP) : seuls les utilisateurs concernés ne peuvent plus se connecter
- Les vrais points de défaillance uniques sont la base et le load balancer, bien plus que les nœuds Keycloak



Base de données externe

Base partagée par tous les nœuds : point de défaillance à rendre HA

- Réplication PostgreSQL, service managé, sauvegardes

Sessions utilisateur persistées en base (défaut Keycloak 26) : dimensionner la BD en conséquence

Pool de connexions :

- *--db-pool-initial-size, --db-pool-min-size, --db-pool-max-size*
- La somme des pools de tous les nœuds doit rester sous la limite de connexions de la base

Découverte des nœuds du cluster :

- --cache=ispn --cache-stack=jdbc-ping (via la base) (défaut) ; kubernetes et les autres stacks sont dépréciées

Keycloak Operator

- Opérateur Kubernetes officiel : déploiement, configuration, montée de version
- CRD *Keycloak* et *KeycloakRealmImport*

```
apiVersion: k8s.keycloak.org/v2alpha1
kind: Keycloak
metadata:
  name: keycloak
spec:
  instances: 3
  db:
    vendor: postgres
    host: postgres-rw
    usernameSecret: { name: kc-db, key: username }
    passwordSecret: { name: kc-db, key: password }
  http:
    tlsSecret: keycloak-tls
  hostname:
    hostname: auth.example.com
  proxy:
    headers: xforwarded
```

Résilience sur Kubernetes

Sondes, configurées par l'Operator :

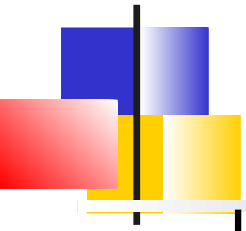
- startup sur /health/started,
  liveness sur /health/live,
  readiness sur /health/ready
- Jamais /ready en liveness : une panne de base ferait redémarrer tous les pods en boucle

Anti-affinité : pods répartis sur des nœuds et des zones différents

PodDisruptionBudget : au plus un pod indisponible pendant les maintenances (drain)

Requests et limits : mémoire cohérente avec le heap de la JVM, sinon le pod est tué (OOMKilled)

Arrêt : ***terminationGracePeriodSeconds*** supérieur à shutdown-delay + shutdown-timeout



Multi-site

- Un cluster protège contre la perte d’un nœud, pas contre celle d’un datacenter
- Architecture multi-site supportée par Keycloak :
  - Deux sites actifs, chacun avec son propre cluster Keycloak
  - Base de données répliquée de façon synchrone entre les deux sites
  - Un Infinispan externe par site, répliqué entre les sites, pour partager les sessions
  - Un load balancer global qui bascule le trafic en cas de perte d’un site
- Elle impose une latence faible entre les sites et une exploitation plus lourde : à réserver aux fortes exigences de disponibilité



Résilience entre sites

Cluster étiré : un seul cluster réparti sur plusieurs sites

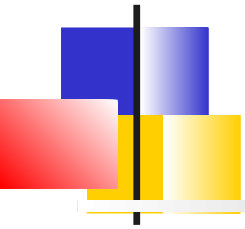
- Viable entre sites proches (faible latence), surtout depuis la persistance des sessions en base (Keycloak 26)
- Simple, mais une panne ou une erreur de cluster touche tous les sites

Multi-site : deux clusters synchronisés (base répliquée, Infinispan externe)

- Plus complexe, mais isole les sites : montée de version site par site, retour possible sur l'autre
- Seule architecture documentée par Keycloak

Dans les deux cas : base répliquée de façon synchrone entre sites, donc sites proches

À tester : perte d'un site et coupure du lien entre sites



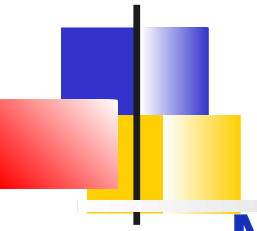
Configuration avancée et durcissement

Performances

Gestion des secrets

Protection du serveur

Comptes et applications



Paramètres JVM et performances

Mémoire : par défaut dimensionnée en pourcentage de la mémoire disponible (MaxRAMPercentage)

- Adapter via JAVA_OPTS_APPEND, fixer les limites du conteneur

CPU : le hachage des mots de passe est le principal consommateur

- Coût du hachage à arbitrer dans la politique de mot de passe (argon2 par défaut, hors FIPS)

Mémoire : caches de sessions et de configuration

Démarrage : image optimisée (kc.sh build, --optimized)

Base de données : taille du pool, index, latence réseau

Mesurer avant d'ajuster : métriques JVM et HTTP (voir supervision)



Pool de connexions (Agroal)

- Chaque accès à la base emprunte une connexion à un pool : en ouvrir une à chaque fois serait trop coûteux
- Chaque nœud a son propre pool :
  - ***--db-pool-initial-size***, ***--db-pool-min-size***, ***--db-pool-max-size*** (défaut 100)
  - Somme des pools de tous les nœuds ≤ limite de connexions de la base
- Pool trop petit : les requêtes attendent une connexion, la latence monte, puis des erreurs apparaissent
- Timeouts et validation : propriétés Quarkus brutes dans conf/quarkus.properties (non supporté)
  - ***quarkus.datasource.jdbc.acquisition-timeout***, ***… background-validation-interval***
- À surveiller : ***agroal_active_count***, ***agroal_awaiting_count*** (doit rester à 0)

Calibrage des sessions

Les durées de session sont un compromis entre confort, sécurité et charge

SSO Session Idle (30 min) et Max (10 h) : durée de connexion sans ressaisie du mot de passe

Access token lifespan (5 min) : un jeton d’accès se révoque mal, il doit rester court

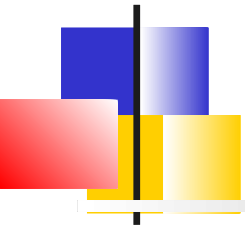
Client session idle et max : raccourcir la durée pour un client sensible, sans toucher au SSO

Offline sessions : applications qui travaillent sans l’utilisateur ; limiter leur durée maximale

Impact en production



- Nombre de sessions actives ≈ logins par heure × durée de session
  - Il détermine la mémoire des caches et le nombre de lignes en base
- Chaque login et chaque rafraîchissement écrit en base
  - Des access tokens très courts multiplient les rafraîchissements, donc les écritures
- Des offline sessions sans limite s’accumulent en base et allongent les migrations
- Ajuster puis mesurer : nombre de sessions (console, métriques des caches), charge de la base



Configuration avancée et durcissement

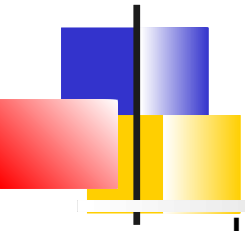
Performances

Gestion des secrets

Protection du serveur

Comptes et applications

Secrets de configuration du serveur



Le serveur a besoin de secrets dès son démarrage : mot de passe de la base, des keystores

Variables d'environnement : simples, mais lisibles par qui accède à l'hôte (docker inspect, /proc)

Keystore de configuration : options sensibles lues dans un keystore chiffré

- ***--config-keystore***, ***--config-keystore-password*** ;
  entrée ***kc.db-password*** créée avec ***keytool -importpass***

HashiCorp Vault : Keycloak ne sait pas faire des appels directs à son API

- Vault Agent, CSI driver (Kubernetes) ou External Secrets (Kubernetes) fournissent fichiers ou variables au conteneur

Jamais de secret dans l'image ni dans Git

Vault de realm



Les realms contiennent aussi des secrets : mot de passe SMTP, bind credential LDAP, secret client des IdP

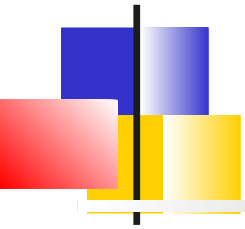
- Stockés en clair en base, ils apparaissent dans les exports

Le vault les remplace par une référence, résolue à l'usage : ***${vault.smtp_password}***

File vault : ***--vault=file --vault-dir=/opt/keycloak/vault***

- Un fichier par secret, ***<realm>_<clé>*** avec « _ » doublés : ***formation_smtp__password***
- Se combine avec les secrets Kubernetes ou Vault Agent montés en fichiers

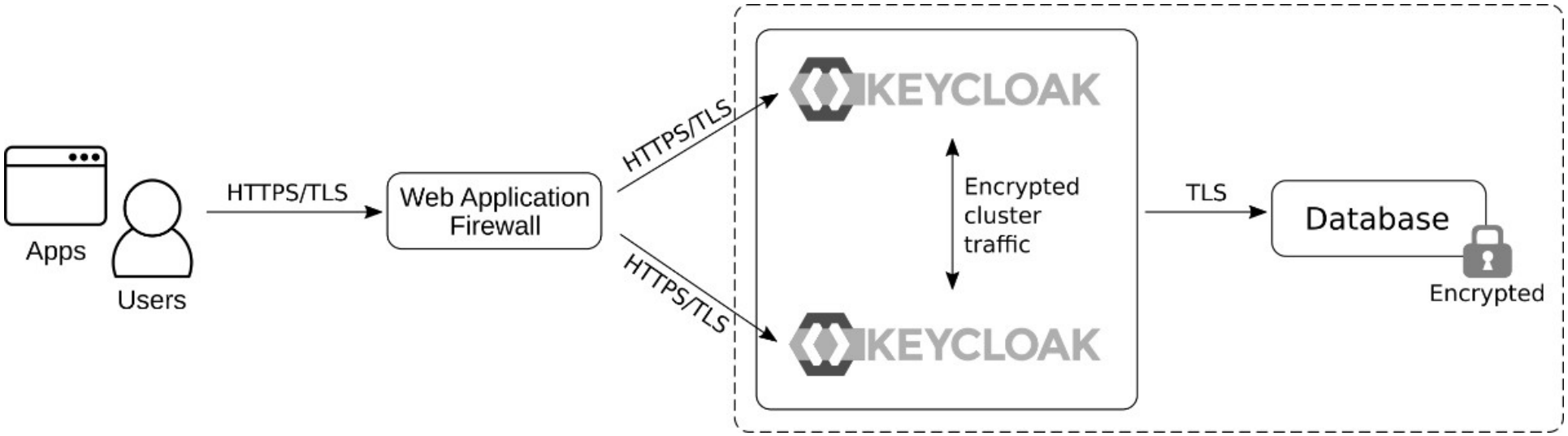
Keystore vault :
--vault=keystore --vault-file=… --vault-pass=…



Configuration avancée et durcissement

Performances
Gestion des secrets
Protection du serveur
Comptes et applications

Architecture sécurisée



Sécurisation Keycloak

Keycloak protège toutes les applications : le compromettre, c'est les compromettre toutes. D'où une défense en couches

Réseau : TLS 1.3 de bout en bout (librairies d'intégration compatibles), ré-établi au proxy plutôt que passthrough ; pare-feu et IPS

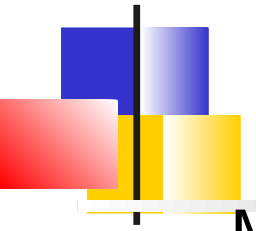
Serveur : hostname configuré (sinon déduit de l'en-tête Host, falsifiable) ; mises à jour régulières ; secrets dans un vault externe

Clés : rotation des clés de signature, par exemple 1 fois par mois (Realm Settings → Keys)

Données : base, sauvegardes et échanges du cluster

Realms : brute force, en-têtes de sécurité, clients stricts

Endpoints exposés



Moins de chemins exposés, moins de surface d'attaque

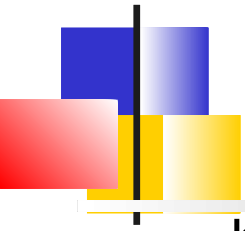
Publics : ***/realms/*** (login, jetons, discovery) et ***/resources/*** (styles, scripts)

Réservés au réseau interne :

- ***/admin/*** : console et API d'administration
- Port 9000 : ***/health*** et ***/metrics*** révèlent l'état du serveur
- ***/realms/master/*** : le realm des super-administrateurs

Console sur une adresse interne : ***--hostname-admin*** ; filtrage au proxy et au pare-feu

Sécuriser la BD

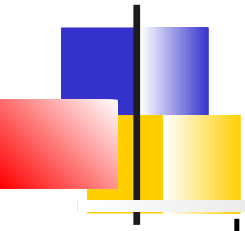


Keycloak stocke beaucoup de données sensibles dans sa base, certaines en clair :

- Secrets des clients, des fournisseurs d'identité, du bind LDAP et du SMTP (sauf vault), graines OTP
- Mots de passe hachés ; sessions persistées en base depuis Keycloak 26
- On doit sécuriser la BD, ses backups et les exports complets

Recommandations minimales :

- Protéger la BD avec un pare-feu : seuls les nœuds Keycloak y accèdent
- Activer l'authentification et le contrôle d'accès : compte dédié à Keycloak, droits limités à son schéma
- Chiffrer les connexions entre Keycloak et la base (TLS)
- Chiffrer les backups et les exports



Cluster

Les nœuds échangent les sessions utilisateur

- Moins sensibles que la base, ces échanges doivent quand même être protégés

Deux exigences :

- Authentifier les nœuds : seul un nœud légitime peut rejoindre le cluster
- Chiffrer les échanges entre nœuds

Depuis Keycloak 26 : mTLS JGroups activé par défaut (--cache-embedded-mtls-enabled), certificats générés et renouvelés automatiquement soit par Keycloak soit en intégrant sa solution de PKI

Ports du cluster (7800, 57800) accessibles uniquement entre nœuds Keycloak

Brute force

Sans protection, un attaquant peut tester des milliers de mots de passe

Désactivée par défaut :
Realm settings → Security defenses → Brute force detection

Verrouillage temporaire (durée croissante), permanent, ou permanent après plusieurs verrouillages temporaires

Paramètres : Max login failures, Wait increment, Max wait, Failure reset time

Quick login check : bloque les tentatives trop rapprochées (robots)

Attention : le verrouillage permanent permet à un attaquant de bloquer des comptes connus, c'est un déni de service ; préférer le temporaire

Protection par compte, pas par IP : la compléter par une limitation de débit au proxy ou au WAF

Les échecs sont partagés entre les nœuds (cache loginFailures)



En-têtes et clients

Realm settings → Security defenses → Headers

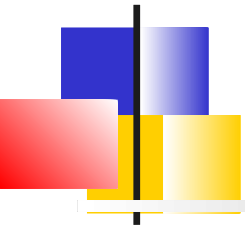
Content-Security-Policy, X-Frame-Options : protection contre le clickjacking

X-Content-Type-Options, X-Robots-Tag

Strict-Transport-Security : HTTPS obligatoire

Valeurs par défaut sûres : ne les assouplir que pour un besoin précis (ex. page de login dans une iframe)

Clients : redirect URIs exactes sans joker, web origins restreintes



Configuration avancée et durcissement

Performances
Gestion des secrets
Protection du serveur
Comptes et applications

Comptes administrateurs

Compte bootstrap temporaire au 1er démarrage, à remplacer par des comptes nominatifs

Realm master limité aux super-administrateurs

Administrateurs de realm via les rôles du client realm-management

MFA obligatoire pour les comptes d’administration

Console d’administration non exposée publiquement :
--hostname-admin, filtrage au proxy

Comptes de service dédiés pour l’automatisation, rôles minimaux

Traçabilité :
admin events activés

Recommandations OAuth 2.0

OAuth 2.0 et OpenID Connect sont très flexibles : certaines configurations ne sont pas sécurisées

Références (oauth.net/2) :

- RFC 9700 (2025) : bonnes pratiques de sécurité, met à jour le modèle de menace (RFC 6819)
- RFC 8252 : applications natives et mobiles
- OAuth 2.0 for Browser-Based Apps (draft) : SPA et pattern BFF
- OAuth 2.1 (draft) : intègre ces bonnes pratiques dans la spécification

L'essentiel :

- Code flow + PKCE (S256) pour tous les clients ; ni implicit, ni password grant
- Redirect URIs exactes ; jamais de jeton dans l'URL
- Jetons d'accès courts ; rotation des refresh tokens pour les clients publics
- Jetons liés au client (DPoP, mTLS) dans les contextes sensibles

Dans Keycloak : les client policies (profils FAPI) imposent ces règles à tous les clients

Configuration des clients

Consentement requis : activer cette option pour toute application tierce

Client authentication : ON (client confidentiel) si l'on peut conserver le secret en toute sécurité côté serveur ; sinon client public

- Mieux qu'un secret partagé : JWT signé ou certificat (mTLS)

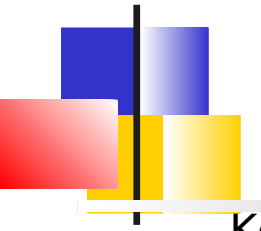
PKCE : méthode S256 imposée pour les clients publics (onglet Advanced)

Implicit flow : jamais, sauf application legacy

Direct access grants : jamais, sauf application legacy

Valid redirect URIs : correspondance exacte, sans joker

Web origins : limitées aux origines de l'application, jamais *



Algorithmes de signature

Keycloak supporte plusieurs algorithmes de signature :

- **RSA (RS256, PS256)** : algorithme par défaut, le plus largement supporté ; PS256 exigé par FAPI
- **ECDSA (ES256)** : même sécurité avec des clés et des jetons plus courts ; signature plus rapide, vérification plus lente
- **EdDSA (Ed25519)** : rapide en signature comme en vérification ; support à vérifier dans les bibliothèques clientes
- **HMAC (HS256…)** : symétrique, à secret partagé ; réservé aux jetons internes de Keycloak (refresh tokens)

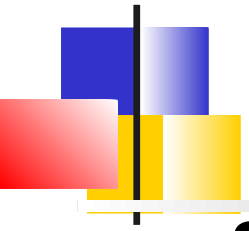
Longueur de hachage : avec des jetons de courte durée de vie, 256 bits est considéré correct

Réglage :
Realm settings → Tokens (défaut du realm) ou par client (*Advanced*)

- Nécessite une clé du type choisi (*Realm settings → Keys*)

Avant de changer : vérifier que toutes les applications et API savent valider l'algorithme choisi

Checklist de mise en production (1/2)



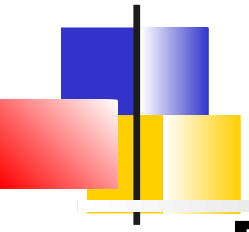
Serveur

- Mode start (jamais start-dev), image optimisée
- TLS de bout en bout, hostname configuré, proxy-headers cohérents
- Au moins 2 nœuds, health checks, sessions affines
- Base de données externe HA et sauvegardée

Administration

- Console admin non exposée, comptes nominatifs avec MFA
- Compte bootstrap supprimé

Checklist de mise en production (2/2)

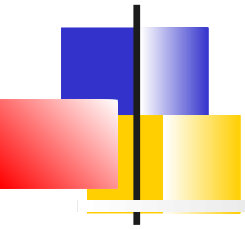


Realms et clients

- Politique de mot de passe, brute force detection, en-têtes de sécurité
- Durées de sessions et de jetons revues, rotation des refresh tokens
- Redirect URIs exactes, full scope désactivé, consentement pour les applications tierces

Exploitation

- Events activés avec expiration
- Métriques et logs centralisés
- Procédure de sauvegarde et de restauration testée



Exploitation d’un cluster

Gestion des nœuds

Montées de version

Diagnostic du cluster

Incidents courants



Ajouter un nœud à chaud

Ajouter un nœud permet d’absorber une hausse de charge ou de remplacer une machine, sans interruption

Le nouveau nœud démarre avec la même image et la même configuration

Il se découvre via *jdbc-ping* et rejoint la vue du cluster

Infinispan rééquilibre les caches distribués : le nouveau nœud reçoit sa part des entrées

Le load balancer ne lui envoie du trafic qu’une fois /health/ready au vert

Sur Kubernetes, il suffit de modifier le nombre d’instances : l’Operator fait le reste

Retirer un nœud



Retrait pour maintenance, mise à jour ou réduction de capacité

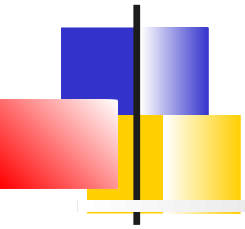
Arrêt par SIGTERM : readiness, shutdown-delay et load balancer assurent la transition

Les nœuds restants rééquilibrent les caches : chaque entrée retrouve 2 copies

Un nœud à la fois, en attendant la fin du rééquilibrage

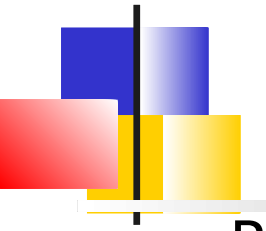
- Avec 2 copies par entrée, arrêter deux nœuds ensemble peut perdre des entrées
- Sessions retrouvées en base ; logins en cours et jetons d’action perdus

Arrêt brutal : détection de défaillance, nouvelle vue du cluster, requêtes en cours perdues



Exploitation d’un cluster

Gestion des nœuds
Montées de version
Diagnostic du cluster
Incidents courants



Politique de versions

Plusieurs versions majeures par an

- Seule la dernière version reçoit les correctifs de la communauté
- Support long terme : ***Red Hat build of Keycloak***

Suivre les versions correctives : failles de sécurité (CVE)

Avant chaque montée de version, lire :

- Les release notes
- Le guide Upgrading :
  changements incompatibles, options supprimées

Monter régulièrement : plusieurs versions d'écart multiplient les changements à absorber



Rolling update ou recreate

Deux stratégies pour changer de version ou de configuration

Rolling update : nœuds remplacés un par un, le service reste disponible

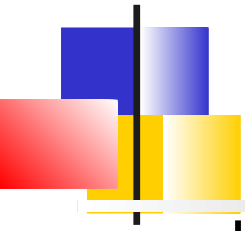
- Possible seulement si l'ancienne et la nouvelle version forment un cluster ensemble

Recreate : arrêt de tous les nœuds, puis redémarrage dans la nouvelle version

- Interruption de service, à planifier

Rolling : thème, provider, option, version corrective d'une même major.minor

Recreate : version mineure ou majeure, changement de cache, de stack ou de base



update-compatibility

Keycloak fournit une commande qui tranche, utilisable dans un pipeline

Photographie de l'existant :

```
kc.sh update-compatibility metadata --file=old.json
```

Vérification de compatibilité avec la nouvelle version :

```
kc.sh update-compatibility check --file=old.json
```

Code retour 0 : rolling update possible ; 3 : recreate obligatoire

L'Operator Kubernetes l'utilise pour sa stratégie Auto

Montée de version

Préparation :

- Sauvegarde de la base
- Recompilation des providers sur la nouvelle version, vérification des thèmes
- Test en recette sur une copie de la base de production
  - kc.sh update-compatibility check : rolling update possible ou arrêt complet nécessaire ?

Exécution :

- Arrêt de tous les nœuds pour une version majeure
- Nouvelle image, démarrage d'un nœud : migration automatique du schéma
- Contrôles puis démarrage des autres nœuds

Versions correctives : mise à jour progressive possible (Operator, stratégie Auto)

Migration de la base

Au démarrage, Keycloak met à jour le schéma (Liquibase)

Mode manuel pour faire valider le script par un DBA :

```
kc.sh start --spi-connections-jpa--quarkus--migration-strategy=manual
```

Génère *keycloak-database-update.sql* sans l'appliquer

Points d'attention :

- Pas de retour arrière hors restauration de la base
- Durée de migration sur les grosses bases (sessions, événements) : purger avant
- Compatibilité des applications et des librairies clientes



Retour arrière

Une montée de version peut échouer après coup : régression fonctionnelle, provider incompatible

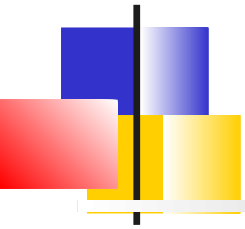
Après une migration de schéma, l'ancienne version ne peut plus démarrer sur la base

- Retour arrière = restauration de la sauvegarde et de l'ancienne image
- Les sessions et modifications faites depuis la sauvegarde sont perdues

Sans changement de schéma (rolling update) : redéployer l'ancienne image nœud par nœud

À préparer avant l'opération :

- Critères de go/no-go, tests de validation, durée de la fenêtre
- Ancienne image et sauvegarde conservées jusqu'à la validation



Exploitation d’un cluster

Gestion des nœuds
Montées de version
Diagnostic du cluster
Incidents courants

État du cluster

Un cluster dégradé fonctionne souvent encore : il faut chercher les signes

Logs : ***ISPN000094*** nouvelle vue avec les membres ; ***ISPN000093*** vue fusionnée après une partition (split brain)

Métriques :

- ***vendor_cluster_size*** : membres vus par chaque nœud, identique partout
- ***vendor_jgroups_stats_sync_requests_seconds*** : temps de réponse entre nœuds
- ***vendor_statistics_approximate_entries***, ***…_evictions*** : remplissage des caches

Base : table ***JGROUPS_PING*** des nœuds inscrits ; réseau : ports 7800 et 57800, horloges

Dégradation des performances



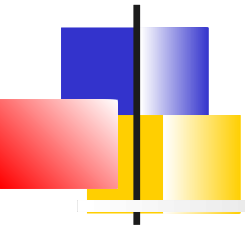
Une dégradation se voit d'abord dans la latence, avant les erreurs

Symptômes :

- Latence p95 de /token en hausse, requêtes HTTP actives qui s'accumulent
- ***agroal_awaiting_count*** > 0 : pool de connexions saturé
- Pauses GC, CPU saturé par le hachage, timeouts de réplication Infinispan

Isoler la couche : un nœud ou tous ? base ? réseau ? dépendance externe ?

Logs JSON par nœud, DEBUG ciblé (***org.infinispan***, ***org.jgroups***), outils JVM (***jcmd***, logs GC)



Exploitation d’un cluster

Gestion des nœuds
Montées de version
Diagnostic du cluster
Incidents courants

Démarche de diagnostic

Qualifier : tous les utilisateurs, un realm, un client, un nœud ?

Vérifier l’état : /health/ready de chaque nœud, métriques

Consulter :

- Les logs Keycloak et du proxy, sur la période concernée
- Les events LOGIN_ERROR et leur code d’erreur

Isoler la couche : proxy, Keycloak, base, cluster, fournisseurs externes

Reproduire avec curl et un jeton décodé

Augmenter le niveau de log de façon ciblée et temporaire

Incidents courants (1/2)

Redirections vers une mauvaise URL, erreurs d'issuer

- hostname ou proxy-headers mal configurés

Boucle de login, « Cookie not found »

- Sessions affines absentes, cookies SameSite, horloges désynchronisées

Démarrage bloqué

- Base inaccessible, migration en cours, verrou Liquibase (table DATABASECHANGELOGLOCK)

Les nœuds ne forment pas de cluster

- Découverte JGroups (jdbc-ping, DNS), port 7800 filtré

Incidents courants (2/2)

Lenteurs lors des pics de connexion

- CPU saturé par le hachage, pool BD épuisé, mémoire (OOMKilled)

Utilisateurs LDAP rejetés

- Bind credential expiré, certificat LDAPS, annuaire injoignable

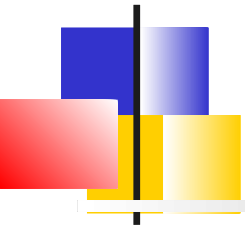
401 généralisés côté API

- Rotation de clés trop rapide, cache JWKS des applications

Emails non reçus

- Configuration SMTP, test depuis Realm settings → Email

Perte d'accès administrateur : kc.sh bootstrap-admin



Supervision, maintenance et sauvegarde

Métriques et supervision

Logs et événements

Sauvegarde et restauration

Bonnes pratiques et audit

Métriques Keycloak

Keycloak expose des métriques au format Prometheus

- Activation : --metrics-enabled=true
- Endpoint /metrics sur le port de management (9000)

Familles de métriques :

- JVM : mémoire, GC, threads
- HTTP : nombre et durée des requêtes par URI et statut
- Pool de connexions BD (agroal_*)
- Caches Infinispan

Métriques d'événements utilisateur :
--event-metrics-user-enabled=true

- Compteur keycloak_user_events_total par realm, client, type et erreur

Métriques utiles

Activité : logins réussis et en échec, émission et rafraîchissement de jetons

Performance : latence des endpoints token et auth (percentiles)

- Histogrammes : --http-metrics-histograms-enabled=true

Ressources : heap, pauses GC, CPU

Base de données : connexions actives, requêtes en attente de connexion

Cluster : taille du cluster, statistiques des caches

Disponibilité : état readiness des nœuds

Prometheus

Collecte des métriques de chaque nœud :

```
scrape_configs:
  - job_name: keycloak
    metrics_path: /metrics
    scheme: https
    static_configs:
      - targets: ["kc1:9000", "kc2:9000"]
```

Sur Kubernetes : ServiceMonitor ou PodMonitor (Prometheus Operator)

Exemples d’alertes :

- Taux de LOGIN_ERROR anormal
- Latence p95 de l’endpoint token dégradée
- Requêtes en attente de connexion BD, nœud non ready

Dashboards Grafana



Le projet Keycloak fournit des tableaux de bord Grafana prêts à l'emploi

- Troubleshooting : erreurs, latences, état du cluster et des caches
- Capacity planning : charge, volumétrie des sessions, dimensionnement

Organisation conseillée :

- Vue synthèse : disponibilité, logins par minute, taux d'erreurs
- Vue par nœud : JVM, CPU, pool BD
- Vue par realm et client : événements utilisateur



Health checks

- Activation : --health-enabled=true, exposés sur le port de management

```
/health/started # démarrage terminé
/health/live # processus vivant
/health/ready # prêt à recevoir du trafic (BD accessible)
/health # agrégat
```

- Usages :
  - Sondes startup, liveness et readiness de Kubernetes
  - Health checks du load balancer
  - Supervision externe (disponibilité)
- Ne pas exposer le port de management publiquement

Supervision globale

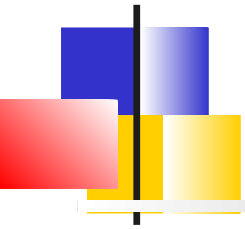


Supervision applicative :

- Sonde synthétique : parcours de login complet et obtention d'un jeton
- Validité du endpoint discovery et des clés JWKS

Supervision infrastructure :

- Nœuds : CPU, mémoire, disque, redémarrages de pods
- Base de données : réplication, espace, verrous, sauvegardes
- Réseau : load balancer, certificats TLS et leur expiration
- Dépendances : LDAP, IdP externes, SMTP



Supervision, maintenance et sauvegarde

Métriques et supervision

Logs et événements

Sauvegarde et restauration

Bonnes pratiques et audit

Configuration des logs

Première source de diagnostic : tout se règle par des options du serveur, sans fichier de configuration dédié

Niveau global et par catégorie :

- --log-level=INFO,org.keycloak.events:debug,org.infinispan:warn

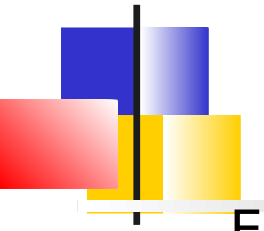
Sorties : --log=console,file,syslog

- console : par défaut, adaptée aux conteneurs
- file : --log-file=… (rotation à prévoir)
- syslog : vers un collecteur ou un SIEM

Format : texte par défaut, JSON pour l'indexation : --log-console-output=json

Niveau propre à chaque sortie : --log-console-level, --log-file-level…

Options d'exécution : modifiables sans reconstruire l'image (ex. KC_LOG_LEVEL)



Logs : bonnes pratiques

En production : INFO global, WARN pour les bibliothèques bavardes

Diagnostic : DEBUG ponctuel et ciblé sur une catégorie

- org.keycloak.authentication (flows), org.keycloak.protocol.oidc
- org.keycloak.storage.ldap (fédération), org.infinispan et org.jgroups (cluster)

Attention : changer le niveau impose un redémarrage ; en cluster, nœud par nœud

Données sensibles : en DEBUG ou TRACE, des jetons ou des attributs peuvent apparaître

- Jamais durablement en production ; logs protégés comme des données personnelles

Corrélation : JSON avec le nom du nœud ; traçage distribué OpenTelemetry (--tracing-enabled) pour suivre une requête de l'application jusqu'à Keycloak

Centralisation des logs

- Sortie JSON vers la console, collectée par la plateforme

```
kc.sh start --log-console-output=json
```

- Chaînes classiques :
  - Fluent Bit ou Vector → Loki ou Elasticsearch → Grafana ou Kibana
  - Syslog : --log=console,syslog --log-syslog-endpoint=siem:514
- Rétention définie selon les exigences d’audit et le RGPD

Événements vers un SIEM

Le listener jboss-logging écrit les événements dans les logs

- Erreurs en WARN, succès en DEBUG par défaut
- Tracer aussi les succès en INFO :

```
--spi-events-listener--jboss-logging--success-level=info
```

Listener personnalisé (SPI EventListenerProvider) :

- Envoi vers Kafka, un webhook HTTP ou directement le SIEM
- Découple l’audit de la base Keycloak

Garder une expiration courte des événements stockés en base

Audit d’authentification

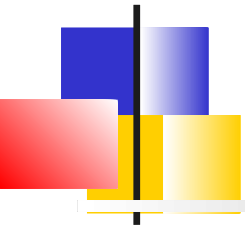
Indicateurs à surveiller :

- Échecs répétés par utilisateur ou par IP (brute force, credential stuffing)
- Connexions depuis des IP ou horaires inhabituels
- Comptes verrouillés, réinitialisations de mot de passe

Admin events sensibles :

- Attribution de rôles d’administration
- Régénération de secrets clients, modification des clés
- Création de clients ou de fournisseurs d’identité

Revue périodique et alertes sur les événements critiques



Supervision, maintenance et sauvegarde

Métriques et supervision
Logs et événements
Sauvegarde et restauration
Bonnes pratiques et audit

Que sauvegarder ?

La base de données : source de vérité

- Realms, clients, utilisateurs, credentials, clés de signature, sessions persistées

Les fichiers du serveur :

- conf/keycloak.conf, providers/, themes/
- Certificats TLS et keystores
- Dockerfile ou image, manifests Kubernetes

Les secrets : mots de passe BD, secrets des comptes de service

Les exports de realms : complément utile, pas un substitut à la sauvegarde BD



Sauvegarde de la base

Exemple PostgreSQL :

```
pg_dump -Fc -h db -U keycloak keycloak > kc-$(date +%F).dump
```

Sauvegarde cohérente à chaud (transactionnelle)

Pour un RPO faible : archivage continu des WAL et restauration à un instant donné (PITR)

Service managé : sauvegardes automatiques du fournisseur

Chiffrer les sauvegardes : elles contiennent hachés, secrets et clés privées

Copie hors site et durée de rétention définie

Export et import de realms

En ligne de commande, sur une instance arrêtée ou dédiée :

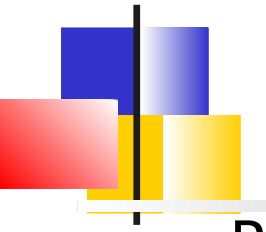
```
kc.sh export --dir /backup --realm myrealm --users realm_file
kc.sh import --dir /backup --override false
```

Au démarrage : *start --import-realm* (fichiers de data/import, realm existant non écrasé)

Depuis la console : Realm settings → Action → Partial export

- Sans utilisateurs, secrets masqués

Usages : migration entre environnements, initialisation, archive de configuration



Restauration

- Procédure type :
  - Arrêter tous les nœuds
  - Restaurer la base : pg_restore --clean -d keycloak kc.dump
  - Redéployer la même version de Keycloak, providers et thèmes
  - Démarrer un seul nœud, contrôler logs et /health/ready
  - Tester un login et l’obtention d’un jeton, puis démarrer les autres nœuds
- Les caches embarqués sont reconstruits au redémarrage
- Tester la restauration régulièrement, sur un environnement isolé

Planification des sauvegardes

Exemple : CronJob Kubernetes quotidien

```
apiVersion: batch/v1
kind: CronJob
metadata: { name: kc-db-backup }
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: pgdump
              image: postgres:16
              envFrom: [ { secretRef: { name: kc-db } } ]
              command: ["sh", "-c", "pg_dump -Fc -h postgres-rw \
                -U keycloak keycloak > /backup/kc-$(date +%F).dump"]
              volumeMounts: [ { name: backup, mountPath: /backup } ]
          volumes: [ { name: backup, persistentVolumeClaim: { claimName: kc-backup } } ]
```

Sauvegardes : organisation

- Sauvegarde BD quotidienne, plus WAL si RPO faible
- Export de configuration des realms versionné (partial export)
- Rotation des sauvegardes, chiffrement et copie hors site
- Supervision : alerte si une sauvegarde échoue ou manque
- Test de restauration planifié (mensuel par exemple)

Fenêtres de maintenance

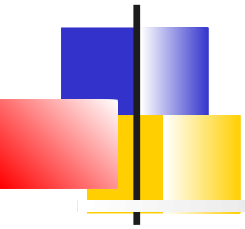
Opérations à planifier :

- Montées de version, changements de configuration serveur
- Rotation des clés de signature des realms
- Purge des événements, renouvellement des certificats

Rotation de clé sans coupure :

- Ajouter une nouvelle clé active de priorité supérieure
- Garder l’ancienne en passive le temps de validité des jetons
- Puis la désactiver et la supprimer

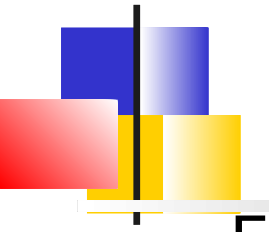
Retirer un nœud du load balancer avant intervention, communiquer auprès des équipes applicatives



Supervision, maintenance et sauvegarde

Métriques et supervision
Logs et événements
Sauvegarde et restauration
Bonnes pratiques et audit

Bonnes pratiques d’exploitation



- Environnements homogènes, configuration versionnée
- Modifications en production via le pipeline, pas depuis la console
- Runbooks pour les opérations et incidents courants
- Alertes sur la disponibilité, les erreurs et les sauvegardes
- Versions correctives appliquées rapidement
- Rotation des secrets et des clés
- Revue périodique des comptes d’administration et des clients

Checklist d’audit (1/2)

Comptes et accès

- Administrateurs nominatifs, MFA, droits minimaux
- Comptes de service : rôles justifiés, âge des secrets
- Comptes inactifs ou orphelins

Clients

- Clients inutilisés à supprimer
- Redirect URIs et web origins exactes
- Type public ou confidentiel adapté, PKCE, full scope désactivé

Checklist d’audit (2/2)

Configuration des realms

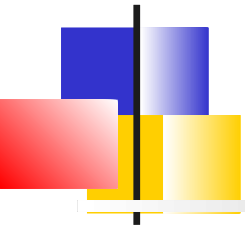
- Politique de mot de passe, brute force detection, en-têtes
- Durées de sessions et de jetons
- Clés : algorithmes, ancienneté

Plateforme

- Version supportée, CVE corrigées
- TLS, console admin non exposée, port de management protégé
- Sauvegardes testées, rétention des événements, alertes actives



Merci pour votre attention !!



Annexes

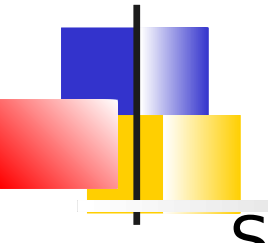
SAML 2.0

FAPI

Authentification mTLS et CIBA

Maximiser la sécurité des standards

SAML 2.0 : acteurs et assertion



Standard OASIS (2005) basé sur XML

Identity Provider (IdP) : authentifie l'utilisateur et émet les assertions

Service Provider (SP) : l'application qui consomme l'assertion

Assertion : document XML signé par l'IdP

- **Subject** : identifiant de l'utilisateur (NameID)
- **Attributs** : email, nom, groupes…
- **Conditions** : période de validité, audience (SP destinataire)

L'assertion joue le rôle de l'ID token d'OpenID Connect

Flux SSO initié par le SP

L’utilisateur accède à l’application (SP)

Le SP redirige le navigateur vers l’IdP avec une *AuthnRequest* (binding HTTP-Redirect)

L’IdP authentifie l’utilisateur ou réutilise sa session SSO

L’IdP renvoie une page qui poste la réponse au SP (binding HTTP-POST)

- Vers l’Assertion Consumer Service (ACS) du SP

Le SP vérifie signature, audience et validité, puis ouvre sa session

Tous les échanges transitent par le navigateur

Variante initiée par l’IdP (portail d’applications), Single Logout (SLO)

Confiance et métadonnées

La confiance IdP / SP repose sur l'échange de métadonnées XML

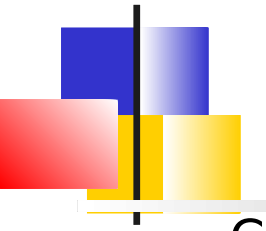
- EntityID : identifiant unique de chaque partie
- Endpoints : SSO, ACS, Single Logout
- Certificats de signature et de chiffrement

NameID : format de l'identifiant (persistent, transient, email)

Mapping des attributs entre IdP et SP

Dans Keycloak :

- IdP SAML : client de type saml, métadonnées sur /realms/{realm}/protocol/saml/descriptor
- SP : fédération d'un IdP SAML externe (ADFS, Entra ID…)



SAML 2.0 et OpenID Connect

Concepts équivalents :

- IdP ↔ OpenID Provider, SP ↔ Relying Party
- Assertion ↔ ID token
- Métadonnées ↔ discovery
- ACS ↔ redirect_uri
- Single Logout ↔ RP-Initiated et Back-Channel Logout

SAML 2.0 : XML, échanges uniquement via le navigateur, pas de jeton d'accès pour appeler des API

- À conserver pour les applications existantes et les progiciels ne supportant que SAML

OpenID Connect : JSON et JWT, adapté aux SPA, au mobile et aux API

- Choix recommandé pour les nouvelles applications

Annexes

FAPI

Authentification mTLS et CIBA

Maximiser la sécurité des standards

Financial Grade API

Financial-grade API (FAPI) est une spécification technique qui durcit oAuth2 et OpenID

- Publiée par l'OpenID Foundation, avec une adoption croissante dans les environnements réglementés (notamment PSD2[1] en Europe).

Objectifs principaux :

- Protéger contre les attaques de type interception de jeton, redirection malveillante, injection de code, etc.
- Renforcer l'authentification et la signature des requêtes.
- Garantir l'intégrité des messages et l'identité du client (preuve de possession).

1. Payment Services Directive 2

Exigence FAPI

- Flow OpenID : **Authorization Code + PKCE** obligatoire
- Authentification client : **mTLS** ou **Private Key JWT**[1]
- Jetons : Doivent être **signés et chiffrés**
- ID Token : Doit inclure des **claims supplémentaires** (`acr`, `auth_time`, etc.)
- Délai de validité : Jetons courts, rotation imposée
- Preuve de possession : Recommandation : **DPoP ou mTLS**

1. Dans ce cas, client génère un JWT signé avec sa clé privé et l'envoie via le paramètre *client_assertion*

Financial-Grade API (FAPI)

- FAPI est un groupe de travail qui essaie de spécifier des profiles de configuration OIDC pour le domaine bancaire. 2 profils sont définis :
  - FAPI 1.0 – Part 1: Baseline API Security Profile
    https://openid.net/specs/openid-financial-api-part-1-1_0.html
  - FAPI 1.0 – Part 2: Advanced Security Profile
    https://openid.net/specs/openid-financial-api-part-2-1_0.html
- Keycloak 15.0.2 a été certifié FAPI OpenID Provider en Janvier 2022

Annexes

FAPI

Authentification mTLS et CIBA

Maximiser la sécurité des standards

Authentification par certificat client (mTLS)

Le client s’authentifie avec un certificat TLS client, via HTTPS mutuel (mTLS).

Keycloak utilise ce certificat pour identifier et authentifier l’utilisateur sans saisie de login/mot de passe.

Avantages de mTLS

- Sécurité forte : authentification à deux facteurs implicite (possession du certificat)
- Sans mot de passe : pas de credential à stocker, pas de phishing
- Adapté aux machines ou environnements contrôlés : terminaux d'entreprise, API inter-applications
- Compatible avec un flow OAuth2 / OIDC standard si bien configuré

Inconvénients / limites de mTLS

Complexité de déploiement :

- Génération et gestion des certificats client (PKI interne)
- Configuration des reverse proxies pour le mTLS

Expérience utilisateur moins fluide :

- Nécessite parfois installation manuelle des certificats sur les terminaux

Pas adapté au grand public (ex. applications mobiles ou utilisateurs anonymes)

Pas natif dans tous les navigateurs/appareils mobiles

Authorization code Flow avec mTLS

Accès initial :

- Le client accède à Keycloak via HTTPS avec authentification mutuelle (mTLS).
- Le certificat client est présenté dès la connexion TLS.

Configuration côté Keycloak :

- Le certificat client est requis ou demandé (HTTPS_CLIENT_AUTH=request|required)
- La validation du certificat est assurée par Keycloak ou par un reverse proxy TLS (ex : NGINX en mTLS).

Authentification de l'utilisateur :

- Keycloak utilise un authenticator SPI X.509 pour mapper le certificat à un utilisateur : mapping via : subjectDN, issuerDN, serialNumber, SAN, etc.
- L'utilisateur est automatiquement connecté, sans saisie de login/mot de passe.

Mise en œuvre :

- Ajouter l'authenticator X.509 dans le flow "browser".
- Configurer un proxy TLS si besoin (pour extraire le certificat).
- Définir les règles de mapping certificat → user dans Keycloak.

CIBA Flow

Client-Initiated Backchannel Authentication (CIBA) est une extension d'OpenID Connect pour l'authentification asynchrone sans redirection navigateur
Idéale pour les appareils sans interface utilisateur (IoT, banques, assistants vocaux…).

Principe :

- Le client n'ouvre pas de navigateur.
- Il initie l'authentification côté serveur (via un backchannel).
- Le fournisseur d'identité notifie l'utilisateur sur un autre canal (ex : mobile, push).
- L'utilisateur accepte/refuse, et le jeton est délivré ensuite.

Terminologie CIBA

Backchannel Authentication Endpoint : Point d'entrée CIBA du serveur d'authentification

login_hint / id_token_hint : Identifie le user à authentifier

Binding Message : Message à afficher à l'utilisateur

Notification Endpoint (optionnel) : endpoint de callback du client pour recevoir le résultat de l'authentification

Poll / Ping / Push : Méthodes de récupération du token après autorisation

Flow CIBA

Flow pour l’obtention du jeton

1) Le client envoie une requête d’authentification backchannel au serveur, avec un *login_hint* ou *id_token_hint*, et optionnellement un *notification_endpoint*.
2) Le serveur notifie le client via le notification_endpoint, quand l’utilisateur a terminé l’authentification, que ce soit un succès ou un échec.
3) Le client utilise cette notification et l’*auth_req_id* qu’il a reçu à l’étape 1 pour récupérer un token auprès du token endpoint.

Flow pour l’authentification

1) Lors d’une requête d’authentication, le serveur d’autorisation notifie un service
2) Le service acquitte puis demande à authentifier l’utilisateur
3) Il informe le serveur du résultat de l’authentification. Si elle réussie, le jeton est disponible pendant x temps

Support dans Keycloak

Activation par client (Enable CIBA)

Configuration générale *Authentication → CIBA Policy*, peut être surchargé par client (pas supporté par l'UI par contre) :

- Mode de livraison du jeton : Seulement ping
- Expiration de la demande authentification (120s par défaut)
- Intervalle de polling
- Comment identifier l'utilisateur : Seulement login_hint

Canal d'authentification implémenté sous forme de SPI

- Soit on implémente son propre canal
- Soit on utilise le SPI prédéfini : ***HTTP Authentication Channel Provider***

Pas de support pour le notification endpoint dynamique

HTTP Authentication Chanel Manager

Doit être activé en temps que SPI :

```
kc.[sh|bat] start --spi-ciba-auth-channel-ciba-http-auth-channel-http-authentication-channel-uri=https://backend.internal.example.com
```

Après une demande d'authentification

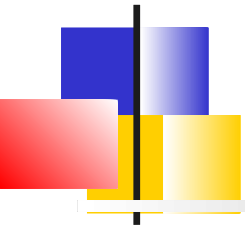
POST Keycloak avec un jeton

Le backend répond avec un 201 et conserve le jeton

Lorsque l'utilisateur est authentifié, le backend ou le client renvoie une requête POST vers :

/realms/[realm]/protocol/openid-connect/ext/ciba/auth/callback avec :

- Le jeton initial
- Le résultat de l'authentification



Annexes

FAPI
Authentification mTLS et CIBA
Maximiser la sécurité des standards

Maximiser la sécurité des standards

Modèle de menaces – RFC 9700

Preuve de possession

FIPS

FAPI

Contexte de RFC 9700

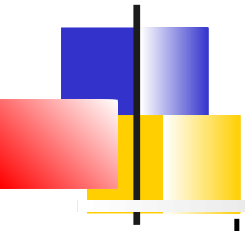
Titre : "OAuth 2.0 Security Best Current Practice"

Publiée en janvier 2025

Actualise les recommandations de sécurité pour OAuth 2.0, en tenant compte des retours d’expérience accumulés depuis la publication initiale d’OAuth

Le RFC 9700 :

- Actualise les modèles de menaces (au-delà des RFC 6749, 6750, 6819),
- Propose de nouvelles contre-mesures face aux attaques connues et émergentes,
- Déprécie certains flux jugés peu sûrs (notamment le Implicit Grant),
- Introduit des exigences de sécurité renforcées, en anticipation de OAuth 2.1.

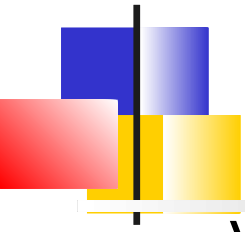


Menaces

Le RFC identifie 5 types d'attaquants (A1–A5), allant de l’attaquant web basique (exécutant un client malveillant) au network attacker capable d’intercepter ou modifier des communications. Il inclut :

- Les attaques via Referer ou historique du navigateur
  Contre mesures : CSP, Referrer-Policy: strict-origin, éviter le stockage des token dans l’url
- Les attaques par mix-up, le code/jeton est renvoyé à un serveur d’autorisation malveillant
  Contre mesure : Reditect URI, claim iss
- L’injection de code d’autorisation ou de tokens d’accès,
- Les attaques sur les flux de communication via navigateur (postMessage...), Exploitation des communications inter-origines dans le navigateur.
- Le PKCE downgrade attack, où l’attaquant tente de forcer une négociation sans PKCE.

Protection des flux basés sur la redirection



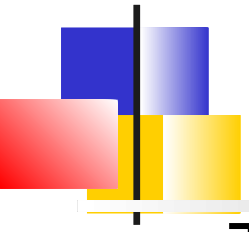
Validation stricte des URI de redirection : utilisation de correspondance exacte, pas de motifs avec jokers, sauf pour les ports dans les apps natives.

Éviter les redirections ouvertes : clients et serveurs doivent empêcher les redirections vers des URI non vérifiés.

Protection contre les attaques CSRF :

- Via state + token CSRF à usage unique,
- Ou via PKCE (Proof Key for Code Exchange),
- Ou via nonce en OpenID Connect (Implémenté nativement dans Keycloak)

Injection et Rejeu de Code d’autorisation

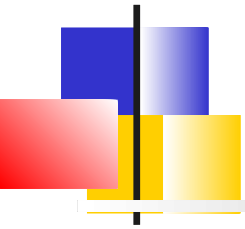


Tous les clients (même confidentiels) doivent implémenter PKCE.

L’usage de méthodes de challenge sûres (S256) est obligatoire.

Les serveurs doivent refuser toute tentative d’échange de code si *code_challenge* est attendu mais absent.

Prévention du rejeu des tokens



Access Tokens : contraindre leur usage à l'émetteur via une preuve de possession :

- mutual TLS (RFC8705),
- ou DPoP (RFC9449).

Refresh Tokens :

- Doivent être tournants (rotation) ou contraints à un émetteur unique.
- Interdiction d'utiliser les mêmes tokens entre clients.

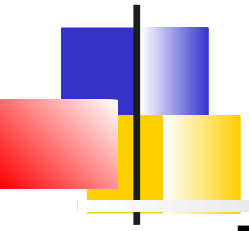
Restrictions des privilèges des tokens

Restreindre les tokens à un périmètre minimum (scopes)

Imposer une audience (aud) claire et vérifiée côté ressource

Utiliser *authorization_details*[1] (RFC9396) si nécessaire.

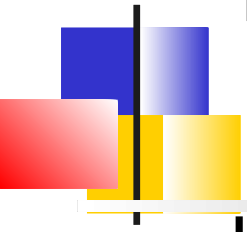
1. Claim utilisé pour spécifier des détails sur les ressources auxquelles le client souhaite accéder, les actions qu'il souhaite effectuer, et d'autres métadonnées pertinentes.



Authentification des clients

- Fortement recommandé d’utiliser une authentification asymétrique :
  - TLS mutuel, ou
  - JWT signé (private_key_jwt).

Métadonnées et configuration automatique



Utiliser Authorization Server Metadata (RFC8414).

Accès des méta-données de configuration du serveur via :
`/.well-known/openid-configuration`
et
`/.well-known/oauth-authorization-server`

Avantages :

- Réduction des erreurs de configuration,
- Détection automatique des capacités de sécurité (PKCE, algorithmes supportés...).

Maximiser la sécurité des standards

Modèle de menaces – RFC 9700

Preuve de possession

FIPS

FAPI

Introduction - DPoP

DPoP – Demonstration of Proof of Possession est définie via la RFC 9449

C’est un mécanisme léger pour démontrer que le client possède une clé privée liée au token

Recommandé par la RFC 9700

Alternative légère à mTLS, adaptée aux apps mobiles, SPA, etc.

Objectif : éviter le rejeu (replay) d’un jeton d’accès intercepté par un attaquant. Le jeton devient inutile sans la clé privée du client.

Mécanismes

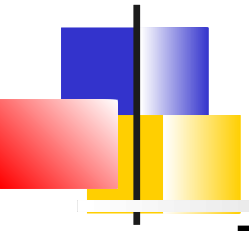
1) Le client génère une paire de clés asymétriques (EC ou RSA)

2) Lors de la requête d'authentification :
   Il envoie sa clé publique encodée dans un header DPoP (JWT signé)
   Le serveur d'autorisation enregistre le thumbprint de cette clé et l'insère dans l'access token sous le claim ***cnf.jkt***

3) Lors de chaque appel API avec le jeton d'accès :
   Le client signe un nouveau JWT DpoP qui contient la clé publique
   Ce JWT est ajouté dans l'en-tête de la requête

4) Le serveur de ressources :
   Vérifie que le JWT DPoP est valide
   Vérifie que le token d'accès a été émis pour cette clé en recalculant le thumbprint avec le même algorithme

Résultat : l'appel ne peut être effectué que par le détenteur de la clé privée.



DpoP et Keycloak

Depuis la v22+, Keycloak supporte DpoP en mode Preview

Ce mécanisme est désactivé par défaut. Pour l'activer :

```
kc.sh start --features=dpop
```

Ensuite pour un client, activer DpoP dans la configuration

Implémentation côté client

Générer une clé privée/publique (ex : EC P-256)

- Construire un JWT DPoP signé avec la clé privée
- Inclure les claims : htu, htm, jti, iat
- Ajouter l'en-tête DPoP à chaque requête vers l'API
- Stocker et réutiliser la clé

Librairies utiles :

- JavaScript jsonwebtoken (npm)
- Java jjwt (Java JWT), Nimbus JOSE+JWT
- Python PyJWT
- C#/.NET System.IdentityModel.Tokens.Jwt

L'en-tête DPoP doit être généré pour chaque appel, car le JWT est horodaté (iat) et lié à l'URL (htu).

Exemple avec *jjwt*

```
KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
 keyGen.initialize(2048);
 KeyPair keyPair = keyGen.generateKeyPair();

 // Construction du JWK (simplifié pour l'exemple)
 Map<String, Object> jwk = new HashMap<>();
 jwk.put("kty", "RSA");
 jwk.put("n", Base64.getUrlEncoder().withoutPadding() .encodeToString(keyPair.getPublic().getEncoded()));
 jwk.put("e", "AQAB");

 // Header personnalisé DPoP
 Map<String, Object> header = new HashMap<>();
 header.put("typ", "dpop+jwt");
 header.put("alg", "RS256");
 header.put("jwk", jwk);

 // Claims DPoP requis
 Map<String, Object> claims = new HashMap<>();
 claims.put("jti", UUID.randomUUID().toString());
 claims.put("htm", "POST");
 claims.put("htu", "https://exemple.com/oauth2/token");
 claims.put("iat", System.currentTimeMillis() / 1000);

 // Génération du JWT signé
 String jwt = Jwts.builder()
     .setHeader(header)
     .setClaims(claims)
     .signWith(keyPair.getPrivate(), SignatureAlgorithm.RS256)
     .compact();
```



Avantages et limites

Avantages :

- Légèreté → pas besoin de certificat client ou infrastructure PKI
- Facilement intégrable dans des applications SPA ou mobiles
- Réduit considérablement le risque de rejeu de jeton

Limites :

- Ne sécurise que les clients honnêtes (pas d'attestation du matériel comme mTLS)
- Nécessite une gestion locale de clés (durée de vie, persistance)
- Dépend du support correct côté ressource server

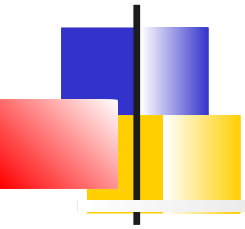
Pour les environnements critiques (FAPI Advanced), DPoP peut être insuffisant → préférer mTLS.



Preuve de possession avec mTLS

- Il est également possible d'effectuer la preuve de possession avec mTLS
  - Il faut configurer Keycloak afin qu'il associe le certificat client au jeton.
    Advanced – Advanced Settings - OAuth 2.0 Mutual TLS Certificate Bound Access Tokens Enabled
  - Keyclak renvoie alors un jeton avec le claim cnt contenant l'empreinte du certificat (thumbprint)
  - Lorsque le serveur d'api reçoit le jeton un peut vérifier le certificat client et le thumbprint

Maximiser la sécurité des standards



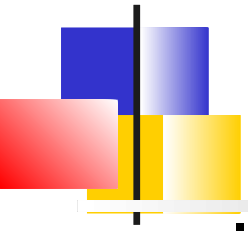
Modèle de menaces – RFC 9700
Preuve de possession
FIPS
FAPI

Introduction

FIPS (Federal Information Processing Standards) est un ensemble de standards de sécurité informatique définis par le gouvernement des États-Unis.

Le plus connu est FIPS 140-2 / 140-3, qui spécifie les exigences pour les modules cryptographiques (logiciels ou matériels).

Objectif : Assurer que les algorithmes cryptographiques utilisés sont robustes, éprouvés et conformes



Implications

- Interdiction des algorithmes faibles ou obsolètes (ex : MD5, SHA-1, RSA 1024)
- Imposition de l’usage de :
  - RSA ≥ 2048 bits
  - AES-256 / GCM
  - SHA-256, SHA-384, SHA-512
  - Signatures PSS (RSA-PSS)
- Nécessité d’utiliser une JVM et des fournisseurs cryptographiques compatibles FIPS

Conditions pour keycloak

Le serveur doit tourner sur un système FIPS-activé (ex. RHEL/Fedora, ou Ubuntu FIPS via Ubuntu Pro)

Keycloak utilise un Java FIPS-compliant via OpenJDK.

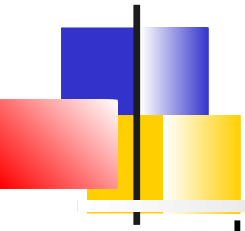
BouncyCastle FIPS : la version par défaut de BouncyCastle utilisé pour la cryptography n'est pas FIPS compliant.

=> Il faut installer les librairies adéquates dans ***KEYCLOAK_HOME/providers*** :

- *bc-fips-2.0.0.jar*
- *bctls-fips-2.0.19.jar*
- *bcpkix-fips-2.0.7.jar*

Le keystore doit être compatible FIPS :

- PKCS12 : fonctionne en mode non-approved
- BCFKS : requis pour le mode FIPS strict



Démarrage

Une fois les conditions remplies, on peut démarrer keycloak en mode ***fips*** ou en ***strict fips***.

Modifier le fichier *java.security* afin que la JVM ne tente pas un algorithme interdit. Ex :

```
security.provider.1=org.bouncycastle.jcajce.provider.BouncyCastleFipsProvider
security.provider.2=org.bouncycastle.jsse.provider.BouncyCastleJsseProvider
security.overridePropertiesFile=true

export JAVA_OPTS_APPEND="-Djava.security.properties=/opt/keycloak/conf/fips-java.security"
```

Activer la features fips et éventuellement positionner le mode script

```
bin/kc.[sh|bat] start --features=fips --fips-mode=strict --hostname=localhost \
     --https-key-store-password=passwordpassword \
     --log-level=INFO,org.keycloak.common.crypto:TRACE,org.keycloak.crypto:TRACE
```

Impacts de FIPS sur Keycloak et les applications

Avantages

- Conformité réglementaire (USA, institutions financières ou gouvernementales)
- Usage exclusif d'algorithmes robustes
- Accroît la sécurité globale de votre chaîne d'authentification
- Souvent exigé pour FAPI Advanced, OpenBanking, ou projets certifiables

Limitations / Effets de bord

- Algorithmes : Algos faibles désactivés (ex. SHA-1, RSA-1024)
- Signatures : Certaines signatures interdites (ex. none)
- Tokens JWT : JWT doivent être signés avec algo FIPS-compatibles
- Clients existants : Incompatibles si ils utilisent un algo non autorisé
- Intégrations tierces Doivent être compatibles FIPS

Maximiser la sécurité des standards

Modèle de menaces – RFC 9700
Preuve de possession
FIPS
FAPI

Introduction

FAPI (Financial-grade API) est un ensemble de spécifications de sécurité renforcées, conçues pour protéger les échanges OAuth2 / OIDC dans des contextes hautement sensibles, en particulier dans :

- la finance (Open Banking UK, Berlin Group, PSD2)
- la santé
- les données personnelles critiques

Apports :

- Des profils renforcés d'OAuth2 / OIDC
- Des exigences sur les clients, tokens, signatures, preuves de possession
- Une standardisation du niveau de sécurité pour des appels API critiques

Profils FAPI

FAPI 2.0 combine plusieurs recommandations (RFC 9207, RFC 8705, RFC 8707, DPoP...) et impose un ensemble de contraintes, regroupées dans deux profils principaux :

- ***Baseline***
  Renforce OAuth/OIDC standard
  Clients web ou natifs sécurisés
- ***Advanced***
  Sécurité maximale (certification)
  Services financiers, eIDAS, etc.

Principales exigences FAPI

PKCE obligatoire : Même pour clients confidentiels

Preuve de possession (DPoP ou mTLS) : Obligatoire pour les tokens

Signed Authorization Response (JARM) : Réponse d'autorisation encapsulée dans un jeton JWT signée (*response_mode=jwt* dans la requête initiale)

Claims précises (*aud, nonce, acr, client_id*) : Présence vérifiée et horodatée

Flows interdits : *implicit, password, client_credentials* (selon profil)

ID Token signé obligatoirement : Pas de token non signé ou avec alg: none

Refresh tokens bound (DpoP/mTLS) : Refresh token lié à une clé, même pour clients publics

FAPI et Keycloak

Keycloak est conforme FAPI1 depuis la version 15.0.2, il n'est pas officiellement certifié FAPI-2.0

Via les ***client policies***, Keycloak permet d'imposer des exigences FAPI (Baseline, Advanced, etc.) sans avoir à tout configurer manuellement pour chaque client.

Ces politiques contrôlent dynamiquement les conditions d'enregistrement, d'authentification et de traitement des clients en fonction de règles (policies) et de profils (profiles).



Mise en œuvre

Admin → Realm Settings → Client Policies

Créer une politique et définir les clients qui y seront associés (tous, par rôle, par client_id, par client_scope)

Appliquer un profil client

- En utilisant les profils définis
- Ou un profil custom qui spécifie chaque feature requises (*pkce-enforcer, dpop-enforcer, mtls-holder-of-key-enforcer*, ...)

Tout client qui correspond à la policy définie sera soumis automatiquement aux règles FAPI imposées.